Mastering Java Multithreading & Concurrency – Internals, Architecture, and Enterprise Usage Guide

---

Author: Senior Java Architect

Target Audience: Java developers with 3+ years of experience aiming for senior/architect-level roles

Prerequisites: Core Java, Spring Boot, REST APIs, Microservices basics

---

Table of Contents

1. Introduction to Multithreading
2. Process vs Thread
3. Java Thread Lifecycle
4. Creating Threads in Java
5. Synchronization
6. Inter-Thread Communication
7. Locks Framework
8. Java Concurrent Collections
9. Executor Framework
10. CompletableFuture (Java 8+)
11. Fork/Join Framework
12. Parallel Streams
13. Deadlocks
14. Thread Safety Design Patterns
15. Multithreading in Spring Boot Applications
16. Performance Optimization
17. Common Production Issues
18. Interview Questions (60+)
19. Coding Problems (20 Exercises)
20. Best Practices

Chapter 1: Introduction to Multithreading

1.1 What is Multithreading?

Multithreading is the ability of a CPU (or a single process) to execute multiple threads concurrently. Each thread is an independent path of execution within a process. In Java, every application starts with at least one thread — the **main thread** — and can create additional threads to perform work in parallel.

```
public class Main {
    public static void main(String[] args) {
        // This is the "main" thread
        System.out.println("Current thread: " +
Thread.currentThread().getName());
        // Output: Current thread: main
    }
}
```

Concurrency vs Parallelism

Concept	Description	Example
Concurrency	Multiple tasks make progress in overlapping time periods (interleaving on a single core)	A single barista handling 3 coffee orders by switching between them
Parallelism	Multiple tasks execute at the exact same instant (multiple cores)	Three baristas each making one coffee simultaneously

Java's threading model supports **both** through the JVM's thread scheduler and the OS kernel.

How JVM Maps Java Threads to OS Threads

Since Java 1.2, the HotSpot JVM uses a **1:1 threading model** — every `java.lang.Thread` maps directly to a native OS thread (POSIX thread on Linux, Win32 thread on Windows).

```
Java World                          OS World
+------------------+               +-------------------+
|  Thread t1       | ---- JNI ---> |  pthread_create()  |  Linux
|  Thread t2       | ---- JNI ---> |  CreateThread()    |  Windows
|  Thread t3       | ---- JNI ---> |  pthread_create()  |  Linux
+------------------+               +-------------------+
```

```
                                        |
                                   OS Scheduler
                                   (CFS on Linux)
                                        |
                                   +----+----+
                                   | CPU     |
                                   | Cores   |
                                   +---------+

Thread.start() -> native start0() -> JVM_StartThread() -> OS thread created
```

What this means in practice:

- Java thread scheduling is delegated to the **OS scheduler** (Completely Fair Scheduler on Linux)
- Java's `Thread.setPriority()` maps to OS priority levels (but mapping varies by OS)
- Thread context switching is handled at the kernel level
- Each Java thread consumes a real OS thread, which means you are bounded by OS limits (~30,000 threads per process on most systems)

CPU Architecture and its Impact on Multithreading

Understanding CPU hardware is critical for writing high-performance concurrent code:

```
Modern CPU Architecture (8-core, 16-thread CPU):
+-------------------------------------------------------+
|  CPU Package                                           |
|                                                        |
|  Core 0          Core 1          Core 2    ...  Core 7|
|  +----------+   +----------+   +----------+         |
|  | HT-0 HT-1|   | HT-0 HT-1|   | HT-0 HT-1|         |
|  | L1 Cache |   | L1 Cache |   | L1 Cache |         |
|  | (64KB)   |   | (64KB)   |   | (64KB)   |         |
|  +----+-----+   +----+-----+   +----+-----+         |
|       |              |              |                 |
|  +----+--------------+--------------+----+           |
|  |           L2 Cache (256KB per core)    |           |
|  +--------------------+------------------+           |
|                       |                               |
|  +--------------------+-------------------------+   |
|  |          L3 Cache (Shared, 16-32MB)            |   |
|  +------------------------------------------------+   |
|                       |                               |
|  +--------------------+-------------------------+   |
|  |               Main Memory (RAM)                |   |
|  +------------------------------------------------+   |
+-------------------------------------------------------+
```

```
Key Facts for Java Developers:
-----------------------------
- L1 cache access: ~1 ns     (fastest, per-core)
- L2 cache access: ~4 ns
- L3 cache access: ~12 ns    (shared across cores)
- Main memory:     ~100 ns   (100x slower than L1!)
- Cache line size:  64 bytes  (unit of data transfer between cache and memory)

Why this matters:
- volatile forces reads/writes through to main memory, bypassing cache
- false sharing occurs when two threads modify data on the same 64-byte cache
line
- Thread-local data is fast because it stays in L1 cache
- Hyper-threading shares a core's resources — two HTs on one core are NOT the
same as two cores
```

Amdahl's Law — The Ceiling of Parallelism

Amdahl's Law defines the theoretical maximum speedup from parallelism. No matter how many cores you add, the sequential portion of your code limits the total speedup.

```
Formula:
              1
Speedup = -----------------
          S + (1 - S) / N

Where:
  S = fraction of code that is sequential (cannot be parallelized)
  N = number of processor cores

Example Calculations:
---------------------
If 10% of your code is sequential (S = 0.1):
  4 cores:   1 / (0.1 + 0.9/4)   = 1 / 0.325  = 3.08x speedup
  8 cores:   1 / (0.1 + 0.9/8)   = 1 / 0.2125 = 4.71x speedup
  16 cores:  1 / (0.1 + 0.9/16)  = 1 / 0.156  = 6.40x speedup
  ∞ cores:   1 / 0.1             = 10x speedup  <- MAXIMUM POSSIBLE!

If 50% of your code is sequential (S = 0.5):
  4 cores:   1 / (0.5 + 0.5/4)   = 1 / 0.625  = 1.6x speedup
  ∞ cores:   1 / 0.5             = 2x speedup   <- MAXIMUM!
```

> ***Production Insight:*** *In backend services, the "sequential" portion is often the time spent on shared locks (synchronized blocks). Reducing lock contention has a bigger impact on scalability than adding more cores. This is why* `ConcurrentHashMap` *(fine-grained locking) is so much faster than* `Hashtable` *(single global lock).*

Green Threads vs OS Threads vs Virtual Threads

Feature	Green Threads (JDK 1.0)	OS/Platform Threads (JDK 1.2+)	Virtual Threads (JDK 21+)
Managed by	JVM	OS kernel	JVM (mounted on carrier threads)
Memory per thread	~KB	~1 MB (stack)	~KB (resizable stack)
Max threads per JVM	Thousands	~10,000–30,000	**Millions**
Scheduling	JVM cooperative	OS preemptive	JVM (continuation-based)
True parallelism	No (single OS thread)	Yes	Yes (via carrier threads)
Blocking cost	Blocks all threads	Blocks one OS thread	Unmounts from carrier, carrier reused
Use case	Deprecated	Current standard	Future standard for IO-bound

```java
// Platform thread (current standard):
Thread platformThread = new Thread(() -> doWork());
platformThread.start(); // Creates OS thread (~1MB memory)

// Virtual thread (Java 21+):
Thread virtualThread = Thread.ofVirtual().start(() -> doWork());
// Uses ~few KB, millions can coexist

// Virtual thread executor (Java 21+):
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000_000; i++) {
        executor.submit(() -> {
            // Each task gets its own virtual thread
            // Blocking IO automatically unmounts the virtual thread
            return httpClient.send(request,
```

```
HttpResponse.BodyHandlers.ofString());
        });
    }
}
```

> ***Senior Engineer Perspective:*** *Virtual threads (Project Loom) are a game-changer for IO-bound workloads. They eliminate the need for reactive programming frameworks (WebFlux, RxJava) in many scenarios by making blocking calls cheap. However, they do NOT help with CPU-bound tasks — for those, the number of platform threads (= CPU cores) remains the bottleneck.*

1.2 Why Multithreading is Needed

In Enterprise Backend Systems

Problem	Single-Threaded	Multi-Threaded
Handling 10,000 concurrent API requests	Requests queued, >10s response time	Multiple threads serve requests simultaneously, <200ms
Calling 5 microservices for one response	Sequential: 5x200ms = 1000ms	Parallel: max(200ms) = 200ms
Processing 1M database records	~30 minutes	~5 minutes (parallel batches)
Real-time event streaming	Backpressure, missed events	Dedicated consumer threads handle throughput

Key Benefits

1. **Responsiveness** – UI and API endpoints remain responsive while background work runs
2. **Resource Utilization** – Uses idle CPU cores instead of wasting them
3. **Throughput** – More work completed in the same time
4. **Scalability** – Handle more concurrent users/requests
5. **Asynchronous Processing** – Fire-and-forget for non-critical tasks (emails, logging, analytics)

Drawbacks

1. **Complexity** – Harder to reason about, debug, and test
2. **Race Conditions** – Shared mutable state leads to data corruption
3. **Deadlocks** – Threads waiting for each other indefinitely
4. **Memory Overhead** – Each thread consumes ~512KB-1MB stack memory

5. **Context Switching** – CPU overhead switching between threads

---

1.3 Real-World Production Use Cases

Use Case 1: High-Traffic Backend Service

```
// Tomcat handles each HTTP request in a separate thread (thread-per-request
model)
// Default: 200 threads in Spring Boot's embedded Tomcat
@RestController
public class OrderController {
    @GetMapping("/api/orders/{id}")
    public ResponseEntity<OrderDTO> getOrder(@PathVariable Long id) {
        // Each request runs in its own Tomcat worker thread
        // Thread: http-nio-8080-exec-1, http-nio-8080-exec-2, etc.
        return ResponseEntity.ok(orderService.findById(id));
    }
}
```

Use Case 2: Parallel API Calls in Microservices

```
@Service
public class DashboardService {

    public DashboardDTO buildDashboard(Long userId) {
        // Sequential: 200 + 150 + 100 = 450ms
        // Parallel:   max(200, 150, 100) = 200ms  -> 56% faster!

        CompletableFuture<UserProfile> userFuture =
            CompletableFuture.supplyAsync(() ->
userService.getProfile(userId));
        CompletableFuture<List<Order>> ordersFuture =
            CompletableFuture.supplyAsync(() ->
orderService.getRecent(userId));
        CompletableFuture<WalletBalance> walletFuture =
            CompletableFuture.supplyAsync(() ->
walletService.getBalance(userId));

        CompletableFuture.allOf(userFuture, ordersFuture, walletFuture).join();

        return new DashboardDTO(
```

```
            userFuture.join(), ordersFuture.join(), walletFuture.join()
        );
    }
}
```

Use Case 3: Batch Processing

```
@Service
public class ReportGenerationService {

    @Async("reportExecutor")
    public void generateMonthlyReport(YearMonth month) {
        // Runs in a background thread, doesn't block the API call
        List<Transaction> txns = transactionRepo.findByMonth(month);
        byte[] pdf = pdfGenerator.create(txns);
        storageService.upload("reports/" + month + ".pdf", pdf);
        emailService.notify("Report ready for " + month);
    }
}
```

Use Case 4: Kafka Consumer Processing

```
@KafkaListener(topics = "order-events", concurrency = "5")
public void consume(OrderEvent event) {
    // 5 consumer threads process messages in parallel
    // Each partition is consumed by exactly one thread
    orderProcessor.process(event);
}
```

Chapter 2: Process vs Thread

2.1 What is a Process?

A **process** is an independent program in execution with its own **memory space** (heap, stack, code, data segments). The operating system allocates separate resources to each process.

```
+---------------------- Process A ----------------------+
|  +----------+  +----------+  +----------+            |
|  |   Code   |  |   Data   |  |   Heap   |            |
|  | Segment  |  | Segment  |  | (Dynamic)|            |
|  +----------+  +----------+  +----------+            |
|  +----------+                                         |
|  |  Stack   |  <- One stack per thread                |
|  +----------+                                         |
|  PID: 1234  |  Memory: 256MB  |  Own file handles     |
+-------------------------------------------------------+

+---------------------- Process B ----------------------+
|  Completely separate memory space                     |
|  Cannot access Process A's memory directly            |
|  PID: 5678  |  Memory: 128MB  |  Own file handles     |
+-------------------------------------------------------+
```

2.2 What is a Thread?

A **thread** is a lightweight unit of execution **within a process**. Threads within the same process **share the heap memory** but have their own **stack**.

```
+----------------------- Process (JVM) ----------------------+
|                                                             |
|   Shared Memory                                             |
|   +----------------------------------------+                |
|   |  Heap (Objects, Instance Variables)     |                |
|   |  Method Area (Class data, static vars)  |                |
|   |  Code Segment                           |                |
|   +----------------------------------------+                |
|                                                             |
|   Thread 1 (main)    Thread 2          Thread 3             |
|   +----------+      +----------+      +----------+         |
|   |  Stack   |      |  Stack   |      |  Stack   |         |
```

```
|   | (local   |      | (local   |      | (local   |         |
|   |  vars)   |      |  vars)   |      |  vars)   |         |
|   |  PC      |      |  PC      |      |  PC      |         |
|   +----------+      +----------+      +----------+         |
|                                                             |
+-------------------------------------------------------------+
```

2.3 Detailed Comparison

Feature	Process	Thread
Definition	Independent program in execution	Lightweight execution unit within a process
Memory	Own memory space (isolated)	Shares heap with other threads; own stack
Creation Cost	Heavy (OS allocates memory, resources)	Lightweight (~512KB-1MB stack)
Communication	IPC (sockets, pipes, shared files)	Direct via shared heap memory
Context Switch	Expensive (full memory context swap)	Cheaper (shared address space)
Crash Impact	One process crash doesn't affect others	One thread crash can kill the entire process
Isolation	Full isolation	No memory isolation between threads
Example	Two separate JVM instances	Multiple threads inside one Spring Boot app

Context Switching

```
Context Switch Between PROCESSES (Expensive):
---------------------------------------------
1. Save entire CPU state (registers, PC, stack pointer)
2. Save memory mappings (page table)
3. Flush TLB (Translation Lookaside Buffer)
4. Load new process memory mappings
5. Restore new process CPU state
-> Cost: ~1-10 microseconds

Context Switch Between THREADS (Cheaper):
-----------------------------------------
1. Save CPU state (registers, PC, stack pointer)
2. Load new thread's CPU state
(No memory mapping change - same process!)
-> Cost: ~0.1-1 microseconds
```

Backend Scenario

```
// PROCESS-level parallelism: Microservices are separate processes
// UserService (PID 1001) — separate JVM
// OrderService (PID 1002) — separate JVM
// PaymentService (PID 1003) — separate JVM

// THREAD-level parallelism: Within each service
// UserService has 200 Tomcat threads handling HTTP requests concurrently
// All 200 threads share the same heap (Spring beans, caches, etc.)
```

JVM Per-Thread Memory Layout (Deep Dive)

Each Java thread gets its own dedicated memory regions. Understanding this is essential for diagnosing `StackOverflowError` and tuning memory:

```
Per-Thread Memory (Private to each thread):
+-----------------------------------------------------+
|  Thread Stack (default: -Xss512k to -Xss1m)         |
|  +-------------------------------------------------+|
|  | Stack Frame: main()                              ||
|  |   - Local variables: int x = 5                   ||
|  |   - Operand stack: [5, 10]                       ||
|  |   - Return address                               ||
|  +-------------------------------------------------+|
|  | Stack Frame: processOrder(Order o)               ||
|  |   - Local variables: Order ref, String s         ||
|  |   - Operand stack                                ||
|  +-------------------------------------------------+|
|  | Stack Frame: validatePayment(Payment p)          ||
|  |   - (each method call adds a frame)              ||
|  +-------------------------------------------------+|
|                                                      |
|  Program Counter (PC Register)                       |
|  - Address of current bytecode instruction           |
|  - Undefined for native methods                      |
|                                                      |
|  Native Method Stack                                 |
|  - For JNI calls (C/C++ code)                        |
|  - Separate from Java stack                          |
+-----------------------------------------------------+

Memory Calculation for 200 Tomcat threads:
```

```
  Default stack:  512KB x 200 = 100 MB just for thread stacks!
  With 1MB:       1MB x 200   = 200 MB

  Production tip: Use -Xss512k for web servers (most methods have
  shallow call stacks). Use -Xss1m or more for deep recursion.
```

Context Switching: Hidden Performance Killer

Context switching is more expensive than the raw time cost suggests because of **cache pollution**:

```
What actually happens during a thread context switch:
-----------------------------------------------------

Step 1: Save State (~100 ns)
  - Save all CPU registers (16 general-purpose + flags + FPU)
  - Save stack pointer and program counter
  - Save thread-local storage pointer

Step 2: Scheduler Decision (~200 ns)
  - OS scheduler picks next thread (CFS on Linux)
  - Update scheduling statistics

Step 3: Restore State (~100 ns)
  - Load new thread's registers
  - Load new thread's stack pointer

Step 4: CACHE WARM-UP (the real cost!! ~1,000-10,000 ns)
  - New thread accesses different data
  - L1 cache misses (data for old thread, not new)
  - L2 cache misses (pipeline stalls)
  - TLB misses (for process switches)

Real-world impact:
  Measured switch cost:  ~1-5 us (direct overhead)
  Cache pollution cost:  ~5-50 us (indirect, measured by slowdown)

  At 10,000 context switches/second:
    Direct cost:   10K x 5us  = 50ms  (5% of one CPU)
    Indirect cost: 10K x 25us = 250ms (25% of one CPU!) <- the REAL problem

How to check context switching on Linux:
  $ vmstat 1          # Look at 'cs' column
  $ pidstat -w -p PID # Per-process context switches
```

> ***Production Insight:*** *If your JVM application shows high context switching (* `> 50,000/sec` *), you likely have too many threads competing for too few cores. The fix is NOT adding more threads — it's reducing thread count, using non-blocking IO, or switching to virtual threads.*

When to Use Processes vs Threads (Senior Decision Framework)

```
Decision Framework:
------------------

Use PROCESSES (separate JVMs) when:
  [Y] Need fault isolation (crash in one service shouldn't kill others)
  [Y] Different scaling requirements (user service: 4 instances, payment: 2)
  [Y] Different technology stacks (Java + Python + Go)
  [Y] Independent deployment lifecycle
  -> This is why microservices are separate processes

Use THREADS (within one JVM) when:
  [Y] Need to share memory (caches, connection pools, state)
  [Y] Communication must be ultra-fast (no serialization, no network)
  [Y] Tasks are tightly coupled and need consistent view of data
  [Y] Low-latency requirements (IPC overhead too high)
  -> This is why web servers use thread pools, not process pools
```

Chapter 3: Java Thread Lifecycle

3.1 Thread States

Java defines **6 thread states** in `java.lang.Thread.State`:

```
                    +---------+
         start()    |         |
    +---------------->|RUNNABLE |<----------------------------+
    |               |         |                             |
+---+---+           +----+----+                             |
|  NEW  |                |                                  |
+-------+                |                                  |
                    +----+----+                              |
                    | enters  |                              |
                    |sync block|                             |
                    |(locked) |                              |
                    +----+----+                              |
                         v                                   |
                    +---------+     lock acquired           |
                    | BLOCKED |-----------------------------+
                    +---------+

    From RUNNABLE:
                    +-------------+   notify()/
     wait()         |   WAITING   |   notifyAll()
    ------------->   |             |------------> RUNNABLE
                    +-------------+

     sleep(ms)      +-------------+   timeout/
     wait(ms)       |TIMED_WAITING|   notify()
    ------------->   |             |------------> RUNNABLE
                    +-------------+

     run() ends     +-------------+
    ------------->   | TERMINATED  |
                    +-------------+
```

3.2 State Descriptions

NEW

Thread object created but `start()` not yet called.

```
Thread t = new Thread(() -> System.out.println("Hello"));
System.out.println(t.getState()); // NEW
```

RUNNABLE

Thread is executing or ready to execute (in the OS run queue). Java does not distinguish between "running" and "ready."

```
Thread t = new Thread(() -> {
    while (true) { /* busy */ }
});
t.start();
System.out.println(t.getState()); // RUNNABLE
```

BLOCKED

Thread is waiting to acquire a **monitor lock** (intrinsic lock) held by another thread.

```
Object lock = new Object();

Thread t1 = new Thread(() -> {
    synchronized (lock) {
        try { Thread.sleep(10000); } catch (InterruptedException e) {}
    }
});

Thread t2 = new Thread(() -> {
    synchronized (lock) { // t2 BLOCKED here waiting for lock held by t1
        System.out.println("Got lock!");
    }
});

t1.start();
Thread.sleep(100);
t2.start();
Thread.sleep(100);
System.out.println(t2.getState()); // BLOCKED
```

WAITING

Thread is waiting indefinitely for another thread to perform a specific action. Caused by: `Object.wait()`, `Thread.join()`, `LockSupport.park()`

```
Object lock = new Object();
Thread t = new Thread(() -> {
    synchronized (lock) {
        try { lock.wait(); } catch (InterruptedException e) {} // WAITING until
notify()
    }
});
t.start();
Thread.sleep(100);
System.out.println(t.getState()); // WAITING
```

TIMED_WAITING

Thread is waiting for a specified period of time. Caused by: `Thread.sleep(ms)`, `Object.wait(ms)`, `Thread.join(ms)`, `LockSupport.parkNanos()`

```
Thread t = new Thread(() -> {
    try { Thread.sleep(5000); } catch (InterruptedException e) {}
});
t.start();
Thread.sleep(100);
System.out.println(t.getState()); // TIMED_WAITING
```

TERMINATED

Thread has completed execution (either normally or via exception).

```
Thread t = new Thread(() -> System.out.println("Done"));
t.start();
t.join(); // Wait for completion
System.out.println(t.getState()); // TERMINATED
```

JVM Internal: How Thread State Transitions Work

At the JVM level, thread state transitions are managed by the **ObjectMonitor** (for synchronized blocks) and **ParkEvent/Parker** (for j.u.c locks):

```
JVM Internal Thread State Machine:
----------------------------------

Thread.start() -> JVM calls os::create_thread()
  -> OS thread created -> state set to RUNNABLE
  -> OS scheduler picks it up -> run() method executes

Entering synchronized block:
  -> JVM checks ObjectMonitor owner
  -> If free: set owner = current thread, enter block (RUNNABLE)
  -> If owned by other:
      1. Spin briefly (adaptive spinning, ~5000 iterations)
      2. If still locked -> enqueue in ObjectMonitor's _EntryList
      3. Thread state -> BLOCKED
      4. OS parks the thread (removes from scheduler)

Object.wait():
  -> Thread MUST own the monitor (otherwise IllegalMonitorStateException)
  -> Release monitor (set owner = null)
  -> Move thread to ObjectMonitor's _WaitSet
  -> Thread state -> WAITING
  -> OS parks the thread

Object.notify():
  -> Pick ONE thread from _WaitSet
  -> Move it to _EntryList (must re-acquire monitor)
  -> Woken thread state -> BLOCKED (waiting to re-acquire monitor)
  -> When monitor acquired -> RUNNABLE

LockSupport.park() / unpark():
  -> Direct OS-level thread parking
  -> Used internally by ReentrantLock, CountDownLatch, etc.
  -> Thread state -> WAITING or TIMED_WAITING
```

Reading Thread Dumps: State-Based Diagnostics

Thread dumps (`jstack <PID>`) are the #1 tool for diagnosing production threading issues. Here's how to read each state:

```
How to take a thread dump:
--------------------------
$ jstack <PID>                    # Basic dump
```

```
$ jstack -l <PID>                 # With lock info
$ jcmd <PID> Thread.print        # Alternative (recommended)
$ kill -3 <PID>                   # Sends dump to stdout (Linux)

What each state means in a thread dump:
---------------------------------------

1. RUNNABLE — "java.lang.Thread.State: RUNNABLE"
   Normal: Thread is doing work (CPU or IO)
   Problem: If many threads RUNNABLE but CPU is low -> likely doing blocking IO
            that Java reports as RUNNABLE (e.g., socket read, file IO)

   "http-nio-8080-exec-1" #25 daemon prio=5 os_prio=0 tid=0x00007f...
      java.lang.Thread.State: RUNNABLE
        at java.net.SocketInputStream.read(SocketInputStream.java:152)
        at com.mysql.cj.protocol.ReadAheadInputStream.read(...)
   ^ This is actually BLOCKED on network IO, but Java shows it as RUNNABLE

2. BLOCKED — "java.lang.Thread.State: BLOCKED (on object monitor)"
   Problem: Thread is waiting for a synchronized lock
   Action:  Find who holds the lock and why they are slow

   "worker-3" #18 prio=5 os_prio=0 tid=0x00007f...
      java.lang.Thread.State: BLOCKED (on object monitor)
        at com.app.service.OrderService.processOrder(OrderService.java:45)
        - waiting to lock <0x00000000c0035a68> (a java.util.HashMap)
        - locked by "worker-1" #16
   ^ worker-3 is blocked because worker-1 holds the lock on HashMap

3. WAITING — "java.lang.Thread.State: WAITING (parking)"
   Normal: Thread is waiting for work (e.g., thread pool idle)
   Problem: If many threads WAITING on same condition -> possible deadlock

   "pool-1-thread-3" #20 prio=5 os_prio=0 tid=0x00007f...
      java.lang.Thread.State: WAITING (parking)
        at sun.misc.Unsafe.park(Native Method)
        at java.util.concurrent.locks.LockSupport.park(LockSupport.java:175)
        at java.util.concurrent.LinkedBlockingQueue.take(...)
   ^ Normal: thread pool worker waiting for next task

4. TIMED_WAITING — "java.lang.Thread.State: TIMED_WAITING (sleeping)"
   Normal: Thread sleeping or waiting with timeout
   Problem: If business threads are sleeping -> inefficient design

Quick Diagnosis Cheat Sheet:
----------------------------
Many BLOCKED threads    -> Lock contention -> reduce sync scope or use
ConcurrentHashMap
```

```
Many WAITING on IO      -> Connection pool exhaustion -> increase pool or add
timeouts
All threads RUNNABLE    -> CPU-bound -> check for infinite loops or heavy
computation
Deadlock detected       -> Fix lock ordering or use tryLock with timeout
```

> ***Senior Engineer Tip:*** *Take 3 thread dumps 5–10 seconds apart. Compare them. Threads that are stuck in the same state across all 3 dumps are the problem threads. Threads that change state are just doing normal work.*

Chapter 4: Creating Threads in Java

4.1 Extending Thread Class

What

Create a new class that extends `java.lang.Thread` and override the `run()` method.

Why / When

Simplest approach but **tightly couples** the task with the thread mechanism. Use only for simple, one-off scenarios. **Avoid in production** — violates single responsibility principle and wastes the single inheritance slot.

How It Works

```
public class DataFetcherThread extends Thread {

    private final String url;

    public DataFetcherThread(String url) {
        this.url = url;
        setName("DataFetcher-" + url.hashCode());
    }

    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " fetching: " +
url);
        // Simulate API call
        try { Thread.sleep(1000); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println(Thread.currentThread().getName() + " done");
    }
}

// Usage
DataFetcherThread t1 = new DataFetcherThread("https://api.example.com/users");
DataFetcherThread t2 = new DataFetcherThread("https://api.example.com/orders");
t1.start(); // Creates new OS thread and calls run()
t2.start();
```

```
t1.join();  // Wait for t1 to finish
t2.join();  // Wait for t2 to finish
```

Under the hood: `Thread.start()` -> calls native `start0()` -> JVM creates a new OS thread -> OS thread invokes the `run()` method.

4.2 Implementing Runnable

What

Implement the `Runnable` functional interface and pass it to a `Thread` constructor.

Why / When

Preferred over extending Thread. Separates the task from the thread. Class can extend another class. Task can be reused, submitted to thread pools, or used with lambda expressions.

How It Works

```java
// Using lambda (Java 8+)
Runnable fetchTask = () -> {
    System.out.println(Thread.currentThread().getName() + " executing task");
    // Business logic here
};

Thread t = new Thread(fetchTask, "Worker-1");
t.start();

// Enterprise example: Background cleanup
@Service
public class CacheCleanupService implements Runnable {

    @Autowired
    private CacheManager cacheManager;

    @Override
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            cacheManager.evictExpired();
            try {
                Thread.sleep(60000); // Every 60 seconds
            } catch (InterruptedException e) {
```

```
                Thread.currentThread().interrupt();
                break;
            }
        }
    }
}
```

4.3 Callable + Future

What

`Callable<V>` is like `Runnable` but **returns a result** and **can throw checked exceptions**. `Future<V>` represents the result of an asynchronous computation.

Why / When

When you need a **return value** from a thread or need to handle **exceptions** thrown during execution. Used with `ExecutorService`.

How It Works

```java
// Callable returns a value
Callable<Double> priceCalculator = () -> {
    Thread.sleep(2000); // Simulate heavy computation
    return 99.95;
};

ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Double> future = executor.submit(priceCalculator);

// Do other work while computation runs...
System.out.println("Doing other work...");

// Get result (blocks if not ready yet)
Double price = future.get();           // Blocks until done
// or with timeout:
Double price2 = future.get(5, TimeUnit.SECONDS); // Throws TimeoutException

System.out.println("Price: " + price);

executor.shutdown();
```

Future API

```
boolean isDone = future.isDone();          // Check if complete
boolean cancelled = future.cancel(true);   // Cancel with interrupt
boolean isCancelled = future.isCancelled();
V result = future.get();                   // Block until result
V result = future.get(timeout, unit);      // Block with timeout
```

Limitations of Future

- `get()` is **blocking** — defeats the purpose of async
- No way to **chain** or **compose** futures
- No way to **combine** multiple futures
- No manual completion
- No exception handling callbacks

-> **Solution: CompletableFuture** (Chapter 10)

4.4 ExecutorService

What

A higher-level abstraction for managing threads through **thread pools**. Part of the `java.util.concurrent` package.

Why / When

Always use in production. Creating raw threads is wasteful (thread creation ~1ms, ~1MB memory each). ExecutorService reuses threads, manages lifecycle, bounds concurrency.

How It Works

```
// Create thread pool
ExecutorService executor = Executors.newFixedThreadPool(10);

// Submit tasks
Future<String> result = executor.submit(() -> {
    return callExternalAPI();
});
```

```
// Submit Runnable (no return value)
executor.execute(() -> {
    sendEmailNotification(user);
});

// Shutdown
executor.shutdown();                        // Graceful: finish queued tasks
executor.awaitTermination(30, TimeUnit.SECONDS); // Wait for completion
// executor.shutdownNow();                 // Forceful: interrupt running tasks
```

Production Example

```
@Configuration
public class ThreadPoolConfig {

    @Bean("apiCallExecutor")
    public ExecutorService apiCallExecutor() {
        return new ThreadPoolExecutor(
            5,                          // core pool size
            20,                         // max pool size
            60L, TimeUnit.SECONDS,      // keep-alive for idle threads
            new LinkedBlockingQueue<>(100), // work queue capacity
            new ThreadFactoryBuilder()
                .setNameFormat("api-call-%d")
                .setDaemon(true)
                .build(),
            new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy
        );
    }
}

@Service
public class ExternalApiService {

    @Autowired
    @Qualifier("apiCallExecutor")
    private ExecutorService executor;

    public List<ApiResponse> callMultipleApis(List<String> urls) {
        List<Future<ApiResponse>> futures = urls.stream()
            .map(url -> executor.submit(() -> restTemplate.getForObject(url,
ApiResponse.class)))
            .collect(Collectors.toList());

        return futures.stream()
```

```
            .map(f -> {
                try { return f.get(5, TimeUnit.SECONDS); }
                catch (Exception e) { return ApiResponse.error(e.getMessage());
}
            })
            .collect(Collectors.toList());
    }
}
```

4.5 CompletableFuture (Modern Approach)

What

Introduced in **Java 8**, `CompletableFuture<T>` is a powerful, non-blocking, composable asynchronous programming model.

Why / When

The modern standard for async programming in Java. Supports chaining, combining, error handling, and callbacks — all without blocking.

Quick Preview (Full deep dive in Chapter 10)

```
@Service
public class OrderAggregatorService {

    public OrderDetails getOrderDetails(Long orderId) {

        CompletableFuture<Order> orderFuture =
            CompletableFuture.supplyAsync(() ->
orderService.findById(orderId));

        CompletableFuture<Customer> customerFuture = orderFuture
            .thenCompose(order ->
                CompletableFuture.supplyAsync(() ->
                    customerService.findById(order.getCustomerId())));

        CompletableFuture<List<Payment>> paymentsFuture =
            CompletableFuture.supplyAsync(() ->
paymentService.findByOrder(orderId));

        return orderFuture
```

```
            .thenCombine(customerFuture, (order, customer) ->
                new OrderDetails(order, customer))
            .thenCombine(paymentsFuture, (details, payments) -> {
                details.setPayments(payments);
                return details;
            })
            .exceptionally(ex -> {
                log.error("Failed to build order details", ex);
                return OrderDetails.empty();
            })
            .join(); // Terminal operation
    }
}
```

Comparison of All Approaches

Feature	Thread/Runnable	Callable+Future	ExecutorService	CompletableFuture
Return value	No (Runnable)	Yes	Yes	Yes
Exception handling	Manual	`get()` throws	`get()` throws	`exceptionally()`, `handle()`
Thread pool	No (raw thread)	No	**Yes**	Yes (ForkJoinPool default)
Chaining	No	No	No	**Yes**
Combining	No	No	Manual	**Yes** (`thenCombine`, `allOf`)
Non-blocking	No	`get()` blocks	`get()` blocks	**Yes** (callbacks)
Production use	[N] Avoid	⚠️ Limited	[Y] Good	[Y] **Best**

Thread.start() Internals: What Actually Happens

When you call `thread.start()`, here is the complete chain of events inside the JVM:

```
Java code: thread.start()
    |
    +--> Thread.start() [Java method]
    |      - Checks state != NEW -> throws IllegalThreadStateException if already
started
    |      - Adds thread to ThreadGroup
    |      - Calls native start0()
    |
```

```
    +--> JVM_StartThread() [C++ code in JVM]
    |      - Allocates JavaThread object (JVM internal representation)
    |      - Sets stack size (from -Xss or Thread constructor)
    |      - Calls os::create_thread()
    |
    +--> os::create_thread() [Platform-specific]
    |      - Linux:   pthread_create(&tid, &attr, thread_native_entry, thread)
    |      - Windows: CreateThread(NULL, stack_size, thread_native_entry, thread,
...)
    |      - macOS:   pthread_create() (same as Linux)
    |
    +--> thread_native_entry() [C++ callback, runs in NEW OS thread]
    |      - Initializes thread-local storage
    |      - Sets thread state to RUNNABLE
    |      - Calls thread->run() -> JavaThread::run()
    |
    +--> JavaThread::run()
          - Installs signal handlers
          - Calls Thread.run() [back to Java code!]
          - When run() returns -> thread cleanup -> state = TERMINATED

Key points:
  - Thread.start() can ONLY be called once. Calling it again throws
    IllegalThreadStateException. This is because OS threads cannot be restarted.
  - You can verify: thread.start(); thread.start(); // throws on second call
  - This is why thread pools REUSE threads instead of creating new ones.
```

Daemon Threads: Deep Understanding

Daemon threads are background service threads that **do not prevent JVM shutdown**. When all non-daemon (user) threads finish, the JVM terminates — killing all daemon threads abruptly.

```java
// Creating daemon threads
Thread daemon = new Thread(() -> {
    while (true) {
        cleanupExpiredCache();
        try { Thread.sleep(60000); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
});
daemon.setDaemon(true);  // MUST be set BEFORE start()
daemon.setName("cache-cleanup-daemon");
daemon.start();
```

```
// Daemon thread properties:
// 1. MUST call setDaemon(true) BEFORE start()
// 2. Child threads inherit daemon status from parent
// 3. Daemon threads are killed abruptly — no finally blocks executed!
// 4. Do NOT use for tasks that require cleanup (file writes, DB commits)
```

```
When to use daemon vs non-daemon threads:
-----------------------------------------
Daemon threads:                         Non-daemon threads:
  - GC, monitoring, heartbeats            - Business logic
  - Cache cleanup                         - File I/O that must complete
  - Metrics collection                    - Database transactions
  - Thread pool workers (usually)         - Message processing

⚠️ In Spring Boot: @Async threads are non-daemon by default.
   Tomcat worker threads are also non-daemon.
   Set daemon=true for fire-and-forget background tasks.
```

Handling InterruptedException: The Right Way

`InterruptedException` is how Java signals a thread to stop. Handling it correctly is a mark of senior engineering:

```
// [N] ANTI-PATTERN 1: Swallowing the interrupt (NEVER DO THIS)
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Silently ignored — the thread continues running!
    // Callers who tried to interrupt this thread are puzzled why it won't stop
}

// [N] ANTI-PATTERN 2: Just logging and continuing
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    log.error("Interrupted", e);  // Loses the interrupt flag!
}

// [Y] CORRECT PATTERN 1: Propagate the exception
public void doWork() throws InterruptedException {
    Thread.sleep(1000);  // Let caller handle it
}
```

```java
// [Y] CORRECT PATTERN 2: Restore interrupt flag + terminate
public void run() {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            processNextItem();
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();  // RESTORE the flag!
            break;                               // Exit the loop
        }
    }
    cleanup();  // Graceful cleanup before thread dies
}

// [Y] CORRECT PATTERN 3: Convert to unchecked exception
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException("Thread was interrupted", e);
}
```

Why restoring the interrupt flag matters: *When* `InterruptedException` *is thrown, the interrupt flag is automatically cleared. If you don't restore it with* `Thread.currentThread().interrupt()` *, higher-level code (like ExecutorService.shutdownNow()) has no way to know the thread was interrupted.*

Thread Creation: Common Anti-Patterns in Production

```java
// [N] ANTI-PATTERN 1: Creating threads in a loop
@GetMapping("/api/process")
public void processRequest() {
    for (Item item : items) {
        new Thread(() -> process(item)).start(); // 10,000 items = 10,000
threads = CRASH
    }
}

// [Y] FIX: Use ExecutorService with bounded pool
@Autowired @Qualifier("processingPool")
private ExecutorService executor;
```

```java
@GetMapping("/api/process")
public void processRequest() {
    items.forEach(item -> executor.submit(() -> process(item)));
}

// [N] ANTI-PATTERN 2: Not naming threads
new Thread(() -> doWork()).start();
// Thread dump shows: "Thread-47" — impossible to debug

// [Y] FIX: Always name threads
new Thread(() -> doWork(), "order-processor-1").start();
// Thread dump shows: "order-processor-1" — instantly meaningful

// [N] ANTI-PATTERN 3: Ignoring thread pool shutdown
@Bean
public ExecutorService executor() {
    return Executors.newFixedThreadPool(10);
    // Never shut down -> threads leak on application context refresh
}

// [Y] FIX: Graceful shutdown
@Bean(destroyMethod = "shutdown")
public ExecutorService executor() {
    return Executors.newFixedThreadPool(10);
}
```

Chapter 5: Synchronization

5.1 Race Conditions

What is a Race Condition?

A **race condition** occurs when two or more threads access shared data concurrently, and the outcome depends on the unpredictable order of thread execution. The program produces different results on different runs because thread scheduling is non-deterministic.

Why Race Conditions are Dangerous

Race conditions are among the most dangerous bugs in software because:

- They are **non-deterministic** — the bug may appear only 1 in 10,000 runs
- They are **hard to reproduce** — adding logging or debugging often changes thread timing and hides the bug
- They cause **data corruption** — financial calculations become wrong, inventory goes negative
- They can lead to **security vulnerabilities** — TOCTOU (Time-of-Check to Time-of-Use) attacks exploit race conditions

When Race Conditions Occur

A race condition requires ALL three of these conditions simultaneously:

1. **Shared mutable state** — multiple threads access the same variable/object
2. **At least one thread writes** — if all threads only read, there is no race
3. **No synchronization** — no mechanism prevents concurrent access

> ***Key Insight:*** *If you eliminate ANY ONE of these three conditions, the race condition disappears. This is the foundation of all thread-safety strategies.*

Real-World Production Scenarios

Scenario	What Goes Wrong
Two users buy the last item simultaneously	Inventory goes to -1, both get confirmation
Two threads update the same bank balance	Money appears or disappears
Two services claim the same resource	Double-booking, duplicate processing

Counter incremented by multiple threads	Final count is less than expected

Step-by-Step: How a Race Condition Happens

```
Thread A                            Thread B
--------                            --------
Step 1: Read balance (1000)
                                    Step 2: Read balance (1000)  <- stale!
Step 3: balance = 1000 - 800
        balance = 200               Step 4: balance = 1000 - 800
                                            balance = 200  <- OVERWRITES Thread
A's write!

Result: Only $800 deducted instead of $1600. Bank loses $800.
```

Example: The Classic Bank Account Problem

```java
public class BankAccount {
    private double balance = 1000.0; // Shared mutable state

    public void withdraw(double amount) {
        if (balance >= amount) {          // Step 1: CHECK
            // Thread could be preempted here!
            balance -= amount;            // Step 2: ACT
        }
    }

    public double getBalance() {
        return balance;
    }
}

// Two threads withdrawing simultaneously:
BankAccount account = new BankAccount(); // balance = 1000

// Thread A: withdraw(800)
// Thread B: withdraw(800)

// Possible interleaving:
// Thread A: checks balance >= 800 -> true (balance is 1000)
// Thread B: checks balance >= 800 -> true (balance is STILL 1000, A hasn't
subtracted yet!)
```

```
// Thread A: balance -= 800 -> balance = 200
// Thread B: balance -= 800 -> balance = -600  <- OVERDRAFT! Data corruption!
```

5.2 Critical Sections

What is a Critical Section?

A **critical section** is a block of code that accesses shared resources and must be executed by only one thread at a time. It is the smallest possible region of code that needs protection.

Why Critical Sections Matter

The goal is to make the critical section **as small as possible**. Locking too much code reduces concurrency (threads wait longer). Locking too little causes race conditions.

Design Principle: Minimize Lock Scope

```
BAD:  Lock entire method (other threads blocked for everything)
+-----------------------------------------------------+
|  🔒 LOCKED                                          |
|  Read config (safe)  ->  Access shared data  ->  Log  |
+-----------------------------------------------------+

GOOD: Lock only the shared data access (other threads blocked minimally)
+--------------+ +-----------------+ +--------------+
|  Read config | |  🔒 LOCKED       | |  Log         |
|  (no lock)   | |  Access shared   | |  (no lock)   |
+--------------+ +-----------------+ +--------------+
```

Execution Flow

```
Thread A ---> [Enter Critical Section] ---> [Read/Write Shared Data] ---> [Exit]
                      🔒 LOCKED
Thread B ---> [Waiting...             ] ---------------------------> [Enter] ---
> [...]
```

5.3 The synchronized Keyword

What is synchronized?

The `synchronized` keyword provides **mutual exclusion** (mutex) — only one thread can execute a synchronized block/method at a time for a given lock object.

Why synchronized Exists

Java needed a simple, language-level mechanism to protect critical sections. Before `java.util.concurrent` (added in Java 5), `synchronized` was the **only** way to achieve thread safety.

When to Use synchronized

Use synchronized When	Don't Use synchronized When
Simple mutual exclusion needed	You need tryLock or timeout
Few threads contending	High contention (100+ threads)
Code simplicity matters	You need read/write lock separation
Lock scope is a single method/block	You need to lock across methods

Alternatives to synchronized

Alternative	When to Use Instead
`ReentrantLock`	Need tryLock, timeout, fair ordering, multiple conditions
`ReadWriteLock`	Read-heavy workload, few writes
`StampedLock`	Optimistic reads, highest performance
`Atomic*` classes	Simple counters, flags (no locking needed)
`ConcurrentHashMap`	Thread-safe map without external locking

How It Works Internally

Every Java object has an **intrinsic lock** (also called **monitor**). When a thread enters a synchronized block:

1. Thread attempts to acquire the object's monitor lock
2. If lock is free -> thread acquires it and enters the block
3. If lock is held -> thread goes to **BLOCKED** state
4. When thread exits the block -> lock is released

5. One waiting thread is awakened and acquires the lock

```
Object's Monitor:
+-------------------------+
|  Owner: Thread-A        |  <- Currently holds the lock
|  Entry Set: [Thread-B,  |  <- Threads waiting to acquire
|              Thread-C]  |
|  Wait Set:  [Thread-D]  |  <- Threads that called wait()
+-------------------------+
```

Synchronized Method

```java
public class BankAccount {
    private double balance = 1000.0;

    // Lock object = "this" (the BankAccount instance)
    public synchronized void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        }
    }

    public synchronized void deposit(double amount) {
        balance += amount;
    }

    public synchronized double getBalance() {
        return balance;
    }
}
```

Synchronized Block (More Granular)

```java
public class InventoryService {
    private final Map<String, Integer> stock = new HashMap<>();
    private final Object stockLock = new Object(); // Dedicated lock object

    public boolean reserveItem(String itemId, int quantity) {
        synchronized (stockLock) { // Only lock the critical section
            Integer available = stock.getOrDefault(itemId, 0);
            if (available >= quantity) {
```

```
                stock.put(itemId, available - quantity);
                return true;
            }
            return false;
        }
        // Non-critical code runs without holding the lock
    }

    public void restockItem(String itemId, int quantity) {
        synchronized (stockLock) {
            stock.merge(itemId, quantity, Integer::sum);
        }
    }
}
```

Static Synchronized Method

```
public class ConnectionPool {
    private static final List<Connection> pool = new ArrayList<>();

    // Lock object = ConnectionPool.class (the Class object)
    public static synchronized Connection getConnection() {
        if (pool.isEmpty()) {
            return createNewConnection();
        }
        return pool.remove(pool.size() - 1);
    }

    public static synchronized void returnConnection(Connection conn) {
        pool.add(conn);
    }
}
```

Synchronized Block vs Method

Feature	Synchronized Method	Synchronized Block
Lock object	`this` (instance) or `Class` (static)	Any specified object
Granularity	Entire method	Specific code section
Performance	May hold lock too long	Fine-grained, better concurrency

Flexibility	Less flexible	Can use different locks for different resources
Best for	Simple cases	Production code with multiple shared resources

Enterprise Example: Concurrent Microservice Updates

```java
@Service
public class OrderProcessingService {

    private final Map<Long, Order> orderCache = new ConcurrentHashMap<>();

    // Use per-order locking to allow concurrent updates to DIFFERENT orders
    public void updateOrderStatus(Long orderId, OrderStatus newStatus) {
        Order order = orderCache.get(orderId);
        if (order == null) return;

        synchronized (order) { // Lock on the specific order, not the entire
cache
            if (order.getStatus().canTransitionTo(newStatus)) {
                order.setStatus(newStatus);
                order.setUpdatedAt(Instant.now());
                orderRepository.save(order);
                eventPublisher.publish(new OrderStatusChangedEvent(orderId,
newStatus));
            }
        }
    }
}
```

5.4 Volatile Keyword

What is volatile?

The `volatile` keyword ensures **visibility** of changes across threads. When a variable is declared volatile, reads/writes go directly to main memory (no CPU cache).

Why volatile Exists — The JMM Visibility Problem

Without `volatile`, each thread may work with a **cached copy** of a variable in its CPU cache. Thread A updates a variable, but Thread B never sees the update because it reads its own stale cached copy.

```
Without volatile:
+--------------+     +--------------+
|  Thread A    |     |  Thread B    |
|  CPU Cache   |     |  CPU Cache   |
|  running=F   |     |  running=T   |  <- STALE! Thread B never sees the update
+------+-------+     +------+-------+
       |                    |
   +---+--------------------+---+
   |     Main Memory            |
   |     running = false        |
   +----------------------------+

With volatile:
  Thread A writes -> goes DIRECTLY to main memory
  Thread B reads  -> reads DIRECTLY from main memory
  No caching, always fresh value
```

When to Use volatile

- Simple flags (boolean state): `volatile boolean running = true;`
- When only **one thread writes** and others read
- Status indicators, configuration flags, shutdown signals

When NOT to Use volatile

- **Compound operations** (check-then-act, increment) — NOT atomic with volatile
- When **multiple threads write** to the same variable
- When you need **mutual exclusion** — volatile provides only visibility, not atomicity

```java
public class GracefulShutdown {
    private volatile boolean running = true; // Visible to all threads

    public void run() {
        while (running) { // Other thread can set running = false
            processNextItem();
        }
        cleanup();
    }

    public void shutdown() {
        running = false; // Immediately visible to the run() thread
    }
}
```

volatile vs synchronized

Feature	volatile	synchronized
Atomicity	No (only read/write of variable)	Yes (entire block)
Visibility	Yes	Yes
Mutual exclusion	No	Yes
Performance	Faster (no locking)	Slower (lock acquisition)
Use case	Simple flags, single writes	Compound operations

5.5 Java Memory Model (JMM) — Deep Dive

Why the JMM Exists

The Java Memory Model (JSR-133) defines the rules for how threads interact through memory. Without it, the JIT compiler, CPU, and memory hardware are free to **reorder** and **cache** operations in ways that break multithreaded code.

```
The Problem Without JMM Guarantees:
-----------------------------------

Java code:              What CPU/compiler might actually do:
  x = 1;                  y = 2;     <- REORDERED! (faster this way)
  y = 2;                  x = 1;

This reordering is invisible in single-threaded code (same result).
But in multithreaded code, Thread B might see y=2 but x=0 (old value)!

Sources of reordering:
1. Compiler (JIT) reordering — for register optimization
2. CPU instruction reordering — out-of-order execution pipelines
3. Store buffer reordering — writes buffered before flushing to cache
4. Cache coherence delays — different cores see updates at different times
```

Happens-Before Relationship

The **happens-before** relationship is the foundation of JMM. If action A *happens-before* action B, then A's effects are **guaranteed visible** to B.

```
Happens-Before Rules (Memorize These!):
----------------------------------------

1. PROGRAM ORDER RULE
   Within a single thread, each statement happens-before the next.
   x = 1;          <- happens-before ->
   y = x + 1;      <- guaranteed to see x = 1

2. MONITOR LOCK RULE
   An unlock() on a monitor happens-before every subsequent lock() on that
monitor.
   Thread A: synchronized(lock) { x = 1; }        <- unlock happens
   Thread B: synchronized(lock) { print(x); }     <- lock happens -> sees x = 1

3. VOLATILE VARIABLE RULE
   A write to a volatile variable happens-before every subsequent read of that
variable.
   Thread A: volatile_flag = true;     <- write
   Thread B: if (volatile_flag) { }    <- read -> guaranteed to see true

4. THREAD START RULE
   Thread.start() happens-before any action in the started thread.
   x = 42;
   thread.start();     <- thread.run() sees x = 42

5. THREAD TERMINATION RULE
   Any action in a thread happens-before Thread.join() returns.
   // In thread: x = 42;
   thread.join();      <- after join, caller sees x = 42

6. TRANSITIVITY
   If A happens-before B, and B happens-before C, then A happens-before C.
   This is how volatile can be used to publish non-volatile data safely.
```

Memory Barriers (Fences)

Memory barriers are CPU instructions that enforce ordering. The JVM inserts them automatically for `volatile` and `synchronized`:

```
Types of Memory Barriers:
-------------------------

LoadLoad   — All reads BEFORE the barrier complete before reads AFTER it
StoreStore — All writes BEFORE the barrier are flushed before writes AFTER it
LoadStore  — All reads BEFORE the barrier complete before writes AFTER it
```

```
StoreLoad  — All writes BEFORE the barrier are flushed before reads AFTER it
             (most expensive, full memory fence)

What the JVM inserts:

volatile READ:
  [LoadLoad barrier]
  read volatile variable
  [LoadStore barrier]

volatile WRITE:
  [StoreStore barrier]
  write volatile variable
  [StoreLoad barrier]    <- this is why volatile writes are expensive

synchronized ENTER:
  acquire monitor lock
  [LoadLoad + LoadStore barrier]    <- refresh all cached values

synchronized EXIT:
  [StoreStore + LoadStore barrier]  <- flush all writes
  release monitor lock
```

The Double-Checked Locking Pattern Explained

This pattern is the classic example of why JMM matters:

```java
public class ConnectionPool {
    // WITHOUT volatile: BROKEN!
    // private static ConnectionPool instance;

    // WITH volatile: CORRECT
    private static volatile ConnectionPool instance;

    public static ConnectionPool getInstance() {
        if (instance == null) {                    // 1st check (no lock)
            synchronized (ConnectionPool.class) {
                if (instance == null) {            // 2nd check (with lock)
                    instance = new ConnectionPool(); // <- THE PROBLEM
                }
            }
        }
        return instance;
    }
}
```

```
// Why volatile is required:
// "instance = new ConnectionPool()" is actually 3 steps:
//   1. Allocate memory for ConnectionPool object
//   2. Initialize the object (constructor runs)
//   3. Assign reference to 'instance' variable
//
// Without volatile, the CPU may REORDER steps 2 and 3:
//   1. Allocate memory
//   3. Assign reference (instance now non-null!)  <- REORDERED!
//   2. Initialize object (constructor hasn't run yet!)
//
// Thread B sees instance != null, returns a PARTIALLY CONSTRUCTED object!
// volatile prevents this reordering with a StoreStore barrier.
```

5.6 False Sharing — The Hidden Performance Killer

What is False Sharing?

False sharing occurs when two threads on different cores modify **different variables** that happen to reside on the **same CPU cache line** (64 bytes). Both cores constantly invalidate each other's cache lines, destroying performance.

```
False Sharing Scenario:
-----------------------

class Counters {
    volatile long counterA = 0;  // Thread 1 writes this
    volatile long counterB = 0;  // Thread 2 writes this
}

Memory layout (both fit in ONE 64-byte cache line):
+-------------------------------------------------------------+
|  Cache Line (64 bytes)                                      |
|  [counterA: 8 bytes] [counterB: 8 bytes] [padding: 48 bytes]  |
+-------------------------------------------------------------+

Core 0 (Thread 1)              Core 1 (Thread 2)
  writes counterA                writes counterB
  -> invalidates Core 1's         -> invalidates Core 0's
    cache line!                     cache line!
  -> Core 1 must reload           -> Core 0 must reload
    from L3/memory                  from L3/memory

Result: 10-100x slower than expected!
```

```
Each "independent" write causes a full cache line transfer (~100ns)
instead of an L1 cache hit (~1ns).
```

How to Fix False Sharing

```java
// Solution 1: @Contended annotation (Java 8+, -XX:-RestrictContended)
import sun.misc.Contended;

class Counters {
    @Contended
    volatile long counterA = 0;   // Padded to its own cache line

    @Contended
    volatile long counterB = 0;   // Padded to its own cache line
}

// Solution 2: Manual padding
class Counters {
    volatile long counterA = 0;
    long p1, p2, p3, p4, p5, p6, p7;  // 56 bytes of padding
    volatile long counterB = 0;
    // Now counterA and counterB are on DIFFERENT cache lines
}

// Real-world example: LongAdder uses @Contended internally
// on its Cell[] array to avoid false sharing between cells
```

> ***Production Insight:*** *False sharing is why* `LongAdder` *is faster than* `AtomicLong` *under high contention.* `LongAdder` *spreads updates across multiple cache-line-padded cells, so different threads don't invalidate each other's cache lines.*

5.7 Atomic Classes — Lock-Free Thread Safety

The CAS (Compare-And-Swap) Hardware Instruction

All Atomic classes are built on top of **CAS** — an atomic CPU instruction that enables lock-free programming:

```
CAS(memoryLocation, expectedValue, newValue):
  Atomically:
```

```
    if (memoryLocation == expectedValue) {
        memoryLocation = newValue;
        return true;   // Success
    } else {
        return false;  // Someone else changed it first, try again
    }

CPU instruction: CMPXCHG on x86, LL/SC on ARM

How AtomicInteger.incrementAndGet() works internally:
  1. Read current value (e.g., 5)
  2. Compute new value (5 + 1 = 6)
  3. CAS(address, 5, 6)
     -> If current is still 5: set to 6, return true [Y]
     -> If current is now 7 (another thread changed it): return false [N]
  4. If CAS failed: RETRY from step 1 (spin loop)
```

Atomic Classes Overview

```java
// AtomicInteger / AtomicLong — atomic int/long operations
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();                      // Thread-safe ++
counter.compareAndSet(expected, newValue);       // CAS
counter.getAndUpdate(x -> x * 2);               // Atomic function application
counter.accumulateAndGet(5, Integer::sum);       // Atomic accumulation

// AtomicBoolean — atomic flag
AtomicBoolean initialized = new AtomicBoolean(false);
if (initialized.compareAndSet(false, true)) {
    // Only ONE thread enters here — guaranteed
    performInitialization();
}

// AtomicReference<V> — atomic reference swap
AtomicReference<Config> configRef = new AtomicReference<>(loadConfig());
// Atomically update config (lock-free)
configRef.updateAndGet(old -> loadNewConfig());

// AtomicStampedReference<V> — solves the ABA problem
AtomicStampedReference<String> ref = new AtomicStampedReference<>("A", 0);
int[] stampHolder = new int[1];
String value = ref.get(stampHolder);   // Gets value AND stamp
ref.compareAndSet("A", "B", stampHolder[0], stampHolder[0] + 1);
```

LongAdder vs AtomicLong (Critical for High-Contention Counters)

```
// AtomicLong: ONE variable, ALL threads CAS on it -> contention bottleneck
AtomicLong atomicCounter = new AtomicLong(0);
atomicCounter.incrementAndGet();  // Under 100 threads: LOTS of CAS retries

// LongAdder: MULTIPLE cells, threads spread across cells -> minimal contention
LongAdder adderCounter = new LongAdder();
adderCounter.increment();         // Under 100 threads: each thread hits
different cell
long total = adderCounter.sum();  // Sum all cells (slightly stale, but fast)
```

```
Performance comparison (100 threads, 1M increments each):
---------------------------------------------------------
AtomicLong:    ~3,200 ms  (all threads fighting over one CAS)
LongAdder:     ~380 ms   (threads spread across padded cells)
synchronized:  ~4,500 ms  (lock acquisition overhead)

When to use which:
  AtomicLong  -> low contention, need exact real-time value
  LongAdder   -> high contention, can tolerate slightly stale sum()
  LongAccumulator -> high contention, custom accumulation function
```

Atomic vs Volatile vs Synchronized Decision Table

Requirement	Use	Why
Simple flag (one writer, many readers)	`volatile`	Cheapest visibility guarantee
Counter (multiple writers)	`AtomicInteger` / `LongAdder`	Lock-free, no blocking
Check-then-act (compare-and-swap)	`AtomicReference.CAS()`	Single atomic operation
Multiple fields must be consistent	`synchronized` / `ReentrantLock`	Only way to group operations
High-contention counter	`LongAdder`	Distributes contention across cells

---

Chapter 6: Inter-Thread Communication

6.1 wait(), notify(), notifyAll()

What is Inter-Thread Communication?

Inter-thread communication allows threads to **coordinate** their actions based on shared state. Instead of busy-waiting (spinning in a loop checking a condition), threads can efficiently **sleep** until another thread signals that the condition has changed.

Why It Exists — The Polling Problem

Without wait/notify, a thread must continuously check a condition:

```
BAD: Busy-waiting (wastes CPU cycles)
while (!dataAvailable) {
    // Thread spins, consuming 100% CPU doing nothing useful
}

GOOD: wait/notify (thread sleeps, zero CPU usage while waiting)
while (!dataAvailable) {
    wait();  // Thread sleeps, uses no CPU. Woken up by notify()
}
```

When to Use wait/notify vs Modern Alternatives

Approach	Use When
`wait()/notify()`	Legacy code, learning fundamentals
`Condition` (from `Lock`)	Need multiple wait conditions on one lock
`BlockingQueue`	**Production code** — handles wait/notify internally
`CompletableFuture`	One-time async result passing
`CountDownLatch`	One thread waits for N threads to finish

> ***Best Practice:*** *In production code, prefer* `BlockingQueue` *or* `CompletableFuture` *over raw wait/notify. They handle synchronization internally and are less error-prone.*

How wait/notify Works Internally

Method	Description
`wait()`	Current thread releases lock and enters WAITING state until notified
`wait(long ms)`	Same as wait() but with timeout (TIMED_WAITING)
`notify()`	Wakes up ONE arbitrary waiting thread
`notifyAll()`	Wakes up ALL waiting threads (recommended)

Step-by-Step Execution Flow

```
1. Thread A calls wait() on object X
   -> Thread A RELEASES the lock on X
   -> Thread A enters WAITING state (uses no CPU)
   -> Thread A is added to X's "wait set"

2. Thread B acquires lock on X, modifies shared state
   -> Thread B calls notifyAll() on X
   -> All threads in X's wait set are moved to "entry set"
   -> Thread B releases lock on X

3. Thread A re-acquires lock on X
   -> Thread A re-checks condition in while loop
   -> If condition met: proceeds. If not: calls wait() again
```

Critical Rules

1. Must be called inside `synchronized` block on the **same object**
2. `wait()` releases the lock; the thread re-acquires it when woken up
3. Always use `while` loop (not `if`) to check condition — guard against **spurious wakeups**
4. Prefer `notifyAll()` over `notify()` to avoid lost signals

Design Pattern: Guarded Suspension

The wait/notify pattern implements the **Guarded Suspension** design pattern — a thread suspends execution until a guard condition becomes true.

6.2 Producer-Consumer Pattern

What is the Producer-Consumer Pattern?

The **Producer-Consumer pattern** decouples data production from data consumption. Producers generate data and place it in a shared buffer. Consumers take data from the buffer and process it. Neither needs to know about the other.

Why This Pattern is Used

- **Decoupling** — producers and consumers work independently at different speeds
- **Buffering** — absorb temporary speed mismatches (producer faster than consumer)
- **Scalability** — easily add more producers or consumers

Real-World Examples

System	Producer	Buffer	Consumer
REST API	HTTP request handler	BlockingQueue	Background worker threads
Kafka	Event publisher	Kafka topic	Consumer group
Batch processing	File reader	In-memory queue	Record processor
Logging	Application thread	Log buffer	Log writer thread

```java
public class MessageQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public MessageQueue(int capacity) {
        this.capacity = capacity;
    }

    // Producer: Add message to queue
    public synchronized void produce(T message) throws InterruptedException {
        // WHILE (not IF) — guards against spurious wakeups
        while (queue.size() == capacity) {
            System.out.println("Queue full. Producer waiting...");
            wait(); // Release lock, wait for consumer to consume
        }

        queue.offer(message);
        System.out.println("Produced: " + message + " | Queue size: " +
queue.size());
        notifyAll(); // Wake up waiting consumers
    }
```

```java
    // Consumer: Take message from queue
    public synchronized T consume() throws InterruptedException {
        while (queue.isEmpty()) {
            System.out.println("Queue empty. Consumer waiting...");
            wait(); // Release lock, wait for producer to produce
        }

        T message = queue.poll();
        System.out.println("Consumed: " + message + " | Queue size: " +
queue.size());
        notifyAll(); // Wake up waiting producers
        return message;
    }
}

// Usage
MessageQueue<String> queue = new MessageQueue<>(5);

// Producer thread
Thread producer = new Thread(() -> {
    for (int i = 0; i < 20; i++) {
        try {
            queue.produce("Message-" + i);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

// Consumer thread
Thread consumer = new Thread(() -> {
    for (int i = 0; i < 20; i++) {
        try {
            queue.consume();
            Thread.sleep(500); // Simulate processing
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

producer.start();
consumer.start();
```

Step-by-Step Execution Flow

```
1. Producer starts, queue is empty (size 0)
2. Producer calls produce("Message-0")
   -> Acquires lock on queue object
   -> queue.size() (0) != capacity (5), so no wait
   -> Adds message, queue size = 1
   -> Calls notifyAll() (no one waiting yet)
   -> Releases lock

3. Producer produces Messages 1-4 similarly (queue size -> 5)

4. Producer calls produce("Message-5")
   -> Acquires lock
   -> queue.size() (5) == capacity (5)!
   -> Calls wait() -> RELEASES lock, enters WAITING state

5. Consumer starts, calls consume()
   -> Acquires lock (producer released it via wait())
   -> Queue not empty, polls "Message-0"
   -> Calls notifyAll() -> WAKES UP producer
   -> Releases lock

6. Producer wakes up, re-acquires lock
   -> Re-checks while condition: queue.size() (4) != 5
   -> Adds "Message-5", notifies all
   -> Releases lock

7. Cycle continues until all messages produced and consumed
```

Modern Alternative: BlockingQueue

```java
// In production, use BlockingQueue instead of manual wait/notify
BlockingQueue<String> queue = new ArrayBlockingQueue<>(5);

// Producer
queue.put("message");  // Blocks if full

// Consumer
String msg = queue.take();  // Blocks if empty
```

Chapter 7: Locks Framework (java.util.concurrent.locks)

Why a Separate Locks Framework?

The `synchronized` keyword was Java's only locking mechanism until Java 5. While simple, it has limitations:

- Cannot attempt a lock without blocking (no tryLock)
- Cannot timeout while waiting for a lock
- Cannot interrupt a thread waiting for a lock
- Cannot have fairness guarantees
- Only one wait/notify condition per lock

The `java.util.concurrent.locks` package was introduced in **Java 5** to address all of these limitations while maintaining backward compatibility.

7.1 ReentrantLock

What is ReentrantLock?

A **reentrant mutual exclusion lock** with extended capabilities beyond `synchronized`. The same thread can acquire the lock multiple times without deadlocking (hence "reentrant" — re-entering the same lock).

Why "Reentrant" Matters

```
Scenario: Method A calls Method B, both need the same lock

Non-reentrant lock:     Reentrant lock:
  methodA() {             methodA() {
    lock.lock();            lock.lock();       // count = 1
    methodB();  <- DEADLOCK! methodB();         // works fine
  }                       }
  methodB() {             methodB() {
    lock.lock(); <- blocks   lock.lock();       // count = 2
    forever!                // ... work ...
  }                         lock.unlock();     // count = 1
                          }
```

When to Use ReentrantLock vs synchronized

Use ReentrantLock When	Stick with synchronized When
Need tryLock() or lock timeout	Simple mutual exclusion
Need fair lock ordering	Code simplicity matters most
Need multiple Condition objects	Short critical sections
Need interruptible lock waiting	No need for advanced features
Complex locking scenarios	Low contention scenarios

Why Use Over synchronized

- **tryLock()** — Non-blocking lock attempt
- **tryLock(timeout)** — Timed lock attempt
- **lockInterruptibly()** — Can be interrupted while waiting
- **Fairness policy** — Optional FIFO ordering of waiting threads
- **Multiple Condition objects** — More flexible than wait/notify

> ***Production Rule:*** *Always use* `try-finally` *with ReentrantLock. If you forget* `unlock()` *in a* `finally` *block, threads will deadlock permanently.*

How It Works

```java
public class ThreadSafeCounter {
    private int count = 0;
    private final ReentrantLock lock = new ReentrantLock();

    public void increment() {
        lock.lock();            // Acquire lock
        try {
            count++;            // Critical section
        } finally {
            lock.unlock();      // ALWAYS unlock in finally!
        }
    }

    public boolean tryIncrement() {
        if (lock.tryLock()) {   // Non-blocking attempt
            try {
```

```
                count++;
                return true;
            } finally {
                lock.unlock();
            }
        }
        return false; // Lock not available, skip
    }

    public void incrementWithTimeout() throws InterruptedException {
        if (lock.tryLock(2, TimeUnit.SECONDS)) { // Wait up to 2 seconds
            try {
                count++;
            } finally {
                lock.unlock();
            }
        } else {
            throw new TimeoutException("Could not acquire lock within 2
seconds");
        }
    }
}
```

Fair vs Unfair Lock

```
// Unfair (default) — Better throughput, but thread starvation possible
ReentrantLock unfairLock = new ReentrantLock();       // unfair
ReentrantLock fairLock = new ReentrantLock(true);     // fair — FIFO ordering

// Fair lock: threads acquire lock in the order they requested it
// Unfair lock: a thread can "barge in" and acquire the lock ahead of waiting
threads
// Unfair is usually preferred for better throughput
```

Condition Variables (Advanced wait/notify)

```
public class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();   // Separate
condition
```

```java
    private final Condition notEmpty = lock.newCondition();  // Separate condition

    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    public void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity) {
                notFull.await();     // Wait until not full
            }
            queue.offer(item);
            notEmpty.signal();       // Signal that buffer is not empty
        } finally {
            lock.unlock();
        }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                notEmpty.await();    // Wait until not empty
            }
            T item = queue.poll();
            notFull.signal();        // Signal that buffer is not full
            return item;
        } finally {
            lock.unlock();
        }
    }
}
```

7.2 ReadWriteLock

What is ReadWriteLock?

Allows multiple **concurrent readers** OR one **exclusive writer**. Dramatically improves throughput for read-heavy scenarios.

Why ReadWriteLock Exists

With `synchronized` or `ReentrantLock`, even read operations block each other. In a system where 95% of operations are reads, this wastes concurrency. ReadWriteLock allows all readers to proceed

simultaneously — only writers need exclusive access.

When to Use ReadWriteLock

- **Read-heavy workloads** (90%+ reads, few writes)
- Configuration caches, reference data, lookup tables
- When reads are significantly more frequent than writes

When NOT to Use ReadWriteLock

- **Write-heavy workloads** — overhead of read/write tracking adds no benefit
- **Short critical sections** — lock overhead may exceed actual work
- Consider `ConcurrentHashMap` for concurrent map access instead

Real-World Scenario

```
Configuration Cache (1000 reads/sec, 1 write/min):

synchronized:          ReadWriteLock:
  [R1] [R2] [R3]...     [R1] [R2] [R3]  <- all read simultaneously
  Each waits for         [R4] [R5]       <- no waiting!
  the previous one       [W1]            <- only writer blocks
  Throughput: LOW        Throughput: HIGH
```

How It Works

```
Reader threads:    [R1] [R2] [R3] — All can read simultaneously
Writer thread:     [W1]          — Exclusive access, blocks all readers and
other writers

Rules:
- Multiple readers CAN hold the read lock simultaneously
- Only ONE writer can hold the write lock
- Writer blocks all readers and other writers
- Readers block writers
```

Implementation

```java
@Service
public class ConfigurationCache {
```

```java
    private final Map<String, String> config = new HashMap<>();
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();
    private final Lock readLock = rwLock.readLock();
    private final Lock writeLock = rwLock.writeLock();

    // Multiple threads can read concurrently
    public String getConfig(String key) {
        readLock.lock();
        try {
            return config.getOrDefault(key, "");
        } finally {
            readLock.unlock();
        }
    }

    // Only one thread writes at a time (exclusive)
    public void updateConfig(String key, String value) {
        writeLock.lock();
        try {
            config.put(key, value);
            log.info("Config updated: {} = {}", key, value);
        } finally {
            writeLock.unlock();
        }
    }

    // Bulk reload requires exclusive access
    public void reloadAll(Map<String, String> newConfig) {
        writeLock.lock();
        try {
            config.clear();
            config.putAll(newConfig);
        } finally {
            writeLock.unlock();
        }
    }
}
```

7.3 StampedLock (Java 8+)

What

An advanced lock that adds an **optimistic read mode** to ReadWriteLock. Optimistic reads don't acquire any lock — they check afterward if data was modified.

Three Modes

Mode	Description	Use Case
Read Lock	Shared lock (like ReadWriteLock)	Read when writes are frequent
Write Lock	Exclusive lock	Write operations
Optimistic Read	No locking, just a stamp check	Read-heavy, occasional writes

How Optimistic Read Works

```java
public class Point {
    private double x, y;
    private final StampedLock lock = new StampedLock();

    public void move(double deltaX, double deltaY) {
        long stamp = lock.writeLock();      // Exclusive write lock
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            lock.unlockWrite(stamp);
        }
    }

    public double distanceFromOrigin() {
        long stamp = lock.tryOptimisticRead();  // No locking! Just get a stamp
        double currentX = x;
        double currentY = y;

        if (!lock.validate(stamp)) {            // Check if a write happened
            // Write happened during our read -> fall back to read lock
            stamp = lock.readLock();
            try {
                currentX = x;
                currentY = y;
            } finally {
                lock.unlockRead(stamp);
            }
        }

        return Math.sqrt(currentX * currentX + currentY * currentY);
```

```
    }
}
```

Lock Comparison Table

Feature	synchronized	ReentrantLock	ReadWriteLock	StampedLock
Fairness	No	Configurable	Configurable	No
tryLock	No	Yes	Yes	Yes
Interruptible	No	Yes	Yes	Yes
Condition	1 (wait/notify)	Multiple	Multiple	No
Reentrant	Yes	Yes	Yes	No
Read/Write split	No	No	Yes	Yes
Optimistic read	No	No	No	**Yes**
Performance	Good	Good	Better (reads)	**Best** (reads)
Complexity	Low	Medium	Medium	High

7.4 AbstractQueuedSynchronizer (AQS) — The Backbone

Almost every synchronization utility in `java.util.concurrent` is built on top of **AQS**: ReentrantLock, Semaphore, CountDownLatch, ReentrantReadWriteLock, and more.

```
AQS Internal Structure:
------------------------

+-----------------------------------------------------+
|  AbstractQueuedSynchronizer                         |
|                                                     |
|  state: int (volatile)                              |
|    - ReentrantLock: 0 = unlocked, N = lock count    |
|    - Semaphore: number of permits remaining          |
|    - CountDownLatch: count remaining                 |
|                                                     |
|  CLH Wait Queue (FIFO):                             |
|  +------+    +------+    +------+                   |
|  |Head  | ->  |Node 1| ->  |Node 2| -> null           |
|  |(dummy)|    |(T1)  |    |(T2)  |                   |
```

```
|  +------+    +------+    +------+                |
|               Thread 1    Thread 2                 |
|               WAITING     WAITING                  |
|                                                    |
|  exclusiveOwnerThread: Thread                      |
|    - Which thread currently owns the lock          |
+----------------------------------------------------+

How ReentrantLock.lock() works internally:
  1. Try CAS(state, 0, 1) — fast path (no contention)
     -> Success: set exclusiveOwnerThread = currentThread, return
  2. If CAS fails: is currentThread == owner? (reentrant check)
     -> Yes: state++ (increment hold count), return
  3. Otherwise: create CLH Node, add to queue tail (CAS)
     -> Park thread using LockSupport.park()
     -> When unparked: retry acquisition from queue head
```

> ***Interview Gold:*** *When asked "how does ReentrantLock work internally?", explain the AQS state variable, CLH queue, and the fast-path CAS acquisition. This demonstrates deep JVM knowledge.*

7.5 Synchronization Utilities

CountDownLatch — Wait for N Events

Use case: Main thread waits for multiple worker threads to complete initialization.

```java
// Production scenario: Service startup — wait for all caches to warm up
@Component
public class ApplicationStartup {

    @EventListener(ApplicationReadyEvent.class)
    public void onStartup() throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(3);  // Wait for 3 tasks

        executor.submit(() -> {
            loadUserCache();        // Takes ~2 seconds
            latch.countDown();      // 3 -> 2
        });
        executor.submit(() -> {
            loadProductCatalog();   // Takes ~3 seconds
            latch.countDown();      // 2 -> 1
        });
        executor.submit(() -> {
```

```
            warmUpConnectionPool(); // Takes ~1 second
            latch.countDown();      // 1 -> 0
        });

        boolean completed = latch.await(30, TimeUnit.SECONDS);  // Wait for all
3
        if (!completed) {
            throw new IllegalStateException("Startup timed out!");
        }
        log.info("All caches warmed. Application ready to serve traffic.");
    }
}
```

```
CountDownLatch internals (backed by AQS):
  state = N (initial count)
  countDown() -> decrementAndGet(state)
  await()     -> if state > 0, park thread
  When state reaches 0 -> unpark ALL waiting threads

Key properties:
  - ONE-TIME USE — cannot be reset (use CyclicBarrier for reusable)
  - Multiple threads can await() on the same latch
  - countDown() can be called by different threads
```

CyclicBarrier — Synchronize N Threads at a Barrier Point

Use case: Multiple threads must all reach a checkpoint before any can proceed (common in iterative algorithms).

```
// Production scenario: Parallel data processing with merge step
public class ParallelReportGenerator {

    public Report generateReport(List<DataSource> sources) throws Exception {
        int parties = sources.size();
        List<PartialReport> results = Collections.synchronizedList(new
ArrayList<>());

        // Barrier action runs when ALL threads arrive
        CyclicBarrier barrier = new CyclicBarrier(parties, () -> {
            log.info("All {} partitions processed. Merging results...",
parties);
        });
```

```
        for (DataSource source : sources) {
            executor.submit(() -> {
                try {
                    PartialReport partial = processDataSource(source);
                    results.add(partial);
                    barrier.await(60, TimeUnit.SECONDS);  // Wait for all
threads
                } catch (Exception e) {
                    log.error("Partition failed", e);
                }
            });
        }

        return mergeReports(results);
    }
}
```

```
CyclicBarrier vs CountDownLatch:
---------------------------------
                    CountDownLatch        CyclicBarrier
  Reusable?         No (one-time)         Yes (reset after trip)
  Who waits?        Any thread            Only the participating threads
  Who counts down?  Any thread            The waiting threads themselves
  Barrier action?   No                    Yes (Runnable at trip point)
  Use case          "Wait for N events"   "All N threads synchronize"
```

Semaphore — Control Concurrent Access to a Resource

Use case: Limit the number of concurrent connections, API calls, or resource accesses.

```
// Production scenario: Rate-limiting external API calls
@Service
public class ExternalApiClient {

    // Only 10 concurrent API calls allowed (rate limit from provider)
    private final Semaphore semaphore = new Semaphore(10);

    public ApiResponse callExternalApi(ApiRequest request) {
        try {
            semaphore.acquire();                    // Block if 10 calls in
progress
            try {
                return restTemplate.postForObject(API_URL, request,
ApiResponse.class);
```

```
            } finally {
                semaphore.release();               // Release permit
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new RuntimeException("Interrupted waiting for API permit",
e);
        }
    }

    // Non-blocking alternative
    public Optional<ApiResponse> tryCallApi(ApiRequest request) {
        if (semaphore.tryAcquire()) {             // Don't block, return
Optional.empty
            try {
                return Optional.of(restTemplate.postForObject(API_URL, request,
ApiResponse.class));
            } finally {
                semaphore.release();
            }
        }
        return Optional.empty();                  // API at capacity
    }
}
```

```
Semaphore internals (backed by AQS):
  state = N (number of permits)
  acquire() -> if state > 0: CAS(state, state-1), proceed
               if state == 0: park thread in CLH queue
  release() -> CAS(state, state+1), unpark one waiting thread

Binary Semaphore (permits=1) vs ReentrantLock:
  - Semaphore is NOT reentrant — same thread acquiring twice will deadlock!
  - Semaphore has no "owner" — any thread can release()
  - Use Semaphore for "resource pool" semantics, ReentrantLock for "mutual
exclusion"
```

Phaser — Dynamic Multi-Phase Synchronization

Use case: When the number of participating threads may change dynamically, or when processing has multiple phases.

```java
// Production scenario: Multi-phase ETL pipeline
public class EtlPipeline {

    public void runPipeline(List<DataFile> files) {
        Phaser phaser = new Phaser(1);  // Register self as coordinator

        for (DataFile file : files) {
            phaser.register();  // Dynamically register each worker
            executor.submit(() -> {
                try {
                    // Phase 0: Extract
                    RawData data = extract(file);
                    phaser.arriveAndAwaitAdvance();  // Wait for all
extractions

                    // Phase 1: Transform
                    TransformedData transformed = transform(data);
                    phaser.arriveAndAwaitAdvance();  // Wait for all
transformations

                    // Phase 2: Load
                    load(transformed);
                    phaser.arriveAndDeregister();    // Done, deregister
                } catch (Exception e) {
                    phaser.arriveAndDeregister();    // Deregister on failure
too
                }
            });
        }

        phaser.arriveAndDeregister();  // Coordinator deregisters
    }
}
```

Synchronization Utilities Summary

Utility	Purpose	Reusable	Typical Use Case
`CountDownLatch`	Wait for N events	No	Startup initialization
`CyclicBarrier`	Synchronize N threads at a point	Yes	Iterative parallel algorithms
`Semaphore`	Limit concurrent access	Yes	Connection pools, rate limiting
`Phaser`	Dynamic multi-phase sync	Yes	Multi-phase processing pipelines

Exchanger	Two threads swap data	Yes	Pipeline stage handoffs

7.6 JIT Lock Optimizations

The JIT compiler (HotSpot C2) automatically optimizes locking when it can prove it's safe:

```
Lock Coarsening:
----------------
JIT combines adjacent synchronized blocks on the same object:

Before optimization:            After optimization:
  synchronized(lock) { a++; }     synchronized(lock) {
  synchronized(lock) { b++; }       a++;
  synchronized(lock) { c++; }       b++;
                                    c++;
                                  }
Result: 3 lock/unlock pairs -> 1 lock/unlock pair

Lock Elision (Biased Locking):
------------------------------
JIT detects locks that can never be contended (escape analysis):

  void method() {
      StringBuilder sb = new StringBuilder();  // Thread-local object
      sb.append("Hello");                       // sb never escapes this method
      sb.append(" World");                      // sb.append is synchronized
internally
      return sb.toString();
  }

  JIT eliminates ALL synchronization on sb because it proves
  sb never escapes to another thread. Zero-cost locking!

Lock Striping (Manual technique):
---------------------------------
Split one lock into N locks, each protecting 1/N of the data:
  Before: ONE lock for entire HashMap -> all threads contend
  After:  16 locks, each for a range of buckets -> 16x less contention
  This is exactly what ConcurrentHashMap does internally.
```

Chapter 8: Java Concurrent Collections

Why Concurrent Collections Exist

Before Java 5, the only thread-safe collections were `Vector`, `Hashtable`, and `Collections.synchronizedXxx()` wrappers. These use **coarse-grained locking** — every method synchronizes on the entire collection, creating a bottleneck. Concurrent collections use **fine-grained locking**, **lock-free algorithms (CAS)**, and **copy-on-write** strategies for dramatically better performance.

Collection Type	Old Thread-Safe	Modern Concurrent	Performance Gain
Map	`Hashtable`, `Collections.synchronizedMap()`	`ConcurrentHashMap`	10-100x under contention
List	`Vector`, `Collections.synchronizedList()`	`CopyOnWriteArrayList`	Lock-free reads
Queue	(none)	`ConcurrentLinkedQueue`, `BlockingQueue`	Non-blocking or efficient blocking

8.1 ConcurrentHashMap

What is ConcurrentHashMap?

A thread-safe HashMap designed for **high concurrency**. Uses per-bucket locking (Java 8+) instead of locking the entire map.

Why ConcurrentHashMap Over Hashtable/synchronizedMap

Feature	`Hashtable` / `synchronizedMap`	`ConcurrentHashMap`
Locking	Entire map locked on every operation	Only affected bucket locked
Read performance	Blocked by any write	Lock-free reads (volatile)
Write performance	All writes sequential	Concurrent writes to different buckets
Null keys/values	`Hashtable`: no nulls; `synchronizedMap`: allows	No nulls (by design)
Iterator	Fail-fast (throws CME)	Weakly consistent (no CME)

When to Use ConcurrentHashMap

- In-memory caches, rate limiters, metrics counters
- Any map accessed by multiple threads
- When you need `computeIfAbsent`, `merge`, or other atomic compound operations

When NOT to Use ConcurrentHashMap

- Single-threaded code — use `HashMap` (no overhead)
- Need sorted keys — use `ConcurrentSkipListMap`
- Need to lock multiple operations atomically — external `synchronized` block needed

Internal Architecture (Java 8+)

```
ConcurrentHashMap table[]
+----+
| [0]| -> null
| [1]| -> Node -> Node -> null     <- synchronized on first node (bin-level
lock)
| [2]| -> TreeBin -> ...          <- synchronized on TreeBin
| [3]| -> null                   <- CAS for empty bin insertion (no lock!)
| ...|
| [n]| -> ForwardingNode         <- resize in progress
+----+

Key operations:
- Empty bin insert: CAS (Compare-And-Swap) — no lock!
- Occupied bin insert: synchronized on bin head — only this bin locked
- Read: volatile read — no lock!
```

Production Usage

```java
@Component
public class ApiRateLimiter {

    private final ConcurrentHashMap<String, AtomicLong> requestCounts =
        new ConcurrentHashMap<>();
    private final ConcurrentHashMap<String, Instant> windowStart =
        new ConcurrentHashMap<>();

    private static final int MAX_REQUESTS = 100;
    private static final Duration WINDOW = Duration.ofMinutes(1);
```

```
    public boolean isAllowed(String clientId) {
        Instant now = Instant.now();

        windowStart.compute(clientId, (key, start) -> {
            if (start == null || Duration.between(start, now).compareTo(WINDOW)
> 0) {
                requestCounts.put(clientId, new AtomicLong(0));
                return now;
            }
            return start;
        });

        long count = requestCounts.computeIfAbsent(clientId,
            k -> new AtomicLong(0)).incrementAndGet();
        return count <= MAX_REQUESTS;
    }
}
```

Atomic Compound Operations

```
ConcurrentHashMap<String, LongAdder> metrics = new ConcurrentHashMap<>();

// computeIfAbsent — atomic create-if-missing
metrics.computeIfAbsent("api.calls", k -> new LongAdder()).increment();

// merge — atomic read-modify-write
ConcurrentHashMap<String, Integer> wordCount = new ConcurrentHashMap<>();
wordCount.merge("java", 1, Integer::sum);  // Thread-safe increment

// Bulk operations (Java 8+)
long total = metrics.reduceValues(/* parallelismThreshold */ 4,
    LongAdder::sum, Long::sum);
```

8.2 CopyOnWriteArrayList

What is CopyOnWriteArrayList?

A thread-safe List where every write operation creates a **new copy** of the underlying array. Read operations are completely lock-free because they read from an immutable snapshot.

Why Copy-on-Write?

The fundamental insight is: **if the data never changes in place, reads can never be inconsistent**. By copying the entire array on every write, existing readers continue iterating the old (immutable) array while the new version is prepared.

How It Works Internally

```
State 1: array = [A, B, C]   <- all readers see this

Writer adds D:
  1. Creates new array: [A, B, C, D]
  2. Atomically swaps reference: array -> new array
  3. Old array: readers still iterating it see [A, B, C] — consistent!
  4. New readers see [A, B, C, D]

No locks needed for reads. Iterator never throws
ConcurrentModificationException.
```

When to Use

- **Read-heavy, write-rare** scenarios (e.g., event listeners, configuration lists)
- Small-to-medium sized lists
- When iterator consistency is critical (no ConcurrentModificationException)

When NOT to Use

- **Frequent writes** — each write copies the ENTIRE array (O(n) time and memory)
- **Large lists** — copying a 10,000-element array on every add is expensive
- Use `ConcurrentLinkedQueue` or `Collections.synchronizedList()` instead for write-heavy scenarios

```java
@Component
public class WebhookRegistry {

    private final CopyOnWriteArrayList<WebhookEndpoint> endpoints =
        new CopyOnWriteArrayList<>();

    // Rare write — copies entire array
    public void register(WebhookEndpoint endpoint) {
        endpoints.addIfAbsent(endpoint);
    }

    // Frequent read — no lock, no copy
    public void notifyAll(Event event) {
```

```
        for (WebhookEndpoint endpoint : endpoints) { // Safe iteration, no CME
            try {
                httpClient.post(endpoint.getUrl(), event);
            } catch (Exception e) {
                log.error("Webhook failed: {}", endpoint.getUrl(), e);
            }
        }
    }
}
```

8.3 BlockingQueue

What is BlockingQueue?

A Queue that blocks the thread on `put()` when full and on `take()` when empty. **The go-to data structure for producer-consumer patterns** in production Java applications.

Why BlockingQueue Over Manual wait/notify

BlockingQueue encapsulates all the synchronization complexity (wait/notify, while loops, lock management) into a simple `put()` and `take()` API. It eliminates the most common concurrency bugs.

When to Use Which Implementation

If You Need...	Use...	Why
Fixed-size buffer	`ArrayBlockingQueue`	Bounded, predictable memory
Unbounded or large buffer	`LinkedBlockingQueue`	Grows as needed
Priority ordering	`PriorityBlockingQueue`	Always dequeues highest priority
Direct handoff (no buffer)	`SynchronousQueue`	Producer waits for consumer
Delayed processing	`DelayQueue`	Elements available only after delay

Implementations

Implementation	Description	Bounded
`ArrayBlockingQueue`	Array-based, fixed capacity	Yes

`LinkedBlockingQueue`	Linked nodes, optional capacity	Optional
`PriorityBlockingQueue`	Priority-based ordering	No
`SynchronousQueue`	Zero-capacity, direct handoff	N/A
`DelayQueue`	Elements available only after delay	No

Key Methods

Method	Blocks?	Behavior when limit reached
`put(e)`	Yes	Blocks until space available
`take()`	Yes	Blocks until element available
`offer(e)`	No	Returns false if full
`poll()`	No	Returns null if empty
`offer(e, timeout)`	Timed	Blocks up to timeout
`poll(timeout)`	Timed	Blocks up to timeout

Production: Task Processing Queue

```
@Component
public class OrderProcessingQueue {

    private final BlockingQueue<Order> queue = new ArrayBlockingQueue<>(1000);

    // Called by REST controller — enqueues order
    public boolean submit(Order order) {
        return queue.offer(order); // Non-blocking, returns false if full
    }

    // Background processor threads
    @PostConstruct
    public void startProcessors() {
        int processors = Runtime.getRuntime().availableProcessors();
        for (int i = 0; i < processors; i++) {
            Thread processor = new Thread(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    try {
                        Order order = queue.take(); // Blocks until order
available
```

```
                        processOrder(order);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        break;
                    }
                }
            }, "order-processor-" + i);
            processor.setDaemon(true);
            processor.start();
        }
    }
}
```

8.4 ConcurrentLinkedQueue

What

An unbounded, lock-free, thread-safe queue based on **CAS (Compare-And-Swap)** operations. Non-blocking — never puts a thread to sleep.

When to Use

- High-contention producer-consumer without blocking
- When `size()` is not needed (O(n) traversal for size!)
- Event buffering in real-time systems

```
@Component
public class EventBuffer {

    private final ConcurrentLinkedQueue<AuditEvent> buffer = new
ConcurrentLinkedQueue<>();

    // Multiple producer threads — lock-free
    public void record(AuditEvent event) {
        buffer.offer(event); // Never blocks, never fails (unbounded)
    }

    // Periodic flush to database
    @Scheduled(fixedRate = 5000)
    public void flush() {
        List<AuditEvent> batch = new ArrayList<>();
        AuditEvent event;
        int count = 0;
```

```
        while ((event = buffer.poll()) != null && count < 500) {
            batch.add(event);
            count++;
        }

        if (!batch.isEmpty()) {
            auditRepository.saveAll(batch);
            log.info("Flushed {} audit events", batch.size());
        }
    }
}
```

Thread-Safe Collection Comparison

Collection	Lock Strategy	Blocking	Best For
`ConcurrentHashMap`	Per-bin lock + CAS	No	High-concurrency maps
`CopyOnWriteArrayList`	Copy-on-write	No	Read-heavy small lists
`ArrayBlockingQueue`	ReentrantLock	Yes	Bounded producer-consumer
`LinkedBlockingQueue`	Dual ReentrantLock	Yes	Unbounded producer-consumer
`ConcurrentLinkedQueue`	CAS (lock-free)	No	High-throughput non-blocking
`SynchronousQueue`	Direct handoff	Yes	Thread-to-thread handoff

8.5 ConcurrentHashMap Internals (Java 8+)

Understanding the internal structure helps you know *why* it's so fast and when to expect limitations:

```
ConcurrentHashMap Internal Structure (Java 8+):
-----------------------------------------------

Table (Node[] array):
+------+------+------+------+------+------+------+------+
| Bin 0| Bin 1| Bin 2| Bin 3| Bin 4| Bin 5| Bin 6| Bin 7|
+--+---+------+--+---+------+------+------+--+---+------+
   |             |                            |
   v             v                            v
  [K=A,V=1]    [K=C,V=3]                   [K=F,V=6]
   |             |                            |
   v             v                            v
```

```
  [K=B,V=2]    [K=D,V=4]                    [K=G,V=7]
   |                                          |
   v                                          v
  null                                 * TREEIFIED (TreeNode)
                                         if > 8 entries per bin!

Java 8 Changes from Java 7:
---------------------------
Java 7: Segment-based locking (16 fixed segments)
Java 8: Per-bin locking with CAS + synchronized on the head node

Put operation flow:
  1. Compute hash -> find bin index
  2. If bin is empty -> CAS to insert head node (no lock!)
  3. If bin is not empty -> synchronized(headNode) {
       - Traverse linked list / tree
       - Insert new node or update value
     }
  4. If bin size > 8 (TREEIFY_THRESHOLD) -> convert linked list to Red-Black
Tree
  5. If bin size < 6 (UNTREEIFY_THRESHOLD) -> convert back to linked list

Get operation (LOCK-FREE!):
  1. Compute hash -> find bin
  2. Traverse chain/tree using volatile reads
  3. No locking at all — reads are always lock-free
```

ConcurrentHashMap Pitfalls

```java
// [N] PITFALL 1: Non-atomic check-then-act
if (!map.containsKey(key)) {       // Thread A checks: not present
    map.put(key, computeValue());  // Thread B inserts between check and put!
}

// [Y] FIX: Use atomic operations
map.putIfAbsent(key, computeValue());              // Atomic
map.computeIfAbsent(key, k -> computeValue());     // Atomic, lazy computation

// [N] PITFALL 2: Size is NOT exact (weakly consistent)
int size = map.size();  // May be stale during concurrent modifications

// [N] PITFALL 3: Null keys/values not allowed (by design)
map.put(null, "value");    // NullPointerException!
map.put("key", null);      // NullPointerException!
// Reason: null is used internally as a sentinel for absent values
```

```
// [Y] BEST PRACTICE: Use compute methods for atomic read-modify-write
map.compute(key, (k, v) -> v == null ? 1 : v + 1);  // Atomic increment
map.merge(key, 1, Integer::sum);                     // Atomic merge
```

8.6 ConcurrentSkipListMap / ConcurrentSkipListSet

A **sorted**, thread-safe map implementation using skip lists instead of trees. Provides O(log n) operations with good concurrency.

```
Skip List Structure:
--------------------

Level 2:  HEAD -----------------------> 15 -----------> NULL
             |                          |
Level 1:  HEAD -----> 5 -----------> 15 -----> 25 ---> NULL
             |       |              |        |
Level 0:  HEAD -> 3 -> 5 -> 8 -> 12 -> 15 -> 25 -> 30 -> NULL

Properties:
  - Lock-free reads (CAS-based updates)
  - O(log n) get, put, remove
  - SORTED order (unlike ConcurrentHashMap)
  - Supports range queries: subMap(), headMap(), tailMap()
  - NavigableMap interface (ceiling, floor, higher, lower)
```

```
// When to use ConcurrentSkipListMap vs ConcurrentHashMap:
ConcurrentHashMap<String, Integer> hashMap;     // O(1) get/put, UNSORTED
ConcurrentSkipListMap<String, Integer> skipMap; // O(log n) get/put, SORTED

// Use ConcurrentSkipListMap when you need:
//   - Sorted iteration (by key order)
//   - Range queries (subMap, headMap, tailMap)
//   - NavigableMap operations (ceilingKey, floorKey)
//   - Concurrent sorted set (ConcurrentSkipListSet)

// Production example: Time-series data with range queries
ConcurrentSkipListMap<Instant, Metric> timeSeries = new ConcurrentSkipListMap<>
();
timeSeries.put(Instant.now(), new Metric("cpu", 85.2));

// Get all metrics from last 5 minutes (efficient range query!)
```

```
NavigableMap<Instant, Metric> recent = timeSeries.tailMap(
    Instant.now().minus(Duration.ofMinutes(5)), true);
```

8.7 Choosing the Right Concurrent Collection

```
Decision Flowchart:
-------------------

Need a Map?
  +- Need sorted keys? -> ConcurrentSkipListMap
  +- Don't need sorting? -> ConcurrentHashMap (always)

Need a List?
  +- Read-heavy, small list, rare writes? -> CopyOnWriteArrayList
  +- Write-heavy or large list? -> Use ConcurrentHashMap or synchronize manually

Need a Queue?
  +- Need blocking (producer-consumer)?
  |   +- Need bounded capacity? -> ArrayBlockingQueue
  |   +- Need priority ordering? -> PriorityBlockingQueue
  |   +- Need direct handoff (no buffer)? -> SynchronousQueue
  |   +- General purpose? -> LinkedBlockingQueue
  +- Non-blocking?
      +- ConcurrentLinkedQueue

Need a Set?
  +- Need sorted? -> ConcurrentSkipListSet
  +- Unordered? -> ConcurrentHashMap.newKeySet()
      (or Collections.newSetFromMap(new ConcurrentHashMap<>()))
```

Chapter 9: Executor Framework

What is the Executor Framework?

The **Executor Framework** (introduced in Java 5, `java.util.concurrent` package) is a higher-level replacement for manually creating and managing threads. It decouples **task submission** from **task execution**, allowing you to focus on WHAT to run, not HOW to run it.

Why the Executor Framework Exists

Before Java 5, every concurrent operation required manually creating `Thread` objects. This leads to three critical problems in production:

Problem	Impact
Resource waste	Each thread costs ~1MB memory + ~1ms creation time
No bound on concurrency	10,000 requests = 10,000 threads = OOM crash
No task queuing	When all threads are busy, new tasks are simply lost
No lifecycle management	No graceful shutdown, thread leak on exceptions

Design Pattern: Thread Pool Pattern

The Executor Framework implements the **Thread Pool** pattern — a fixed set of worker threads are created once, and tasks are submitted to a queue. Workers pull tasks from the queue and execute them.

```
Architecture:

Client Code                      Executor Framework
+-------------+              +-------------------------------+
|  Task 1     | -submit-> |  BlockingQueue         |
|  Task 2     | -submit-> |  [Task1][Task2][Task3]  |
|  Task 3     | -submit-> |           |               |
+-------------+              |  Worker-1  <--takes task  |
                              |  Worker-2  <--takes task  |
                              |  Worker-3  (idle)        |
                              +-------------------------------+
```

When to Use Executor Framework

- **Always** use it instead of raw `new Thread()` in production code

- When you need to bound concurrency (fixed number of threads)
- When tasks should be queued and processed in order
- When you need graceful shutdown capabilities

When NOT to Use

- Simple one-off background tasks — consider `CompletableFuture.runAsync()` (uses ForkJoinPool internally)
- Reactive systems — use Project Reactor or RxJava instead
- If using Spring Boot, prefer `@Async` annotation with a configured `TaskExecutor`

9.1 Why Thread Pools?

Creating raw threads for every task is expensive and dangerous:

```
Problem with raw threads:
1. Thread creation: ~1ms + ~1MB memory per thread
2. 10,000 concurrent requests = 10,000 threads = ~10GB memory -> OOM
3. OS thread limit: ~10,000-30,000 threads per process
4. Context switching overhead grows with thread count
5. No control over concurrency level

Solution: Thread Pools
- Reuse a fixed set of threads
- Queue excess tasks
- Bound concurrency
- Manage lifecycle
```

9.2 ThreadPoolExecutor Architecture

Understanding the Constructor Parameters

The `ThreadPoolExecutor` is the core implementation behind all thread pools. Understanding its 7 parameters is essential for production tuning:

```java
public ThreadPoolExecutor(
    int corePoolSize,        // Threads always kept alive
    int maximumPoolSize,     // Max threads under heavy load
    long keepAliveTime,      // How long idle non-core threads survive
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue,   // Queue for waiting tasks
```

```
    ThreadFactory threadFactory,         // Custom thread naming
    RejectedExecutionHandler handler    // What to do when queue is full
)
```

Step-by-Step: How Tasks Are Dispatched

When you submit a task, the ThreadPoolExecutor follows this exact decision tree:

```
New task arrives:
    |
    +- Active threads < corePoolSize?
    |   YES -> Create new thread, execute task immediately
    |   NO v
    |
    +- Work queue has space?
    |   YES -> Add task to queue (existing threads will pick it up)
    |   NO v
    |
    +- Active threads < maximumPoolSize?
    |   YES -> Create new thread (temporary, expires after keepAliveTime)
    |   NO v
    |
    +- Execute RejectedExecutionHandler
        +- AbortPolicy (default): throws RejectedExecutionException
        +- CallerRunsPolicy: caller thread executes the task
        +- DiscardPolicy: silently drops the task
        +- DiscardOldestPolicy: drops oldest queued task, retries
```

9.3 Thread Pool Types (Executors Factory Methods)

Choosing the Right Thread Pool Type

Thread Pool	Core Threads	Max Threads	Queue	Best For
`FixedThreadPool`	n	n	Unbounded	Predictable, steady workload
`CachedThreadPool`	0	∞	SynchronousQueue	Bursty, short-lived tasks
`SingleThreadExecutor`	1	1	Unbounded	Sequential tasks
`ScheduledThreadPool`	n	n	DelayedWorkQueue	Timed/periodic tasks

> ***Production Warning:*** *Never use* `Executors.newFixedThreadPool()` *or* `newCachedThreadPool()` *in production without understanding their dangers. Both can cause OOM — one via unbounded queues, the other via unbounded thread creation. Always create* `ThreadPoolExecutor` *directly with bounded queues.*

FixedThreadPool

```
// Fixed number of threads, unbounded queue
ExecutorService fixed = Executors.newFixedThreadPool(10);

// Internal: new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
//                                  new LinkedBlockingQueue<Runnable>());

// Use when: Predictable, steady workload
// Danger: Unbounded queue can cause OOM if tasks accumulate!
```

CachedThreadPool

```
// Creates threads as needed, reuses idle threads, removes after 60s idle
ExecutorService cached = Executors.newCachedThreadPool();

// Internal: new ThreadPoolExecutor(0, Integer.MAX_VALUE, 60L,
TimeUnit.SECONDS,
//                                  new SynchronousQueue<Runnable>());

// Use when: Many short-lived tasks with variable load
// Danger: Can create unlimited threads -> OOM under burst traffic!
```

SingleThreadExecutor

```
// Single thread, unbounded queue, guarantees sequential execution
ExecutorService single = Executors.newSingleThreadExecutor();

// Use when: Tasks must execute in order (event logging, file writing)
// If thread dies, a new one is created automatically
```

ScheduledThreadPool

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(5);

// One-time delayed execution
scheduler.schedule(() -> sendReminder(), 30, TimeUnit.MINUTES);

// Fixed-rate: execute every N seconds (regardless of task duration)
scheduler.scheduleAtFixedRate(() -> collectMetrics(), 0, 10, TimeUnit.SECONDS);

// Fixed-delay: wait N seconds AFTER previous execution completes
scheduler.scheduleWithFixedDelay(() -> syncData(), 0, 30, TimeUnit.SECONDS);
```

9.4 Production Thread Pool Configuration

```
@Configuration
public class ExecutorConfig {

    @Bean("ioExecutor")
    public ExecutorService ioExecutor() {
        int coreCount = Runtime.getRuntime().availableProcessors();
        return new ThreadPoolExecutor(
            coreCount * 2,              // IO tasks: 2x cores
            coreCount * 4,              // Max under burst
            60L, TimeUnit.SECONDS,
            new LinkedBlockingQueue<>(500),  // Bounded queue!
            new ThreadFactory() {
                private final AtomicInteger count = new AtomicInteger(0);
                public Thread newThread(Runnable r) {
                    Thread t = new Thread(r, "io-worker-" +
count.incrementAndGet());
                    t.setDaemon(true);
                    return t;
                }
            },
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure
        );
    }

    @Bean("cpuExecutor")
    public ExecutorService cpuExecutor() {
        int coreCount = Runtime.getRuntime().availableProcessors();
```

```java
        return new ThreadPoolExecutor(
            coreCount,                  // CPU tasks: exactly cores
            coreCount,                  // No extra threads for CPU work
            0L, TimeUnit.SECONDS,
            new LinkedBlockingQueue<>(100),
            r -> {
                Thread t = new Thread(r, "cpu-worker-" + Thread.activeCount());
                t.setDaemon(true);
                return t;
            },
            new ThreadPoolExecutor.AbortPolicy()
        );
    }

    // Graceful shutdown on application stop
    @PreDestroy
    public void shutdown() {
        ioExecutor().shutdown();
        cpuExecutor().shutdown();
        try {
            if (!ioExecutor().awaitTermination(30, TimeUnit.SECONDS)) {
                ioExecutor().shutdownNow();
            }
        } catch (InterruptedException e) {
            ioExecutor().shutdownNow();
        }
    }
}
```

Thread Pool Sizing Guidelines

Workload	Formula	Rationale
CPU-bound	`cores`	More threads = more context switching, no benefit
IO-bound	`cores x 2` to `cores x 10`	Threads are mostly waiting on IO
Mixed	`cores x (1 + wait_time/compute_time)`	Balance CPU usage
Web server	`100-400` (Tomcat default: 200)	Most time waiting on DB/network

Rejection Policies — What Happens When Pool and Queue Are Full

```
All 4 Built-in Rejection Policies:
----------------------------------

1. AbortPolicy (DEFAULT)
   -> Throws RejectedExecutionException
   -> Use when: task loss is unacceptable, caller must know about failure
   -> Problem: can crash caller if not caught

2. CallerRunsPolicy
   -> The submitting thread runs the task itself
   -> Use when: natural backpressure is desired
   -> Effect: slows down the caller, prevents task loss
   -> * RECOMMENDED for most production systems

3. DiscardPolicy
   -> Silently drops the task
   -> Use when: task is expendable (metrics, analytics)
   -> Problem: silent data loss — hard to debug

4. DiscardOldestPolicy
   -> Drops the oldest task in the queue, retries current task
   -> Use when: latest data matters most (real-time updates)
   -> Problem: may discard important long-waiting tasks
```

```java
// Custom rejection handler for production (log + metrics)
public class MonitoredRejectionPolicy implements RejectedExecutionHandler {
    private final LongAdder rejectedCount = new LongAdder();

    @Override
    public void rejectedExecution(Runnable r, ThreadPoolExecutor executor) {
        rejectedCount.increment();
        log.warn("Task rejected! Pool: {} active, Queue: {} pending",
            executor.getActiveCount(), executor.getQueue().size());

        if (!executor.isShutdown()) {
            // CallerRunsPolicy as fallback — natural backpressure
            r.run();
        }
    }

    public long getRejectedCount() { return rejectedCount.sum(); }
}
```

Thread Pool Monitoring with Micrometer (Production Essential)

```java
@Configuration
public class ThreadPoolMetrics {

    @Bean
    public MeterBinder threadPoolMetrics(@Qualifier("ioExecutor")
ThreadPoolExecutor pool) {
        return registry -> {
            Gauge.builder("threadpool.active", pool,
ThreadPoolExecutor::getActiveCount)
                 .tag("pool", "io").register(registry);
            Gauge.builder("threadpool.queue.size", pool, e ->
e.getQueue().size())
                 .tag("pool", "io").register(registry);
            Gauge.builder("threadpool.pool.size", pool,
ThreadPoolExecutor::getPoolSize)
                 .tag("pool", "io").register(registry);
            FunctionCounter.builder("threadpool.completed", pool,
                 ThreadPoolExecutor::getCompletedTaskCount)
                 .tag("pool", "io").register(registry);
        };
    }
}

// Key Alerts to Set:
// +- queue.size > 80% of capacity -> pool exhaustion approaching
// +- active == maxPoolSize for > 1 min -> pool fully loaded
// +- rejected count increasing -> pool is undersized or has a leak
// +- completed task rate dropping -> possible deadlock or slow tasks
```

Virtual Threads (Java 21+) and the Future of Executors

```java
// Traditional thread pool for IO:
ExecutorService ioPool = Executors.newFixedThreadPool(200);  // 200 OS threads

// Virtual thread executor (Java 21+):
ExecutorService virtualPool = Executors.newVirtualThreadPerTaskExecutor();
// Creates a NEW virtual thread for EACH task — millions possible
// No need to size the pool — virtual threads are cheap (~KB each)

// Spring Boot 3.2+ with virtual threads:
// application.properties:
```

```
//   spring.threads.virtual.enabled=true
// All Tomcat threads + @Async threads become virtual threads automatically!
```

```
When to switch to virtual threads:
  [Y] IO-bound workloads (REST calls, DB queries, file IO)
  [Y] High-concurrency servers (>1000 concurrent requests)
  [N] CPU-bound workloads (still need platform thread pools)
  [N] Code using synchronized heavily (virtual threads pin to carrier)
     -> Migrate synchronized -> ReentrantLock before using virtual threads
```

Chapter 10: CompletableFuture (Java 8+)

10.1 What is CompletableFuture?

`CompletableFuture<T>` is a **non-blocking**, **composable**, **chainable** framework for asynchronous programming. It represents a future result that can be completed manually or by a computation.

Why CompletableFuture Over Future

The old `Future` interface (Java 5) was very limited:

Limitation of Future	CompletableFuture Solution
`get()` blocks the calling thread	`thenApply()`, `thenAccept()` — non-blocking callbacks
Cannot chain operations	`thenCompose()`, `thenCombine()` — fluent chaining
No exception handling	`exceptionally()`, `handle()` — inline error handling
Cannot combine multiple futures	`allOf()`, `anyOf()` — parallel orchestration
No manual completion	`complete()`, `completeExceptionally()`

When to Use CompletableFuture

- Calling multiple microservices in parallel and combining results
- Non-blocking API responses (reactive-style programming)
- Any scenario where you need to chain async operations

When NOT to Use

- Simple fire-and-forget tasks — use `@Async` in Spring Boot
- CPU-bound parallel processing on large datasets — use parallel streams or Fork/Join
- Streaming data processing — use Project Reactor or Kafka Streams

Step-by-Step Execution Flow

```
1. supplyAsync(() -> callServiceA())  -> Task submitted to ForkJoinPool
2. .thenApply(result -> transform(result))  -> Callback registered (NOT executed
yet)
3. .thenCompose(data -> callServiceB(data))  -> Another async callback
registered
4. .exceptionally(ex -> fallback())  -> Error handler registered
```

```
Execution happens ONLY when the pool thread processes Step 1.
Steps 2-4 are chained as callbacks — they execute automatically when Step 1
completes.
```

Design Pattern: Pipeline / Chain of Responsibility

CompletableFuture implements the **Pipeline pattern** — data flows through a chain of transformations, with each stage potentially asynchronous.

Key API Mental Model

Method	Analogy	Returns
`thenApply()`	`map()` in Streams	`CompletableFuture<U>`
`thenCompose()`	`flatMap()` in Streams	`CompletableFuture<U>`
`thenCombine()`	Joining two parallel paths	`CompletableFuture<V>`
`thenAccept()`	`forEach()` in Streams	`CompletableFuture<Void>`
`allOf()`	`Promise.all()` in JavaScript	`CompletableFuture<Void>`
`anyOf()`	`Promise.race()` in JavaScript	`CompletableFuture<Object>`

10.2 Creating CompletableFutures

supplyAsync – Returns a value

```java
// Uses ForkJoinPool.commonPool() by default
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    // Runs in ForkJoinPool thread
    return callExternalApi();
});

// With custom executor
CompletableFuture<String> future = CompletableFuture.supplyAsync(
    () -> callExternalApi(),
    customExecutor  // Better for IO-bound tasks!
);
```

runAsync – No return value

```
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
    sendNotification(user);
});
```

10.3 Transformation and Chaining

thenApply – Transform result (like map)

```
CompletableFuture<String> nameFuture =
    CompletableFuture.supplyAsync(() -> getUserById(123))  // User
        .thenApply(user -> user.getFirstName() + " " + user.getLastName()); //
String

// Chain: supplyAsync -> User -> thenApply -> String
```

thenCompose – Chaining async operations (like flatMap)

```
// When each step returns a CompletableFuture
CompletableFuture<OrderDetails> detailsFuture =
    CompletableFuture.supplyAsync(() -> getOrderId(request))     // String
        .thenCompose(orderId ->
            CompletableFuture.supplyAsync(() -> fetchOrder(orderId)))  // Order
        .thenCompose(order ->
            CompletableFuture.supplyAsync(() -> enrichWithDetails(order))); //
OrderDetails

// thenApply vs thenCompose:
// thenApply:   f(T) -> U              (synchronous transformation)
// thenCompose: f(T) -> CompletableFuture<U>  (asynchronous chaining)
```

thenCombine – Combine two independent futures

```
CompletableFuture<UserProfile> profileFuture =
    CompletableFuture.supplyAsync(() -> userService.getProfile(userId));
```

```
CompletableFuture<List<Order>> ordersFuture =
    CompletableFuture.supplyAsync(() -> orderService.getOrders(userId));

CompletableFuture<DashboardDTO> dashboardFuture = profileFuture
    .thenCombine(ordersFuture, (profile, orders) ->
        new DashboardDTO(profile, orders));
```

10.4 Combining Multiple Futures

allOf – Wait for ALL to complete

```
CompletableFuture<UserProfile> userFuture =
    CompletableFuture.supplyAsync(() -> userService.getProfile(userId),
ioExecutor);
CompletableFuture<List<Order>> ordersFuture =
    CompletableFuture.supplyAsync(() -> orderService.getRecent(userId),
ioExecutor);
CompletableFuture<WalletBalance> walletFuture =
    CompletableFuture.supplyAsync(() -> walletService.getBalance(userId),
ioExecutor);

// Wait for ALL to complete
CompletableFuture<Void> allComplete = CompletableFuture.allOf(
    userFuture, ordersFuture, walletFuture
);

// After all complete, build the response
DashboardDTO dashboard = allComplete.thenApply(v -> new DashboardDTO(
    userFuture.join(),     // Already completed, non-blocking
    ordersFuture.join(),
    walletFuture.join()
)).join();
```

anyOf – Wait for FIRST to complete

```
// Call multiple mirrors, use the fastest response
CompletableFuture<Object> fastest = CompletableFuture.anyOf(
    CompletableFuture.supplyAsync(() -> callMirror1()),
    CompletableFuture.supplyAsync(() -> callMirror2()),
    CompletableFuture.supplyAsync(() -> callMirror3())
);
```

```
String result = (String) fastest.join(); // Fastest mirror wins
```

10.5 Exception Handling

exceptionally – Handle exceptions

```
CompletableFuture<UserProfile> resilientFuture =
    CompletableFuture.supplyAsync(() -> userService.getProfile(userId))
        .exceptionally(ex -> {
            log.error("Failed to fetch profile", ex);
            return UserProfile.defaultProfile(); // Fallback
        });
```

handle – Handle result OR exception

```
CompletableFuture<ApiResponse> responseFuture =
    CompletableFuture.supplyAsync(() -> callExternalApi())
        .handle((result, exception) -> {
            if (exception != null) {
                return ApiResponse.error(exception.getMessage());
            }
            return ApiResponse.success(result);
        });
```

whenComplete – Side-effect after completion (no transformation)

```
CompletableFuture.supplyAsync(() -> processPayment(order))
    .whenComplete((result, exception) -> {
        if (exception != null) {
            alertService.sendAlert("Payment failed: " + exception);
        } else {
            metricsService.recordPayment(result);
        }
    });
```

10.6 Full Production Example: Microservice Aggregation

```java
@Service
public class OrderAggregationService {

    @Autowired @Qualifier("ioExecutor")
    private ExecutorService ioExecutor;

    public CompletableFuture<OrderDetailsResponse> getOrderDetails(Long
orderId) {

        // Step 1: Fetch order (required)
        CompletableFuture<Order> orderFuture = CompletableFuture
            .supplyAsync(() -> orderService.findById(orderId), ioExecutor);

        // Step 2: In parallel, fetch customer AND payment details
        CompletableFuture<Customer> customerFuture = orderFuture
            .thenComposeAsync(order -> CompletableFuture
                .supplyAsync(() ->
customerService.findById(order.getCustomerId()), ioExecutor));

        CompletableFuture<List<Payment>> paymentsFuture = CompletableFuture
            .supplyAsync(() -> paymentService.findByOrderId(orderId),
ioExecutor);

        CompletableFuture<ShippingInfo> shippingFuture = CompletableFuture
            .supplyAsync(() -> shippingService.getStatus(orderId), ioExecutor)
            .exceptionally(ex -> ShippingInfo.unknown()); // Graceful fallback

        // Step 3: Combine all results
        return CompletableFuture.allOf(orderFuture, customerFuture,
paymentsFuture, shippingFuture)
            .thenApply(v -> OrderDetailsResponse.builder()
                .order(orderFuture.join())
                .customer(customerFuture.join())
                .payments(paymentsFuture.join())
                .shipping(shippingFuture.join())
                .build())
            .exceptionally(ex -> {
                log.error("Failed to build order details for {}", orderId, ex);
                throw new OrderDetailsException("Failed to aggregate order
details", ex);
            });
```

```
    }
}
```

CompletableFuture Timeout Handling (Java 9+)

```
// orTimeout — throws TimeoutException if not completed within duration
CompletableFuture<ApiResponse> response = CompletableFuture
    .supplyAsync(() -> callSlowApi(), ioExecutor)
    .orTimeout(5, TimeUnit.SECONDS);  // Throws TimeoutException after 5s

// completeOnTimeout — provides a default value on timeout
CompletableFuture<ApiResponse> response = CompletableFuture
    .supplyAsync(() -> callSlowApi(), ioExecutor)
    .completeOnTimeout(ApiResponse.defaultValue(), 5, TimeUnit.SECONDS);

// Production pattern: timeout + fallback + logging
CompletableFuture<UserProfile> profile = CompletableFuture
    .supplyAsync(() -> userService.getProfile(userId), ioExecutor)
    .completeOnTimeout(UserProfile.anonymous(), 3, TimeUnit.SECONDS)
    .whenComplete((result, ex) -> {
        if (ex != null) {
            log.warn("Profile fetch failed for user {}: {}", userId,
ex.getMessage());
            metrics.counter("profile.fetch.failure").increment();
        }
    });
```

CompletableFuture Anti-Patterns

```
// [N] ANTI-PATTERN 1: Using join()/get() inside a CompletableFuture chain
CompletableFuture.supplyAsync(() -> {
    String result = otherFuture.join();  // BLOCKS the ForkJoinPool thread!
    return process(result);
});
// FIX: Use thenCompose() to chain dependent futures

// [N] ANTI-PATTERN 2: Using common pool for IO-bound tasks
CompletableFuture.supplyAsync(() -> dbQuery());  // Blocks FJP common pool!
// FIX: Always pass custom executor for IO tasks
CompletableFuture.supplyAsync(() -> dbQuery(), ioExecutor);

// [N] ANTI-PATTERN 3: Ignoring exceptions
```

```
CompletableFuture.supplyAsync(() -> riskyOperation())
    .thenApply(r -> transform(r));  // Exception is silently swallowed!
// FIX: Always add exception handling
    .exceptionally(ex -> {
        log.error("Operation failed", ex);
        return fallbackValue();
    });

// [N] ANTI-PATTERN 4: Not understanding *Async variants
thenApply(fn)       // Runs fn on SAME thread that completed the previous stage
thenApplyAsync(fn)  // Runs fn on ForkJoinPool.commonPool()
thenApplyAsync(fn, executor)  // Runs fn on custom executor <- PREFER THIS
// In production, always use the *Async variant with a custom executor
// to avoid unpredictable thread behavior
```

Chapter 11: Fork/Join Framework

11.1 What is Fork/Join?

The Fork/Join framework (Java 7+) is designed for **divide-and-conquer** parallelism. It breaks a large task into smaller subtasks, executes them in parallel, and combines the results.

Why Fork/Join Over Regular Thread Pools

Regular thread pools work well for independent tasks. But when a task needs to **recursively split itself** into subtasks and then **combine results**, a regular thread pool deadlocks — parent tasks block waiting for child results, but child tasks are stuck in the queue.

Fork/Join solves this with the **work-stealing algorithm** — idle threads steal work from busy threads' queues, preventing deadlock and maximizing CPU utilization.

When to Use Fork/Join

- **Large array/collection processing** — summing, sorting, searching
- **Recursive divide-and-conquer** algorithms (merge sort, quicksort)
- CPU-intensive computations that can be split evenly

When NOT to Use Fork/Join

- **IO-bound tasks** — blocked threads waste the pool
- **Small datasets** — splitting overhead exceeds parallel benefit
- **Uneven task sizes** — one large subtask blocks everything
- In most cases, **parallel streams** are simpler and use Fork/Join internally

Architecture

```
ForkJoinPool
|
+- RecursiveTask<V>  -> returns a value (like Callable)
+- RecursiveAction   -> no return value (like Runnable)

Each worker thread has its own deque (double-ended queue):
- Takes tasks from the HEAD of its own deque
- Steals tasks from the TAIL of other threads' deques
```

Core Components

- **ForkJoinPool** — Specialized thread pool with work-stealing
- **RecursiveTask<V>** — Fork/join task that returns a result
- **RecursiveAction** — Fork/join task with no result (void)
- **Work Stealing** — Idle threads steal tasks from busy threads' queues

Work-Stealing Algorithm

```
Thread 1 queue:  [Task-A] [Task-B] [Task-C]     <- Takes from head
Thread 2 queue:  [Task-D]                        <- Done with its tasks
Thread 3 queue:  [Task-E] [Task-F] [Task-G]

Work Stealing:
Thread 2 is idle -> steals Task-G from Thread 3's tail

Thread 1 queue:  [Task-A] [Task-B] [Task-C]
Thread 2 queue:  [Task-G]                        <- Stolen from Thread 3!
Thread 3 queue:  [Task-E] [Task-F]               <- Task-G removed from tail
```

11.2 RecursiveTask Example: Parallel Sum

```java
public class ParallelSumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000;
    private final long[] array;
    private final int start;
    private final int end;

    public ParallelSumTask(long[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        int length = end - start;

        // Base case: small enough to compute directly
        if (length <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) {
                sum += array[i];
            }
            return sum;
```

```
        }

        // Divide: split into two subtasks
        int mid = start + length / 2;
        ParallelSumTask leftTask = new ParallelSumTask(array, start, mid);
        ParallelSumTask rightTask = new ParallelSumTask(array, mid, end);

        leftTask.fork();    // Submit left task to ForkJoinPool
        Long rightResult = rightTask.compute();  // Compute right in current
thread
        Long leftResult = leftTask.join();       // Wait for left result

        // Combine results
        return leftResult + rightResult;
    }
}

// Usage
long[] data = new long[10_000_000]; // 10 million elements
Arrays.fill(data, 1L);

ForkJoinPool pool = new ForkJoinPool(); // Default: cores threads
Long result = pool.invoke(new ParallelSumTask(data, 0, data.length));
System.out.println("Sum: " + result); // 10,000,000
```

11.3 RecursiveAction Example: Parallel File Processing

```
public class FileProcessorAction extends RecursiveAction {
    private static final int THRESHOLD = 10;
    private final List<File> files;

    public FileProcessorAction(List<File> files) {
        this.files = files;
    }

    @Override
    protected void compute() {
        if (files.size() <= THRESHOLD) {
            // Process files sequentially
            for (File file : files) {
                processFile(file);
            }
        } else {
            int mid = files.size() / 2;
```

```
            FileProcessorAction left = new FileProcessorAction(files.subList(0,
mid));
            FileProcessorAction right = new
FileProcessorAction(files.subList(mid, files.size()));
            invokeAll(left, right);  // Fork both, wait for both
        }
    }

    private void processFile(File file) {
        // Parse, transform, index, etc.
        log.info("Processing {} on {}", file.getName(),
Thread.currentThread().getName());
    }
}
```

Chapter 12: Parallel Streams

What are Parallel Streams?

Parallel Streams allow you to process collections using **multiple threads automatically** with a single API change: `.stream()` -> `.parallelStream()`. Under the hood, parallel streams use the **Fork/Join framework** to split, process, and combine data.

Why Parallel Streams Exist

Parallel streams make data parallelism accessible without manually managing threads, pools, or synchronization. They are ideal for **data-parallel** operations where the same transformation is applied to every element independently.

How Parallel Streams Work Internally

```
1. The stream source provides a Spliterator
2. The Spliterator splits the data into roughly equal chunks
3. Each chunk is wrapped in a ForkJoinTask
4. Tasks are submitted to ForkJoinPool.commonPool()
5. Worker threads process chunks independently
6. Results are combined using the Collector's combiner function

   Source:     [1, 2, 3, 4, 5, 6, 7, 8]
   Split:      [1,2,3,4]   [5,6,7,8]
   Split:      [1,2][3,4]  [5,6][7,8]
   Process:     T-1  T-2    T-3  T-4
   Combine:       10          26
   Final:            36
```

> ***Critical Warning:*** *Parallel streams use* `ForkJoinPool.commonPool()`*, which is shared by the* ***entire JVM****. If your parallel stream blocks (e.g., on I/O), it starves ALL other parallel streams in the application.*

Decision Guide: When to Use Parallel Streams

Condition	Use Parallel?	Why
Dataset > 100K elements AND CPU-bound	**Yes**	Meaningful speedup

Dataset < 10K elements	**No**	Split/merge overhead > benefit
I/O operations (REST, DB)	**No**	Blocks shared pool
Order-dependent logic	**Caution**	May break ordering
Shared mutable state	**NEVER**	Race conditions
ArrayList source	**Yes**	Good random-access splitting
LinkedList source	**No**	Poor splitting (sequential only)

12.1 stream() vs parallelStream()

```java
List<Order> orders = orderRepository.findAll(); // 100,000 orders

// Sequential stream — single thread
long count = orders.stream()
    .filter(o -> o.getAmount() > 100)
    .count();

// Parallel stream — uses ForkJoinPool.commonPool()
long count = orders.parallelStream()
    .filter(o -> o.getAmount() > 100)
    .count();
```

12.2 How Parallel Streams Work

```
parallelStream() internally:
1. Spliterator splits the source into chunks
2. Each chunk is submitted to ForkJoinPool.commonPool()
3. Each thread processes its chunk independently
4. Results are combined using the collector's combiner

Data:  [1, 2, 3, 4, 5, 6, 7, 8]
       v split
[1, 2, 3, 4]    [5, 6, 7, 8]
  v split          v split
[1,2] [3,4]    [5,6] [7,8]
  v     v        v     v
 T-1   T-2      T-3   T-4    <- Each thread processes its chunk
  v     v        v     v
```

```
3       7        11      15      <- Partial results
 v      v         v      v
   10                26          <- Combiner merges results
        v
        36                       <- Final result
```

12.3 When to Use Parallel Streams

[Y] Good Candidates

```java
// 1. CPU-heavy computation on large datasets
List<Double> results = largeDataset.parallelStream()
    .map(data -> performHeavyCalculation(data)) // CPU-bound
    .collect(Collectors.toList());

// 2. Embarrassingly parallel operations (no shared state)
long total = sales.parallelStream()
    .mapToDouble(Sale::getAmount)
    .sum();
```

[N] When NOT to Use

```java
// 1. Small datasets — overhead > benefit
List<String> small = List.of("a", "b", "c");
small.parallelStream().map(String::toUpperCase); // SLOWER than sequential!

// 2. IO-bound operations — blocks ForkJoinPool threads
orders.parallelStream()
    .map(order -> restTemplate.getForObject(url, Response.class)) // BAD!
    .collect(Collectors.toList());
// Blocks commonPool threads, starving other parallel streams!

// 3. Ordered operations where order matters
// parallelStream may not maintain encounter order for unordered sources

// 4. Operations with shared mutable state
List<String> results = new ArrayList<>(); // NOT thread-safe!
data.parallelStream()
    .map(this::transform)
    .forEach(results::add); // RACE CONDITION!
```

```
// FIX: Use collect() instead
List<String> results = data.parallelStream()
    .map(this::transform)
    .collect(Collectors.toList()); // Thread-safe collector
```

12.4 Custom ForkJoinPool for Parallel Streams

```
// Don't pollute the common pool with IO-bound tasks
ForkJoinPool customPool = new ForkJoinPool(20); // 20 threads for IO

List<ApiResponse> results = customPool.submit(() ->
    urls.parallelStream()
        .map(url -> callApi(url))
        .collect(Collectors.toList())
).get();

customPool.shutdown();
```

Guidelines

Scenario	Use Parallel?	Reason
< 10,000 elements	No	Split/merge overhead exceeds benefit
CPU-bound, >100K elements	**Yes**	Significant speedup
IO-bound (REST/DB)	**No**	Blocks common pool
Ordered output critical	Caution	May need `forEachOrdered()`
Shared mutable state	**No**	Race conditions
ArrayList source	**Yes**	Good spliterator (random access)
LinkedList source	**No**	Poor spliterator (sequential access)

The NxQ Model for Parallel Stream Decisions

Doug Lea (creator of `java.util.concurrent`) proposed the **NxQ > 10,000** rule:

```
N = number of elements
Q = cost of each operation (in CPU cycles)
```

```
Use parallel stream ONLY when N x Q > 10,000

Examples:
  10 elements x filter (Q=10)     = 100        -> SEQUENTIAL
  1,000 elements x sort (Q=100)   = 100,000    -> PARALLEL [Y]
  100,000 elements x map (Q=5)    = 500,000    -> PARALLEL [Y]
  50 elements x ML inference (Q=10,000) = 500K  -> PARALLEL [Y]

How to estimate Q:
  - Simple filter/map: Q ~= 1-10
  - String operations: Q ~= 10-100
  - Parsing/serialization: Q ~= 100-1,000
  - Complex computations: Q ~= 1,000-100,000
```

Source Data Structure Spliterator Quality

```
Data Source Spliterator Quality (affects parallel stream performance):
----------------------------------------------------------------------

Excellent spliterators (split evenly, random access):
  - ArrayList, arrays -> split in O(1) at midpoint
  - IntStream.range()  -> split in O(1)
  - TreeMap, HashSet   -> reasonable splitting

Poor spliterators (sequential access only):
  - LinkedList         -> must traverse to find midpoint O(n)
  - Stream.iterate()   -> cannot split at all (processes sequentially)
  - BufferedReader.lines() -> poor splitting
  - Custom Iterators   -> depends on implementation

If you have a custom data source:
  Implement your own Spliterator for optimal parallel performance.
```

```java
// Custom Spliterator example for a time-series database
public class TimeSeriesSpliterator implements Spliterator<DataPoint> {
    private final DataPoint[] data;
    private int current;
    private int end;

    @Override
    public Spliterator<DataPoint> trySplit() {
        int mid = (current + end) >>> 1;  // Split in half
```

```java
        if (mid == current) return null;  // Too small to split
        TimeSeriesSpliterator prefix = new TimeSeriesSpliterator(data, current,
mid);
        current = mid;
        return prefix;
    }

    @Override
    public long estimateSize() { return end - current; }

    @Override
    public int characteristics() {
        return SIZED | SUBSIZED | ORDERED | NONNULL | IMMUTABLE;
    }

    @Override
    public boolean tryAdvance(Consumer<? super DataPoint> action) {
        if (current < end) {
            action.accept(data[current++]);
            return true;
        }
        return false;
    }
}

// Usage: create a parallel stream from custom spliterator
Stream<DataPoint> parallelTimeSeries = StreamSupport.stream(
    new TimeSeriesSpliterator(data, 0, data.length), true  // true = parallel
);
```

Fork/Join: Threshold Selection Guidelines

Choosing the right threshold (when to stop splitting and process sequentially) is critical for Fork/Join performance:

```
Threshold Selection Rules of Thumb:
-----------------------------------

1. Start with: threshold = N / (parallelism x 4)
   where N = total elements, parallelism = available cores
   Example: 1,000,000 elements on 8 cores -> threshold ~= 31,250

2. Benchmark with different thresholds:
   Too high:  not enough parallelism (few tasks, some cores idle)
   Too low:   too many tasks (overhead of forking exceeds computation)
   Sweet spot: tasks take 100us to 10ms each
```

```
3. Consider the cost of your computation:
   Cheap operations (sum, filter): threshold = 10,000 - 100,000
   Expensive operations (crypto, ML): threshold = 10 - 100

4. ManagedBlocker for blocking inside ForkJoinPool:
   If your task MUST block (e.g., waiting for IO), use ManagedBlocker
   to tell the pool to create a compensation thread.
```

```java
// ManagedBlocker: safely block inside ForkJoinPool
ForkJoinPool.managedBlock(new ForkJoinPool.ManagedBlocker() {
    private boolean done = false;

    @Override
    public boolean block() throws InterruptedException {
        // This blocks — but ForkJoinPool knows and can compensate
        result = blockingApiCall();
        done = true;
        return true;
    }

    @Override
    public boolean isReleasable() {
        return done;
    }
});
// ForkJoinPool creates a temporary compensation thread so other
// tasks don't starve while this task blocks.
```

Chapter 13: Deadlocks

13.1 What is a Deadlock?

A **deadlock** is a situation where two or more threads are **permanently blocked**, each waiting for a lock held by the other.

Why Deadlocks Are Critical

- The application **hangs silently** — no exception is thrown, no error logged
- Affects only the deadlocked threads initially, but cascades as other threads wait for the same locks
- In production, deadlocks often appear only under **high load** (when thread timing changes)
- They are **permanent** — the application will never recover without intervention

Real-World Production Scenarios

Scenario	What Happens
Money transfer between two accounts	Thread A locks Account-1, Thread B locks Account-2, both wait for the other
Distributed lock across microservices	Service A holds Lock-X, Service B holds Lock-Y, circular dependency
Database row-level locks	Transaction 1 locks Row A, Transaction 2 locks Row B, both try to update the other
Spring bean circular dependency	BeanA depends on BeanB, BeanB depends on BeanA during initialization

```
Thread A holds Lock-1, waiting for Lock-2
Thread B holds Lock-2, waiting for Lock-1
-> Neither can proceed -> Application hangs forever
```

Visualization

```
Thread A -----> [Holds Lock-1] -----> [Waiting for Lock-2] --+
                                                             |
Thread B -----> [Holds Lock-2] -----> [Waiting for Lock-1] --+
                   ^                          |
```

```
                    +-----------------------+
                         DEADLOCK CYCLE
```

13.2 Four Conditions for Deadlock (Coffman Conditions)

ALL four must hold simultaneously:

Condition	Description
Mutual Exclusion	At least one resource is held in non-sharable mode
Hold and Wait	Thread holds one resource while waiting for another
No Preemption	Resources cannot be forcibly taken from a thread
Circular Wait	Circular chain of threads, each waiting for a resource held by the next

13.3 Deadlock Code Example

```java
public class DeadlockExample {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    public void method1() {
        synchronized (lockA) {           // Thread 1 acquires lockA
            System.out.println("Thread 1: Holding lockA");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            synchronized (lockB) {       // Thread 1 waiting for lockB
                System.out.println("Thread 1: Holding lockA & lockB");
            }
        }
    }

    public void method2() {
        synchronized (lockB) {           // Thread 2 acquires lockB
            System.out.println("Thread 2: Holding lockB");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            synchronized (lockA) {       // Thread 2 waiting for lockA ->
DEADLOCK!
                System.out.println("Thread 2: Holding lockB & lockA");
            }
        }
```

```
    }
}

// Run
DeadlockExample example = new DeadlockExample();
new Thread(example::method1, "Thread-1").start();
new Thread(example::method2, "Thread-2").start();
// Both threads will hang forever!
```

13.4 Detection

Using jstack (Command Line)

```
# Find JVM process ID
jps

# Dump threads
jstack <PID>

# Output will show:
# "Thread-1": waiting to lock 0x00000007ab
#   - locked 0x00000007cd
# "Thread-2": waiting to lock 0x00000007cd
#   - locked 0x00000007ab
# Found 1 deadlock.
```

Programmatic Detection

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();
long[] deadlockedThreadIds = bean.findDeadlockedThreads();

if (deadlockedThreadIds != null) {
    ThreadInfo[] infos = bean.getThreadInfo(deadlockedThreadIds, true, true);
    for (ThreadInfo info : infos) {
        System.out.println("Deadlocked thread: " + info.getThreadName());
        System.out.println("Waiting for: " + info.getLockName());
        System.out.println("Held by: " + info.getLockOwnerName());
    }
}
```

13.5 Prevention Strategies

Strategy 1: Lock Ordering (Break Circular Wait)

```java
// ALWAYS acquire locks in the same order
public void transferMoney(Account from, Account to, double amount) {
    // Sort by account ID to ensure consistent lock ordering
    Account first = from.getId() < to.getId() ? from : to;
    Account second = from.getId() < to.getId() ? to : from;

    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
```

Strategy 2: tryLock with Timeout (Break Hold and Wait)

```java
public boolean transferWithTimeout(Account from, Account to, double amount) {
    ReentrantLock lock1 = from.getLock();
    ReentrantLock lock2 = to.getLock();

    boolean gotLock1 = false, gotLock2 = false;
    try {
        gotLock1 = lock1.tryLock(1, TimeUnit.SECONDS);
        gotLock2 = lock2.tryLock(1, TimeUnit.SECONDS);

        if (gotLock1 && gotLock2) {
            from.debit(amount);
            to.credit(amount);
            return true;
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        if (gotLock1) lock1.unlock();
        if (gotLock2) lock2.unlock();
    }
```

```
    return false; // Retry later
}
```

Strategy 3: Avoid Nested Locks

```
// Instead of nested synchronized blocks, use a single coarse-grained lock
private final ReentrantLock transferLock = new ReentrantLock();

public void transfer(Account from, Account to, double amount) {
    transferLock.lock(); // Single lock for all transfers
    try {
        from.debit(amount);
        to.credit(amount);
    } finally {
        transferLock.unlock();
    }
}
```

13.6 Livelock — The Invisible Deadlock

A **livelock** is when threads keep responding to each other but make **no progress**. Unlike a deadlock (threads are frozen), a livelock has threads actively running but doing useless work.

```
// Real-world analogy: Two people in a hallway keep stepping aside for each
other
// Both are "active" but neither makes progress

// Code Example: Livelock in a transfer system
public boolean transfer(Account from, Account to, BigDecimal amount) {
    while (true) {
        if (from.getLock().tryLock()) {
            try {
                if (to.getLock().tryLock()) {
                    try {
                        from.debit(amount);
                        to.credit(amount);
                        return true;
                    } finally {
                        to.getLock().unlock();
                    }
                }
```

```
            } finally {
                from.getLock().unlock();
            }
        }
        // Both threads reach here simultaneously, retry, clash again =
LIVELOCK!
    }
}

// FIX: Add random backoff
if (!to.getLock().tryLock()) {
    from.getLock().unlock();
    Thread.sleep(ThreadLocalRandom.current().nextInt(1, 10));  // Random
backoff
}
```

```
Deadlock vs Livelock vs Starvation:
-----------------------------------
  Deadlock:    Threads FROZEN, waiting for each other
               -> CPU idle, detected by jstack/ThreadMXBean

  Livelock:    Threads ACTIVE, but doing useless work repeatedly
               -> CPU busy, harder to detect (looks like working!)

  Starvation:  Thread never gets CPU time or lock access
               -> Usually caused by unfair locks + high-priority threads
               -> Fix: use fair locks (new ReentrantLock(true))
```

13.7 Thread Leak Detection

A **thread leak** is when threads are created but never properly terminated. Over time, the JVM runs out of OS threads and crashes.

```
// Common causes of thread leaks:
// 1. ExecutorService never shut down
// 2. Daemon threads with infinite loops and no interrupt check
// 3. Thread pools created inside methods (new pool per request!)

// Detection: Monitor thread count over time
@Scheduled(fixedRate = 60000)
public void detectThreadLeak() {
    int threadCount = Thread.activeCount();
    log.info("Active threads: {}", threadCount);
```

```
    if (threadCount > 500) {
        log.error("THREAD LEAK DETECTED! {} active threads", threadCount);
        // Take thread dump for analysis
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        ThreadInfo[] infos = bean.dumpAllThreads(true, true);
        for (ThreadInfo info : infos) {
            log.error(info.toString());
        }
    }
}
```

Deadlock Prevention Checklist (Code Review)

```
Code Review Checklist for Lock Safety:
----------------------------------------
[ ] Do all locks have a consistent global ordering?
[ ] Are all lock acquisitions wrapped in try-finally with unlock()?
[ ] Are there timeouts on all lock acquisitions (tryLock with timeout)?
[ ] Are nested locks minimized or eliminated?
[ ] Are external calls (REST, DB) made outside of lock scope?
[ ] Are thread pools properly shut down in @PreDestroy?
[ ] Are ThreadLocal values cleaned up with remove()?
[ ] Are all executors using bounded queues?
[ ] Is there monitoring for active threads and queue depth?
[ ] Are InterruptedExceptions handled properly (not swallowed)?
```

Chapter 14: Thread Safety Design Patterns

Why Thread Safety Patterns Matter

There are exactly **three strategies** to achieve thread safety. Every pattern falls into one of these categories:

Strategy	How It Works	Example Patterns
Don't share state	Each thread has its own data	ThreadLocal, immutable value objects
Share immutable state	Data cannot change after creation	Immutable objects, final fields
Synchronize access	Only one thread accesses at a time	synchronized, locks, atomic variables

> ***Architect's Principle:*** *Always prefer immutability and no-sharing over synchronization. Synchronization is the most error-prone approach.*

14.1 Immutable Objects

What are Immutable Objects?

Objects whose state **cannot change after construction**. Inherently thread-safe — no synchronization needed.

Why Use Immutable Objects for Thread Safety

- **Zero synchronization overhead** — no locks, no volatile, no atomic operations
- **No race conditions possible** — if the data cannot change, concurrent reads are always safe
- **Safe to share freely** — pass between threads, store in caches, return from methods
- **Easier to reason about** — the object you have will never change unexpectedly

When to Use

- Configuration objects, DTOs, value objects, keys in maps
- Any object shared across threads where mutation is not needed

When NOT to Use

- Objects that MUST be updated frequently (use AtomicReference or synchronized instead)
- Very large objects where copying is expensive

```java
public final class ImmutableConfig {
    private final String host;
    private final int port;
    private final List<String> endpoints;

    public ImmutableConfig(String host, int port, List<String> endpoints) {
        this.host = host;
        this.port = port;
        this.endpoints = Collections.unmodifiableList(new ArrayList<>
(endpoints));
    }

    public String getHost() { return host; }
    public int getPort() { return port; }
    public List<String> getEndpoints() { return endpoints; } // Immutable copy
}

// Thread-safe: multiple threads can read without synchronization
```

Rules for Immutability

1. Make class `final` (prevent subclassing)
2. Make all fields `private final`
3. No setter methods
4. Deep copy mutable objects in constructor
5. Return copies of mutable fields (defensive copies)

14.2 ThreadLocal

What is ThreadLocal?

Each thread gets its **own copy** of a variable. No sharing -> no synchronization needed.

Why ThreadLocal Exists

Sometimes you need per-request data (current user, transaction ID, locale) that flows through many method calls. Passing it as a parameter to every method is impractical. ThreadLocal makes it **implicitly available** to any code running on the same thread.

When to Use

- Per-request context in web applications (user identity, correlation ID)

- Database connections (connection-per-thread pattern)
- `SimpleDateFormat` instances (not thread-safe, ThreadLocal fixes it)

When NOT to Use

- **With thread pools** — ThreadLocal values leak between requests unless explicitly cleaned up
- **With async/reactive code** — tasks may switch threads mid-execution
- **For sharing data between threads** — ThreadLocal is per-thread, not shared

> ***Production Warning:*** *ALWAYS call* `ThreadLocal.remove()` *in a* `finally` *block or interceptor. In thread pools, a thread is reused for the next request — if you don't clean up, the next request sees the previous request's data. This is a* ***security vulnerability****.*

```java
public class RequestContext {

    private static final ThreadLocal<String> correlationId =
        ThreadLocal.withInitial(() -> UUID.randomUUID().toString());

    private static final ThreadLocal<UserInfo> currentUser = new ThreadLocal<>
();

    public static String getCorrelationId() {
        return correlationId.get();
    }

    public static void setCurrentUser(UserInfo user) {
        currentUser.set(user);
    }

    public static UserInfo getCurrentUser() {
        return currentUser.get();
    }

    // CRITICAL: Always clean up to avoid memory leaks in thread pools!
    public static void clear() {
        correlationId.remove();
        currentUser.remove();
    }
}

// Spring MVC interceptor
@Component
public class RequestInterceptor implements HandlerInterceptor {
```

```
    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response, Object handler) {
        RequestContext.setCurrentUser(extractUser(request));
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response, Object handler,
Exception ex) {
        RequestContext.clear(); // MUST clear to prevent leaks in thread pool!
    }
}
```

ThreadLocal Memory Leak Warning

```
⚠️ In thread pools (Tomcat, ExecutorService), threads are REUSED.
If you don't call ThreadLocal.remove(), the value persists
across requests -> memory leak + data leakage between users!

ALWAYS use try-finally or interceptors to clean up ThreadLocal.
```

14.3 Producer-Consumer Pattern

```
@Component
public class EventProcessingSystem {

    private final BlockingQueue<Event> eventQueue = new LinkedBlockingQueue<>
(10_000);

    // Producer: REST controller thread
    @PostMapping("/api/events")
    public ResponseEntity<String> receiveEvent(@RequestBody Event event) {
        if (eventQueue.offer(event)) {
            return ResponseEntity.accepted().body("Queued");
        }
        return ResponseEntity.status(429).body("Queue full");
    }

    // Consumer: Background worker threads
    @PostConstruct
```

```java
    public void startConsumers() {
        for (int i = 0; i < 5; i++) {
            Thread consumer = new Thread(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    try {
                        Event event = eventQueue.take();
                        processEvent(event);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }, "event-consumer-" + i);
            consumer.setDaemon(true);
            consumer.start();
        }
    }
}
```

14.4 Thread-Safe Singleton (Double-Checked Locking)

```java
public class DatabaseConnectionPool {

    // volatile ensures visibility + prevents instruction reordering
    private static volatile DatabaseConnectionPool instance;

    private DatabaseConnectionPool() {
        // Initialize connection pool
    }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {                    // First check (no locking)
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) {            // Second check (with lock)
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}

// Better alternative: Enum Singleton
public enum ConnectionPoolSingleton {
    INSTANCE;
```

```
    private final HikariDataSource dataSource;

    ConnectionPoolSingleton() {
        dataSource = new HikariDataSource();
        dataSource.setJdbcUrl("jdbc:postgresql://localhost:5432/db");
    }

    public Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }
}
```

14.5 Guarded Suspension

Thread waits until a condition is satisfied before proceeding.

```
public class GuardedQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();

    public T get() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                notEmpty.await(); // Guard: suspend until condition met
            }
            return queue.poll();
        } finally {
            lock.unlock();
        }
    }

    public void put(T item) {
        lock.lock();
        try {
            queue.offer(item);
            notEmpty.signal(); // Notify waiting threads
        } finally {
            lock.unlock();
        }
```

```
    }
}
```

14.6 Immutability with Java Records (Java 16+)

Java Records provide the most concise way to create immutable data carriers:

```java
// Before Records: verbose immutable class
public final class OrderEvent {
    private final String orderId;
    private final BigDecimal amount;
    private final Instant timestamp;

    public OrderEvent(String orderId, BigDecimal amount, Instant timestamp) {
        this.orderId = orderId;
        this.amount = amount;
        this.timestamp = timestamp;
    }
    // + getters, equals, hashCode, toString (50+ lines)
}

// With Records: inherently immutable, thread-safe by design
public record OrderEvent(
    String orderId,
    BigDecimal amount,
    Instant timestamp
) {
    // Compact constructor for validation:
    public OrderEvent {
        Objects.requireNonNull(orderId, "orderId must not be null");
        if (amount.compareTo(BigDecimal.ZERO) < 0)
            throw new IllegalArgumentException("amount must be >= 0");
    }
}

// Thread-safe by design:
// - All fields are final (immutable)
// - No setters (state can't change)
// - Can be freely shared between threads without synchronization
// - Perfect for event objects, DTOs, and message payloads
```

14.7 Lock-Free State Management with AtomicReference

```java
// Pattern: Atomic configuration updates without locks
public class DynamicConfig {

    // Immutable config record
    public record AppConfig(int maxRetries, Duration timeout, boolean
featureFlag) {}

    private final AtomicReference<AppConfig> config;

    public DynamicConfig(AppConfig initialConfig) {
        this.config = new AtomicReference<>(initialConfig);
    }

    // Thread-safe read (no lock needed)
    public AppConfig getConfig() {
        return config.get();  // Volatile read
    }

    // Thread-safe update (CAS, no lock needed)
    public void updateTimeout(Duration newTimeout) {
        config.updateAndGet(current ->
            new AppConfig(current.maxRetries(), newTimeout,
current.featureFlag()));
    }

    // Thread-safe conditional update
    public boolean enableFeature() {
        return config.compareAndSet(
            config.get(),  // expected
            new AppConfig(config.get().maxRetries(), config.get().timeout(),
true)
        );
    }
}

// This pattern is used by:
//  - Spring Cloud Config refresh
//  - Feature flag systems (LaunchDarkly, Unleash)
//  - Dynamic routing tables
```

14.8 Thread-Safe Builder Pattern

```
// Builder pattern is inherently thread-safe when used correctly:
// The builder itself is NOT shared — each thread creates its own builder
// The built object is immutable — safe to share across threads

public record HttpRequest(
    String url, String method, Map<String, String> headers, byte[] body
) {
    public static Builder builder() { return new Builder(); }

    public static class Builder {
        private String url;
        private String method = "GET";
        private final Map<String, String> headers = new HashMap<>();
        private byte[] body;

        public Builder url(String url) { this.url = url; return this; }
        public Builder method(String method) { this.method = method; return
this; }
        public Builder header(String k, String v) { headers.put(k, v); return
this; }
        public Builder body(byte[] body) { this.body = body; return this; }

        public HttpRequest build() {
            return new HttpRequest(
                url, method,
                Collections.unmodifiableMap(new HashMap<>(headers)),  //
Defensive copy!
                body != null ? body.clone() : null  // Defensive copy!
            );
        }
    }
}

// Thread-safety guarantee:
// Builder is created and used by a SINGLE thread (no sharing needed)
// HttpRequest record is immutable (safe to share across any threads)
```

Chapter 15: Multithreading in Spring Boot Applications

15.1 @Async – Asynchronous Method Execution

```
// Step 1: Enable async
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    @Bean("taskExecutor")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(30);
        executor.initialize();
        return executor;
    }
}

// Step 2: Use @Async on methods
@Service
public class NotificationService {

    @Async("taskExecutor") // Runs in background thread
    public void sendEmailNotification(String email, String message) {
        // This runs in a thread from taskExecutor pool
        // Does NOT block the calling thread
        emailSender.send(email, message);
        log.info("Email sent to {} on thread {}", email,
Thread.currentThread().getName());
    }

    @Async("taskExecutor")
```

```
    public CompletableFuture<EmailResult> sendEmailAsync(String email, String
message) {
        EmailResult result = emailSender.send(email, message);
        return CompletableFuture.completedFuture(result);
    }
}

// Step 3: Call from controller
@RestController
public class OrderController {

    @PostMapping("/api/orders")
    public ResponseEntity<OrderDTO> createOrder(@RequestBody CreateOrderRequest
request) {
        Order order = orderService.create(request);

        // Fire-and-forget: runs in background thread
        notificationService.sendEmailNotification(
            order.getCustomerEmail(),
            "Order " + order.getId() + " confirmed!"
        );

        return ResponseEntity.status(201).body(mapper.toDTO(order));
        // Response returns immediately, email sends in background
    }
}
```

@Async Rules

```
⚠️ @Async ONLY works when called from OUTSIDE the class.
   Self-invocation bypasses the proxy -> runs synchronously!

   // WRONG - self-invocation
   public void doWork() {
       sendEmailNotification("...", "..."); // Runs synchronously!
   }

   // CORRECT - called from another bean
   @Autowired NotificationService notificationService;
   notificationService.sendEmailNotification("...", "..."); // Async!
```

15.2 Parallel API Calls in Service Layer

```java
@Service
public class UserDashboardService {

    @Autowired @Qualifier("taskExecutor")
    private Executor executor;

    public DashboardResponse getDashboard(Long userId) {
        CompletableFuture<UserProfile> profileFuture = CompletableFuture
            .supplyAsync(() -> userClient.getProfile(userId), executor);

        CompletableFuture<List<Order>> ordersFuture = CompletableFuture
            .supplyAsync(() -> orderClient.getRecentOrders(userId), executor);

        CompletableFuture<Wallet> walletFuture = CompletableFuture
            .supplyAsync(() -> walletClient.getBalance(userId), executor);

        CompletableFuture<List<Notification>> notifFuture = CompletableFuture
            .supplyAsync(() -> notifClient.getUnread(userId), executor)
            .exceptionally(ex -> Collections.emptyList()); // Graceful
degradation

        CompletableFuture.allOf(profileFuture, ordersFuture, walletFuture,
notifFuture)
            .join();

        return DashboardResponse.builder()
            .profile(profileFuture.join())
            .recentOrders(ordersFuture.join())
            .wallet(walletFuture.join())
            .notifications(notifFuture.join())
            .build();
    }
}
```

15.3 Kafka Consumer Concurrency

```java
@Configuration
public class KafkaConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, String>
kafkaListenerFactory() {
```

```
        ConcurrentKafkaListenerContainerFactory<String, String> factory =
            new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory());
        factory.setConcurrency(3); // 3 consumer threads
        return factory;
    }
}

@Service
public class OrderEventConsumer {

    @KafkaListener(topics = "order-events", groupId = "order-processor",
                   concurrency = "5") // 5 concurrent consumer threads
    public void consume(ConsumerRecord<String, OrderEvent> record) {
        log.info("Processing order event on thread: {}",
Thread.currentThread().getName());
        orderProcessor.process(record.value());
    }
}
```

15.4 Scheduled Tasks

```
@Configuration
@EnableScheduling
public class SchedulerConfig implements SchedulingConfigurer {

    @Override
    public void configureTasks(ScheduledTaskRegistrar registrar) {
        registrar.setScheduler(schedulerExecutor());
    }

    @Bean
    public Executor schedulerExecutor() {
        return Executors.newScheduledThreadPool(5,
            r -> new Thread(r, "scheduler-" + System.nanoTime()));
    }
}

@Service
public class MaintenanceService {

    @Scheduled(fixedRate = 300000) // Every 5 minutes
    public void cleanupExpiredSessions() {
        sessionStore.removeExpired();
```

```
    }

    @Scheduled(cron = "0 0 2 * * ?") // Daily at 2 AM
    public void generateDailyReport() {
        reportService.generateAndSend();
    }
}
```

Chapter 16: Performance Optimization

16.1 Thread Pool Tuning

CPU-Bound vs IO-Bound

Workload Type	Characteristics	Pool Size Formula
CPU-Bound	Computation-heavy, rarely waits	`N = number of CPU cores`
IO-Bound	Waiting for network/disk/DB	`N = cores x (1 + W/C)`
Mixed	Both compute and IO	Separate pools for each

```
W = wait time (how long thread waits for IO)
C = compute time (how long thread uses CPU)

Example: 4-core machine, API calls that take 200ms (180ms wait, 20ms compute)
N = 4 x (1 + 180/20) = 4 x 10 = 40 threads for IO pool
```

Production Configuration

```java
@Bean("ioPool")
public ExecutorService ioPool() {
    int cores = Runtime.getRuntime().availableProcessors();
    return new ThreadPoolExecutor(
        cores * 2,                      // Core: 2x cores
        cores * 8,                      // Max: 8x cores under burst
        60, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(500), // Bounded!
        new ThreadPoolExecutor.CallerRunsPolicy()
    );
}

@Bean("cpuPool")
public ExecutorService cpuPool() {
    int cores = Runtime.getRuntime().availableProcessors();
    return new ThreadPoolExecutor(
        cores, cores,                   // Fixed size = cores
        0, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(100)
```

```
    );
}
```

16.2 Blocking vs Non-Blocking

```
// BLOCKING: Thread is idle while waiting (wastes thread)
String result = restTemplate.getForObject(url, String.class); // Thread sleeps
during IO

// NON-BLOCKING: Thread is free to do other work
WebClient.create(url)
    .get()
    .retrieve()
    .bodyToMono(String.class)
    .subscribe(result -> processResult(result)); // Callback on completion
```

16.3 Key Best Practices

```
1. NEVER use Executors.newCachedThreadPool() in production
   -> Unbounded thread creation can crash your JVM

2. ALWAYS use bounded queues
   -> LinkedBlockingQueue without capacity = unbounded = OOM risk

3. ALWAYS name your threads
   -> Debugging "pool-3-thread-7" is miserable; "order-processor-3" is clear

4. ALWAYS set a rejection policy
   -> CallerRunsPolicy provides natural backpressure

5. SEPARATE thread pools for CPU and IO tasks
   -> IO-bound tasks blocking CPU pool = starvation

6. Monitor thread pool metrics
   -> active threads, queue size, rejected tasks, completion time
```

16.4 Reactive Programming vs Multithreading

```
Comparison: When to Use Which
--------------------------------

                       Traditional Multithreading    Reactive (WebFlux/RxJava)
 Thread per request:  1 thread = 1 request           1 thread = 1000s of
requests
 Memory usage:        ~1MB per thread (stack)         ~KB per subscription
 Blocking IO:         Thread sleeps during IO         Event loop continues
 Programming model:   Imperative (familiar)           Declarative (learning
curve)
 Debugging:           Stack traces are clear           Stack traces are cryptic
 Error handling:      try-catch (familiar)             onError operators
 Ecosystem support:   Spring MVC (mature)              Spring WebFlux (growing)
 Best for:            CPU-bound, simple IO             High-concurrency IO-bound
 Throughput ceiling:  ~10K concurrent (thread limit)   ~100K+ concurrent

Decision Framework:
  1. < 500 concurrent connections -> Use traditional threads + Spring MVC
  2. 500-5000 concurrent -> Consider virtual threads (Java 21) first
  3. > 5000 concurrent IO-bound -> Consider reactive (WebFlux)
  4. CPU-bound workloads -> Always traditional threads
  5. Team familiarity matters -> Choose what your team can debug!
```

16.5 Virtual Threads Deep Dive (Project Loom, Java 21+)

```java
// Virtual threads fundamentally change the threading model:

// OLD MODEL: 1 platform thread per request (limited to ~10K concurrent)
@Configuration
public class TomcatConfig {
    // server.tomcat.threads.max=200  (OS THREAD LIMIT)
}

// NEW MODEL: 1 virtual thread per request (millions possible)
// application.properties:
// spring.threads.virtual.enabled=true  (Spring Boot 3.2+)

// What happens internally:
// 1. Virtual thread starts on a carrier (platform) thread
// 2. When virtual thread hits blocking IO (JDBC, HTTP, file read):
//    - Virtual thread is UNMOUNTED from carrier
//    - Carrier thread is FREE to run other virtual threads
// 3. When IO completes:
```

```
//    - Virtual thread is MOUNTED on any available carrier
//    - Execution continues seamlessly

// Diagram:
// Carrier threads (platform):  [C1] [C2] [C3] [C4]  (= CPU cores)
// Virtual threads:             [V1] [V2] [V3] ... [V1000000]
//
// V1 runs on C1 -> V1 blocks on IO -> V1 unmounts -> C1 picks up V47
// V1's IO completes -> V1 mounts on C3 (any free carrier) -> continues
```

```
Virtual Thread Caveats:
------------------------

1. PINNING: Virtual threads PIN to carrier when inside:
   - synchronized blocks/methods (cannot unmount!)
   - JNI/native code
   FIX: Replace synchronized with ReentrantLock

2. ThreadLocal: Works but each virtual thread gets its own copy
   Risk: 1 million virtual threads = 1 million ThreadLocal copies = OOM
   FIX: Use ScopedValue (Java 21 preview) instead of ThreadLocal

3. CPU-bound work: Virtual threads offer NO benefit
   They only help when threads spend most time WAITING (IO)

4. Thread pools: DON'T pool virtual threads
   They are cheap to create — just create a new one per task
   Use: Executors.newVirtualThreadPerTaskExecutor()
```

Chapter 17: Common Production Issues

17.1 Thread Starvation

Problem

All threads in a pool are busy (or blocked), and new tasks can't execute.

```
// Example: All 10 threads blocked on a slow database
ExecutorService pool = Executors.newFixedThreadPool(10);
for (int i = 0; i < 100; i++) {
    pool.submit(() -> {
        // Slow database query that takes 30 seconds
        jdbcTemplate.query("SELECT * FROM huge_table", rowMapper);
        // All 10 threads stuck here, remaining 90 tasks wait in queue
    });
}
```

Solution

```
// 1. Use separate pools for different task types
// 2. Add timeouts to IO operations
// 3. Use circuit breakers (Resilience4j)
// 4. Monitor queue depth and active thread count
```

17.2 Memory Leaks with ThreadLocal

```
// PROBLEM: ThreadLocal not cleaned up in thread pool
public class UserContext {
    private static final ThreadLocal<UserData> userData = new ThreadLocal<>();

    public static void set(UserData data) { userData.set(data); }
    public static UserData get() { return userData.get(); }
    // Missing: remove()!
}
```

```
// In thread pool: Thread handles Request A, sets UserData for User #1
// Thread returns to pool
// Next Request B gets the same thread -> still sees User #1's data!

// SOLUTION: Always clean up
public static void clear() { userData.remove(); }
// Call in finally block, filter, or interceptor
```

17.3 Excessive Thread Creation

```
// ANTI-PATTERN: Creating new thread for every request
@GetMapping("/api/data")
public ResponseEntity<Data> getData() {
    new Thread(() -> cacheService.refresh()).start(); // New thread every
request!
    return ResponseEntity.ok(data);
}

// FIX: Use a thread pool
@Async("taskExecutor") // Bounded, managed thread pool
public void refreshCache() { cacheService.refresh(); }
```

17.4 Blocking Operations in Reactive/Async Pipelines

```
// PROBLEM: Blocking call in CompletableFuture using common pool
CompletableFuture.supplyAsync(() -> {
    return restTemplate.getForObject(url, Data.class); // BLOCKS a ForkJoinPool
thread!
});

// FIX: Use a dedicated IO executor
CompletableFuture.supplyAsync(() -> {
    return restTemplate.getForObject(url, Data.class);
}, ioExecutor); // IO pool designed for blocking calls
```

17.5 Production Thread Debugging Toolkit

jcmd — The Modern Alternative to jstack

```
Production debugging commands:
------------------------------

$ jcmd <PID> Thread.print              # Thread dump (like jstack, but better)
$ jcmd <PID> Thread.dump_to_file <f>   # Dump to file (Java 21+)
$ jcmd <PID> VM.native_memory summary  # Native memory usage
$ jcmd <PID> GC.heap_dump <file>       # Heap dump
$ jcmd <PID> VM.uptime                 # JVM uptime
$ jcmd <PID> VM.flags                  # All JVM flags

Java Flight Recorder (JFR) — Zero-overhead production profiling:
$ jcmd <PID> JFR.start name=recording duration=60s filename=recording.jfr
$ jcmd <PID> JFR.stop name=recording

Analyze JFR recording:
$ jfr print --events jdk.ThreadPark recording.jfr   # Thread parking events
$ jfr print --events jdk.JavaMonitorWait recording.jfr  # Lock wait events

Or use JDK Mission Control (jmc) GUI for visual analysis.
```

Thread Dump Analysis Methodology (Step-by-Step)

```
5-Step Thread Dump Analysis:
------------------------------

1. TAKE MULTIPLE DUMPS (3 dumps, 10 seconds apart)
   $ for i in 1 2 3; do jstack <PID> > dump_$i.txt; sleep 10; done

2. LOOK FOR DEADLOCKS (auto-detected by jstack)
   Search for: "Found one Java-level deadlock"

3. COUNT THREAD STATES
   $ grep "java.lang.Thread.State:" dump_1.txt | sort | uniq -c
     150 RUNNABLE          <- threads doing work or blocking on IO
      35 WAITING (parking)  <- idle pool threads (normal)
      15 BLOCKED            <- waiting for locks (problem if high)
       5 TIMED_WAITING      <- sleeping or waiting with timeout

4. IDENTIFY STUCK THREADS
   Compare dump_1, dump_2, dump_3:
   Thread in SAME STATE + SAME STACK in ALL 3 dumps = STUCK
```

```
5. FIND THE ROOT CAUSE
   For BLOCKED threads: follow the chain
     Thread-3 waiting for lock -> held by Thread-1 -> Thread-1 is doing what?
   For RUNNABLE but stuck (IO):
     Look at bottom of stack: SocketInputStream.read() = waiting on network
```

Connection Pool Starvation Diagnosis

```java
// Symptom: All requests timeout, but CPU is low
// Root cause: All HikariCP connections are checked out, new requests wait
forever

// Detection: HikariCP metrics
@Bean
public MeterBinder hikariMetrics(HikariDataSource ds) {
    return registry -> {
        Gauge.builder("hikari.active", ds,
HikariDataSource::getHikariPoolMXBean)
             .description("Active connections").register(registry);
        Gauge.builder("hikari.pending", ds.getHikariPoolMXBean(),
             HikariPoolMXBean::getThreadsAwaitingConnection)
             .description("Threads waiting for connection").register(registry);
    };
}

// Prevention:
// spring.datasource.hikari.maximum-pool-size=20
// spring.datasource.hikari.connection-timeout=5000  (fail fast, don't hang)
// spring.datasource.hikari.leak-detection-threshold=10000  (detect leaks!!)
//
// When leak-detection-threshold is set, HikariCP logs a WARNING if a
connection
// is held for more than 10 seconds — this catches forgotten close() calls.
```

Chapter 18: Build It Yourself — Async Order Processing System

> ***Goal:*** *Build a Spring Boot application that processes orders asynchronously using ThreadPool, CompletableFuture, BlockingQueue, and @Async.*

Concept Overview

Component	Threading Pattern	Why
Order submission	`@Async`	Non-blocking — API returns immediately
Parallel API calls	`CompletableFuture.allOf()`	Call payment + inventory simultaneously
Background queue	`BlockingQueue` + workers	Producer-consumer decoupling
Thread pool	`ThreadPoolTaskExecutor`	Bounded pool prevents exhaustion

Step 1: Configure Thread Pool

Concept: Always use bounded pools in production. Never create raw threads.

```java
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    @Bean("orderPool")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);                // 5 threads always alive
        executor.setMaxPoolSize(20);                // Scale to 20 under load
        executor.setQueueCapacity(100);             // Max 100 waiting tasks
        executor.setThreadNamePrefix("order-");     // Named for debugging
        executor.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy());  // Backpressure
        executor.initialize();
        return executor;
```

```
    }
}
```

Key Takeaways:

- `corePoolSize` = threads always alive (even idle)
- `queueCapacity` = MUST be bounded (prevent OOM)
- `CallerRunsPolicy` = slows producer when overwhelmed

Step 2: Build Async Order Service

Concept: `@Async` runs method in background thread. `CompletableFuture.allOf()` runs multiple calls in parallel.

```java
@Service
@RequiredArgsConstructor
public class OrderService {

    private final PaymentClient paymentClient;
    private final InventoryClient inventoryClient;

    @Async("orderPool")
    public CompletableFuture<OrderResult> processOrderAsync(OrderRequest
request) {
        log.info("Processing on thread: {}", Thread.currentThread().getName());

        // PARALLEL calls — 200ms total instead of 400ms sequential
        CompletableFuture<PaymentResult> payment = CompletableFuture
            .supplyAsync(() -> paymentClient.charge(request.getPayment()));
        CompletableFuture<InventoryResult> inventory = CompletableFuture
            .supplyAsync(() -> inventoryClient.reserve(request.getItems()));

        CompletableFuture.allOf(payment, inventory).join();

        return CompletableFuture.completedFuture(
            new OrderResult(payment.join().getTxnId(),
inventory.join().getReservationId()));
    }
}
```

Step 3: Create Producer-Consumer Queue

Concept: BlockingQueue decouples submission from processing. Producers add, consumers process independently.

```
@Service
public class OrderQueueService {

    private final BlockingQueue<OrderRequest> queue =
        new LinkedBlockingQueue<>(10_000);          // Bounded queue

    public boolean submit(OrderRequest order) {
        return queue.offer(order);                  // Non-blocking, returns
false if full
    }

    @PostConstruct
    public void startConsumers() {
        for (int i = 0; i < 5; i++) {
            Thread worker = new Thread(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    try {
                        OrderRequest order = queue.take();   // Blocks until
available
                        processOrder(order);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }, "order-worker-" + i);
            worker.setDaemon(true);
            worker.start();
        }
    }
}
```

Step 4: REST Controller

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {
```

```
    @Autowired private OrderService orderService;

    @PostMapping
    public ResponseEntity<String> submitOrder(@RequestBody OrderRequest
request) {
        orderService.processOrderAsync(request);    // Returns immediately
        return ResponseEntity.accepted().body("Order submitted");
    }
}
```

Key Takeaways

Pattern	Use When	Avoid When
`@Async`	Fire-and-forget (emails, logs)	Need result immediately
`CompletableFuture.allOf()`	Parallel independent calls	Sequential dependencies
`BlockingQueue` + consumers	High-throughput processing	Simple one-off tasks
`ThreadPoolTaskExecutor`	Always in production	Never create raw threads

Chapter 18: Interview Questions (60+)

BEGINNER LEVEL (Q1–Q15)

Q1. What is the difference between a process and a thread? A process is an independent program with its own memory space. A thread is a lightweight execution unit within a process that shares the heap with other threads but has its own stack. Thread creation is cheaper (~1MB) vs process creation (full memory allocation).

Q2. What are the two ways to create a thread in Java?

1. Extend `Thread` class and override `run()`. 2) Implement `Runnable` interface and pass to `Thread` constructor. Implementing `Runnable` is preferred because Java allows only single inheritance, and it separates the task from the thread mechanism.

Q3. What is the difference between `start()` and `run()`? `start()` creates a new OS thread and calls `run()` in that new thread. Calling `run()` directly executes it in the current thread — no new thread is created. Always use `start()`.

Q4. What are the thread states in Java? NEW (created, not started), RUNNABLE (executing or ready), BLOCKED (waiting for monitor lock), WAITING (waiting indefinitely for notification), TIMED_WAITING (waiting with timeout), TERMINATED (completed execution).

Q5. What is the difference between `Runnable` and `Callable`? `Runnable.run()` returns void and cannot throw checked exceptions. `Callable.call()` returns a value and can throw checked exceptions. Callable is used with `ExecutorService.submit()` and `Future`.

Q6. What is `Thread.sleep()`? Static method that pauses the current thread for the specified duration. The thread does NOT release any locks it holds. Throws `InterruptedException`.

Q7. What is `Thread.join()`? Causes the calling thread to wait until the target thread terminates. `t.join()` blocks until thread `t` finishes. Can specify timeout: `t.join(5000)`.

Q8. What is `Thread.yield()`? Hints to the scheduler that the current thread is willing to give up its CPU time. The scheduler may ignore this hint. Rarely used in practice.

Q9. What is a daemon thread? A background thread that doesn't prevent JVM shutdown. When all non-daemon threads finish, the JVM terminates (killing all daemon threads). Set via `thread.setDaemon(true)` before `start()`. Example: garbage collector.

Q10. What is the `volatile` keyword? Ensures visibility of variable changes across threads by reading/writing directly to main memory (not CPU cache). Does NOT provide atomicity for compound operations like `count++`.

Q11. What is thread safety? Code that functions correctly when accessed by multiple threads concurrently, without data corruption or unexpected behavior. Achieved via synchronization, immutable objects, thread-local storage, or atomic classes.

Q12. What is a race condition? When the outcome depends on the unpredictable order of thread execution accessing shared mutable state. Example: two threads incrementing the same counter simultaneously.

Q13. What is the `synchronized` keyword? Provides mutual exclusion — only one thread can execute a synchronized block/method at a time for a given lock object. Uses the object's intrinsic monitor lock.

Q14. What is the difference between synchronized method and synchronized block? Synchronized method locks `this` (or `Class` for static). Synchronized block can lock any object and only protects a specific code section — more granular, better performance.

Q15. What is `InterruptedException`? Thrown when a thread that is sleeping, waiting, or blocked is interrupted via `Thread.interrupt()`. The thread should handle this by either terminating or restoring the interrupt flag: `Thread.currentThread().interrupt()`.

INTERMEDIATE LEVEL (Q16–Q35)

Q16. What is a deadlock? How to prevent it? Two or more threads permanently blocked, each waiting for a lock held by the other. Prevention: consistent lock ordering, tryLock with timeout, avoid nested locks, use concurrent utilities.

Q17. What is the difference between `wait()`, `notify()`, and `notifyAll()`? `wait()` releases the monitor lock and suspends the thread until notified. `notify()` wakes one arbitrary waiting thread. `notifyAll()` wakes all waiting threads. Must be called inside `synchronized` block.

Q18. Why should `wait()` be called in a while loop? To guard against **spurious wakeups** — a thread can wake up without being notified. The while loop re-checks the condition:

```
while (!condition) { wait(); }
```

Q19. What is `ReentrantLock`? A lock implementation with capabilities beyond `synchronized`: tryLock (non-blocking), tryLock with timeout, interruptible locking, fairness policy, and multiple Condition objects.

Q20. What is the difference between `ReentrantLock` and `synchronized`?

Feature	synchronized	ReentrantLock
tryLock	No	Yes
Timeout	No	Yes
Interruptible	No	Yes
Fairness	No	Configurable
Conditions	1 (wait/notify)	Multiple
Requires finally	No (auto-release)	Yes (manual unlock)

Q21. What is `ReadWriteLock`? Allows multiple concurrent readers OR one exclusive writer. `ReentrantReadWriteLock` is the standard implementation. Improves throughput for read-heavy scenarios.

Q22. What is `ThreadLocal`? Provides per-thread copies of a variable. Each thread has its own isolated value. Used for request context, user sessions, database connections. Must call `remove()` in thread pools to avoid memory leaks.

Q23. What is the `ExecutorService`? A framework for managing thread pools. Decouples task submission from thread management. Provides `submit()`, `execute()`, `shutdown()`, `awaitTermination()`.

Q24. What are the types of thread pools? `FixedThreadPool` (fixed threads, unbounded queue), `CachedThreadPool` (elastic, 60s idle timeout), `SingleThreadExecutor` (sequential execution), `ScheduledThreadPool` (delayed/periodic tasks).

Q25. What is `ThreadPoolExecutor` and its parameters? Core pool size, max pool size, keep-alive time, work queue, thread factory, rejection handler. Tasks first fill core threads, then queue, then create up to max threads, then reject.

Q26. What is the difference between `submit()` and `execute()`? `execute(Runnable)` returns void, exceptions are uncaught. `submit(Callable/Runnable)` returns `Future`, exceptions are captured and thrown on `future.get()`.

Q27. What is `CompletableFuture`? Java 8 non-blocking async programming. Supports chaining (thenApply), composition (thenCompose), combining (thenCombine), error handling (exceptionally), and multi-future coordination (allOf, anyOf).

Q28. What is the difference between `thenApply` and `thenCompose`? `thenApply(f)`: transforms result synchronously, `f: T -> U`. `thenCompose(f)`: chains async operations, `f: T -> CompletableFuture<U>`. thenCompose is like flatMap, thenApply is like map.

Q29. What is `ConcurrentHashMap`? Thread-safe HashMap. Java 8+ uses CAS for empty bins, per-bin synchronized for occupied bins. No null keys/values. Provides atomic operations: `computeIfAbsent`, `merge`, `compute`.

Q30. What is `CopyOnWriteArrayList`? Thread-safe List where every write creates a new array copy. Reads are lock-free. Best for read-heavy, write-rare scenarios (event listeners, configuration). Iteration is snapshot-based (no CME).

Q31. What is `BlockingQueue`? A Queue that blocks `put()` when full and `take()` when empty. Implementations: `ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `SynchronousQueue` (handoff).

Q32. What is the `AtomicInteger` class? Provides atomic operations (incrementAndGet, compareAndSet) on an int variable using CAS hardware instructions. No locking needed:

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // Thread-safe, lock-free
```

Q33. What are `CountDownLatch` and `CyclicBarrier`? `CountDownLatch`: One-time barrier. Threads wait until count reaches zero. Cannot be reset. Use: wait for N services to start. `CyclicBarrier`: Reusable barrier. Threads wait for each other, then proceed together. Use: parallel iterative algorithms.

Q34. What is `Semaphore`? Controls access to a resource with a limited number of permits. `acquire()` gets a permit (blocks if none available), `release()` returns a permit. Use: connection pools, rate limiting.

Q35. What is the `Phaser` class? A reusable synchronization barrier supporting a variable number of parties. More flexible than CyclicBarrier. Supports multiple phases of execution.

ADVANCED LEVEL (Q36–Q50)

Q36. What is `StampedLock` and how does optimistic locking work? StampedLock provides three modes: write lock, read lock, and optimistic read. Optimistic read takes no lock — gets a stamp, reads data, then validates the stamp. If a write occurred, falls back to a full read lock. Best for read-dominated workloads.

Q37. Explain the Fork/Join Framework and work-stealing algorithm. ForkJoinPool uses divide-and-conquer: tasks split into subtasks (fork), execute in parallel, combine results (join). Work-stealing: idle threads steal tasks from busy threads' double-ended queues, balancing load automatically.

Q38. What happens internally when `CompletableFuture.supplyAsync()` is called? Creates a CompletableFuture, wraps the Supplier in an AsyncSupply task, submits to ForkJoinPool.commonPool() (or custom executor). When the task completes, the result is stored and dependent stages are triggered.

Q39. Why are virtual threads (Java 21) important? Virtual threads are lightweight (few KB vs 1MB for platform threads). Millions can exist simultaneously. They eliminate the need for reactive programming for IO-bound tasks by making blocking calls cheap. JVM manages scheduling on carrier threads.

Q40. What is the happens-before relationship in JMM? The Java Memory Model defines ordering guarantees:

- Monitor unlock happens-before subsequent lock of same monitor
- Volatile write happens-before subsequent read of same variable
- Thread.start() happens-before any action in the started thread
- Thread termination happens-before join() returns

Q41. What is false sharing and how to avoid it? When two threads modify different variables that reside on the same CPU cache line (64 bytes), both cores invalidate their cache lines. Fix: use `@Contended` annotation or pad variables with extra fields.

Q42. What is thread starvation? A thread cannot get CPU time because higher-priority threads dominate. Causes: unfair locks, priority scheduling, resource hogging. Prevention: fair locks, bounded pools, priority balancing.

Q43. Explain CAS (Compare-And-Swap) mechanism. An atomic CPU instruction: `CAS(memory, expected, new)` — if current value == expected, set to new. Returns success/failure. Used by AtomicInteger, ConcurrentHashMap, and lock-free algorithms. ABA problem is a known caveat.

Q44. What is the ABA problem? Thread reads value A, another thread changes A->B->A, first thread's CAS succeeds thinking nothing changed. Solution: `AtomicStampedReference` with a version stamp.

Q45. How does `ConcurrentHashMap` handle resizing? Uses `ForwardingNode` markers indicating migration. Multiple threads can help transfer buckets. Reads remain unblocked during resize. Each bin is migrated independently.

Q46. What is `LongAdder` and when to use it over `AtomicLong`? `LongAdder` distributes writes across multiple cells, reducing contention. Better for high-contention counters where frequent updates happen from many threads. Sum is computed by adding all cells.

Q47. How does `parallelStream()` work internally? Uses Spliterator to split the source into chunks. Each chunk is submitted to ForkJoinPool.commonPool(). The collector's combiner merges results. Thread count = available processors.

Q48. What are memory barriers/fences? CPU instructions that enforce ordering of memory operations. Prevent compiler/CPU reordering. Volatile reads/writes insert memory barriers. StoreStore, LoadLoad, LoadStore, StoreLoad barriers.

Q49. Explain the difference between `CountDownLatch`, `CyclicBarrier`, `Phaser`, and `Semaphore`.

Feature	CountDownLatch	CyclicBarrier	Phaser	Semaphore
Reusable	No	Yes	Yes	N/A
Parties	Fixed	Fixed	Dynamic	N/A
Action at barrier	No	Optional	Optional	N/A
Use case	Wait for N events	Sync N threads	Multi-phase sync	Permit-based access

Q50. How to monitor thread pool health in production?

```
ThreadPoolExecutor pool = (ThreadPoolExecutor) executor;
pool.getActiveCount();        // Currently executing threads
pool.getPoolSize();           // Current pool size
pool.getQueue().size();       // Tasks waiting in queue
pool.getCompletedTaskCount(); // Tasks completed
pool.getLargestPoolSize();    // Peak thread count
```

Expose via Spring Actuator or Micrometer metrics.

ARCHITECT LEVEL (Q51–Q65)

Q51. Design a high-throughput event processing system using Java concurrency. Use multiple BlockingQueues (one per partition) -> consumer threads per queue -> ConcurrentHashMap for deduplication -> CompletableFuture for async persistence -> separate executor for IO. Monitor queue depths and consumer lag.

Q52. How would you implement a rate limiter using Java concurrency primitives? Use `ConcurrentHashMap<String, Deque<Instant>>` for sliding window. Each request adds timestamp, removes expired ones, checks count <= limit. Use `Semaphore` for simpler fixed-rate limiting.

Q53. When would you choose reactive programming (WebFlux) over CompletableFuture? WebFlux for very high concurrency with many IO-bound operations (10K+ concurrent connections). CompletableFuture for moderate concurrency, blocking IO that's hard to rewrite, or when team expertise is in imperative style.

Q54. How to handle thread safety in a distributed caching layer? Local: ConcurrentHashMap with read/write separation. Cross-node: distributed locks (Redis/Zookeeper). Cache invalidation: pub/sub events. Consistency: cache-aside pattern with TTL. Bulk operations: StampedLock for read-heavy access.

Q55. Design a circuit breaker using thread-safe collections.

```
ConcurrentHashMap<String, AtomicInteger> failureCounts;
ConcurrentHashMap<String, Instant> lastFailureTime;
ConcurrentHashMap<String, CircuitState> circuitStates;
// States: CLOSED (normal) -> OPEN (failing) -> HALF_OPEN (testing)
```

Q56. How do you tune thread pools for a microservice handling both CPU and IO work? Create separate pools: CPU pool (size = cores), IO pool (size = cores x (1 + W/C)). Use CallerRunsPolicy for backpressure. Monitor with metrics. Adjust based on load testing results.

Q57. What is the impact of context switching on system performance? Each context switch costs 1-10us. At 10,000 threads with frequent switching, that's 10-100ms overhead per second. Solution: right-size thread pools, use non-blocking IO, virtual threads (Java 21+).

Q58. How to implement graceful shutdown in a multithreaded Spring Boot application?

```
@PreDestroy
public void shutdown() {
    executor.shutdown(); // Stop accepting new tasks
    try {
        if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
            executor.shutdownNow(); // Force shutdown
```

```
            executor.awaitTermination(10, TimeUnit.SECONDS);
        }
    } catch (InterruptedException e) {
        executor.shutdownNow();
    }
}
```

Q59. How to debug a production deadlock?

1. Get thread dump: `jstack <PID>` or JMX ThreadMXBean. 2) Look for "Found N deadlocks" in output. 3) Identify the lock ordering issue. 4) Fix with consistent ordering or tryLock. 5) Add deadlock detection monitoring.

Q60. What are the trade-offs between lock-based and lock-free concurrency? Lock-based: simpler code, works everywhere, but contention reduces throughput. Lock-free (CAS): higher throughput under contention, no deadlocks, but harder to implement correctly, ABA problem, spin-wait overhead.

Q61. How to implement a thread-safe connection pool? Use a BlockingQueue of connections. `take()` acquires (blocks if none available). `put()` returns. MaxPoolSize = queue capacity. Use Semaphore for bounded permits. Add health check and idle timeout eviction.

Q62. Compare `synchronized`, `ReentrantLock`, `StampedLock`, and `ConcurrentHashMap` for a shared cache.

- Low contention: `synchronized` (simplest)
- Medium contention, need tryLock: `ReentrantLock`
- Read-heavy: `StampedLock` (optimistic reads) or `ReentrantReadWriteLock`
- Key-value store: `ConcurrentHashMap` (built-in thread safety, atomic ops)

Q63. How do virtual threads (Java 21) change concurrency architecture? No more thread pool sizing games. One virtual thread per request. Blocking IO becomes cheap. Can have millions of concurrent VTs. No need for reactive frameworks for IO-bound work. Platform threads still better for CPU-bound work.

Q64. How would you test multithreaded code? Use `CountDownLatch` to synchronize thread starts. Use `CyclicBarrier` for stress testing. Use `AtomicReference` to capture exceptions. Use jcstress for correctness testing. Use JMH for performance benchmarks.

Q65. How to prevent memory leaks in concurrent applications? Clear ThreadLocal variables after use. Weak references in caches (WeakHashMap). Bounded collections with eviction. Properly shut down executors. Monitor with heap dumps and profilers.

SCENARIO-BASED (Q66–Q80)

Q66. A production Spring Boot app becomes unresponsive under load, but CPU usage is only 5%. What's your debugging approach? Low CPU + unresponsive = threads waiting on something (locks, IO, connections). Steps: 1) Take 3 thread dumps 10s apart. 2) Count BLOCKED and WAITING threads. 3) If many BLOCKED -> lock contention (find the lock). 4) If many WAITING on connection pool -> pool exhaustion (increase pool or add timeouts). 5) Check HikariCP metrics for connection leaks.

Q67. Your team wants to use parallelStream() for batch database inserts. What's your advice? DON'T do it. parallelStream() uses ForkJoinPool.commonPool() which is shared across the entire JVM. Blocking DB calls will starve ALL other parallel streams. Instead, use a dedicated ExecutorService with CompletableFuture, or batch the inserts into chunks and use JDBC batch operations.

Q68. You have a microservice making 5 REST calls to downstream services. Currently sequential (2.5s total). How to optimize? Use CompletableFuture with a custom IO executor: `CompletableFuture.allOf(call1, call2, call3, call4, call5)` runs all in parallel. Total time ~~= max of individual calls~~ (500ms instead of 2.5s). Always use `completeOnTimeout()` for each call. If one fails, use `exceptionally()` with fallback.

Q69. What would happen if you used `Executors.newCachedThreadPool()` in a HTTP API handler? Under load spike, each request creates a new thread (since cached threads are reused only if idle <=60s). A burst of 50,000 requests -> 50,000 threads -> ~50GB memory -> OutOfMemoryError. Always use `newFixedThreadPool()` or `ThreadPoolExecutor` with bounded queue.

Q70. You observe intermittent NullPointerException in production, but the code looks correct in single-threaded analysis. What could cause this? Race condition on a shared reference: Thread A reads the object, Thread B sets it to null between read and use. Or: instruction reordering — an object reference published before the constructor finishes (broken double-checked locking without volatile). Fix: make the reference volatile, use AtomicReference, or use proper synchronization.

Q71. A thread dump shows 150 threads in BLOCKED state all waiting on `OrderService.processOrder()`. What's happening? The `processOrder()` method is likely synchronized on a single lock (or uses a synchronized method), creating a bottleneck. Solutions: 1) Reduce synchronized scope. 2) Use ConcurrentHashMap instead of synchronized HashMap. 3) Use read-write lock if mostly reads. 4) Consider lock-free alternatives.

Q72. Your application has 200 Tomcat threads and a HikariCP pool of 10. What happens when all 200 threads need DB access? 190 threads will wait for a connection (only 10 available). If `connectionTimeout` is not set, they wait forever -> request timeouts cascade. Fix: 1) Size connection pool appropriately (but more connections = more DB load). 2) Set `connection-timeout=5000` to fail fast. 3) Use `leak-detection-threshold` to catch leaks.

Q73. When should you use virtual threads vs reactive programming vs traditional thread pools? Virtual threads (Java 21+): Simple blocking IO code, need to handle many concurrent requests, team prefers imperative style. Reactive (WebFlux): Need full backpressure, streaming data pipelines, already have reactive team. Traditional pools: CPU-bound work, legacy systems, need fine-grained control over concurrency.

Q74. A CountDownLatch with count 5 hangs forever in production. What happened? One of the 5 tasks threw an uncaught exception before calling `countDown()`. The latch stays at count > 0 forever. Fix: Always call `countDown()` in a finally block, OR use `latch.await(30, TimeUnit.SECONDS)` with a timeout.

Q75. You need to implement a cache that refreshes every 5 minutes. How to make it thread-safe? Use `AtomicReference<CacheData>` with a scheduled task. The scheduler calls `cacheRef.set(loadFreshData())` atomically. All readers call `cacheRef.get()` (lock-free). Alternatively, use `ConcurrentHashMap.computeIfAbsent()` with TTL-based expiry, or use Caffeine cache (`LoadingCache`).

Q76. What happens if you use `synchronized` keyword inside virtual threads? Virtual threads PIN to the carrier (platform) thread when inside synchronized blocks. The carrier cannot be reused until the synchronized block completes. Under load, this can exhaust all carrier threads. Fix: Replace synchronized with ReentrantLock, which virtual threads can unmount from.

Q77. How would you design a thread-safe audit log system that doesn't slow down the main request? Use a producer-consumer pattern: Main thread submits audit events to a `LinkedBlockingQueue` (non-blocking `offer()`). Background consumers `take()` from queue and batch-insert to DB every 5 seconds. Use `ConcurrentLinkedQueue` if you can't afford any blocking on the producer side.

Q78. Your thread pool monitoring shows queue size keeps growing but completed task count is stable. What's wrong? Producers are faster than consumers. Either: 1) Tasks are slow (check for blocking calls or resource contention). 2) Pool is undersized for the workload. 3) Tasks are deadlocking. Take a thread dump to see what worker threads are doing. Increase pool size or optimize task execution.

Q79. How do you test that a class is thread-safe in unit tests?

```java
@RepeatedTest(10)  // Run 10 times to catch intermittent failures
void testConcurrentAccess() throws InterruptedException {
    ThreadSafeList<String> list = new ThreadSafeList<>();
    int threads = 100;
    CountDownLatch ready = new CountDownLatch(threads);
    CountDownLatch go = new CountDownLatch(1);
```

```java
    CountDownLatch done = new CountDownLatch(threads);

    for (int i = 0; i < threads; i++) {
        final int idx = i;
        new Thread(() -> {
            ready.countDown();
            try { go.await(); } catch (InterruptedException e) {}
            list.add("item-" + idx);
            done.countDown();
        }).start();
    }

    ready.await();  // All threads ready
    go.countDown(); // Release all at once
    done.await();   // Wait for all to finish

    assertEquals(threads, list.size());  // Must always be 100
}
```

Q80. You're reviewing a PR that uses `Collections.synchronizedMap(new HashMap<>())`. What feedback would you give?

1. Replace with `ConcurrentHashMap` for better performance (per-bin locking vs whole-map locking). 2) Even with synchronizedMap, compound operations like `if(!map.containsKey(k)) map.put(k,v)` are NOT atomic — still needs external synchronization or `computeIfAbsent()`. 3) Iteration requires manual synchronization with synchronizedMap but not with ConcurrentHashMap.

Chapter 19: Coding Problems (20 Exercises)

Exercise 1: Producer-Consumer with BlockingQueue

```
public class ProducerConsumer {
    private final BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);

    public void produce() throws InterruptedException {
        for (int i = 0; i < 100; i++) {
            queue.put(i);
            System.out.println("Produced: " + i);
        }
        queue.put(-1); // Poison pill
    }

    public void consume() throws InterruptedException {
        while (true) {
            int val = queue.take();
            if (val == -1) break; // Poison pill
            System.out.println("Consumed: " + val);
            Thread.sleep(100);
        }
    }

    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();
        new Thread(() -> { try { pc.produce(); } catch (InterruptedException e)
{} }).start();
        new Thread(() -> { try { pc.consume(); } catch (InterruptedException e)
{} }).start();
    }
}
```

Exercise 2: Thread-Safe Singleton (Enum-Based)

```
public enum ConfigManager {
    INSTANCE;

    private final Map<String, String> config = new ConcurrentHashMap<>();
```

```
    ConfigManager() {
        config.put("db.host", "localhost");
        config.put("db.port", "5432");
    }

    public String get(String key) {
        return config.get(key);
    }

    public void set(String key, String value) {
        config.put(key, value);
    }
}
```

Exercise 3: Print Numbers 1-100 Using Two Alternating Threads

```
public class AlternateThreads {
    private final Object lock = new Object();
    private volatile boolean oddTurn = true;

    public void printOdd() {
        for (int i = 1; i <= 99; i += 2) {
            synchronized (lock) {
                while (!oddTurn) {
                    try { lock.wait(); } catch (InterruptedException e) {}
                }
                System.out.println(Thread.currentThread().getName() + ": " +
i);
                oddTurn = false;
                lock.notify();
            }
        }
    }

    public void printEven() {
        for (int i = 2; i <= 100; i += 2) {
            synchronized (lock) {
                while (oddTurn) {
                    try { lock.wait(); } catch (InterruptedException e) {}
                }
                System.out.println(Thread.currentThread().getName() + ": " +
i);
                oddTurn = true;
```

```
                lock.notify();
            }
        }
    }

    public static void main(String[] args) {
        AlternateThreads at = new AlternateThreads();
        new Thread(at::printOdd, "OddThread").start();
        new Thread(at::printEven, "EvenThread").start();
    }
}
```

Exercise 4: Thread-Safe Counter with AtomicInteger

```
public class ThreadSafeCounter {
    private final AtomicInteger count = new AtomicInteger(0);

    public void increment() { count.incrementAndGet(); }
    public void decrement() { count.decrementAndGet(); }
    public int get() { return count.get(); }

    public static void main(String[] args) throws InterruptedException {
        ThreadSafeCounter counter = new ThreadSafeCounter();
        int threads = 100;
        CountDownLatch latch = new CountDownLatch(threads);

        for (int i = 0; i < threads; i++) {
            new Thread(() -> {
                for (int j = 0; j < 1000; j++) counter.increment();
                latch.countDown();
            }).start();
        }

        latch.await();
        System.out.println("Final count: " + counter.get()); // Always 100,000
    }
}
```

Exercise 5: Parallel API Aggregation with CompletableFuture

```java
public class ParallelApiCall {

    public static Map<String, Object> aggregateData(Long userId) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        CompletableFuture<String> profileFuture = CompletableFuture
            .supplyAsync(() -> fetchProfile(userId), executor);
        CompletableFuture<List<String>> ordersFuture = CompletableFuture
            .supplyAsync(() -> fetchOrders(userId), executor);
        CompletableFuture<Double> balanceFuture = CompletableFuture
            .supplyAsync(() -> fetchBalance(userId), executor);

        CompletableFuture.allOf(profileFuture, ordersFuture,
balanceFuture).join();

        Map<String, Object> result = new HashMap<>();
        result.put("profile", profileFuture.join());
        result.put("orders", ordersFuture.join());
        result.put("balance", balanceFuture.join());

        executor.shutdown();
        return result;
    }

    private static String fetchProfile(Long id) {
        sleep(200); return "User-" + id;
    }
    private static List<String> fetchOrders(Long id) {
        sleep(300); return List.of("ORD-1", "ORD-2");
    }
    private static Double fetchBalance(Long id) {
        sleep(150); return 999.99;
    }
    private static void sleep(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) {}
    }
}
```

Exercise 6: Dining Philosophers (Deadlock Avoidance)

```java
public class DiningPhilosophers {
    private final ReentrantLock[] forks;
```

```java
    public DiningPhilosophers(int n) {
        forks = new ReentrantLock[n];
        for (int i = 0; i < n; i++) forks[i] = new ReentrantLock();
    }

    public void eat(int philosopher) {
        int left = philosopher;
        int right = (philosopher + 1) % forks.length;

        // Lock ordering: always pick lower-numbered fork first
        int first = Math.min(left, right);
        int second = Math.max(left, right);

        forks[first].lock();
        try {
            forks[second].lock();
            try {
                System.out.println("Philosopher " + philosopher + " is
eating");
                Thread.sleep(100);
            } catch (InterruptedException e) {
            } finally {
                forks[second].unlock();
            }
        } finally {
            forks[first].unlock();
        }
    }
}
```

Exercise 7: Read-Write Cache

```java
public class ReadWriteCache<K, V> {
    private final Map<K, V> cache = new HashMap<>();
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();

    public V get(K key) {
        rwLock.readLock().lock();
        try { return cache.get(key); }
        finally { rwLock.readLock().unlock(); }
    }

    public void put(K key, V value) {
```

```
        rwLock.writeLock().lock();
        try { cache.put(key, value); }
        finally { rwLock.writeLock().unlock(); }
    }

    public int size() {
        rwLock.readLock().lock();
        try { return cache.size(); }
        finally { rwLock.readLock().unlock(); }
    }
}
```

Exercise 8: Countdown Start (Wait for All Threads Ready)

```
public class RaceSimulation {
    public static void main(String[] args) throws InterruptedException {
        int runners = 5;
        CountDownLatch readyLatch = new CountDownLatch(runners);
        CountDownLatch startLatch = new CountDownLatch(1);

        for (int i = 0; i < runners; i++) {
            final int id = i;
            new Thread(() -> {
                System.out.println("Runner " + id + " ready");
                readyLatch.countDown();
                try {
                    startLatch.await(); // All wait for start signal
                    System.out.println("Runner " + id + " started at " +
System.nanoTime());
                } catch (InterruptedException e) {}
            }).start();
        }

        readyLatch.await(); // Wait for all runners to be ready
        System.out.println("GO!");
        startLatch.countDown(); // Start all at once
    }
}
```

Exercise 9: Bounded Thread-Safe Queue

```java
public class BoundedQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;
    private final Semaphore empty;
    private final Semaphore full;
    private final Lock lock = new ReentrantLock();

    public BoundedQueue(int capacity) {
        this.capacity = capacity;
        this.empty = new Semaphore(capacity);
        this.full = new Semaphore(0);
    }

    public void put(T item) throws InterruptedException {
        empty.acquire();
        lock.lock();
        try { queue.offer(item); }
        finally { lock.unlock(); }
        full.release();
    }

    public T take() throws InterruptedException {
        full.acquire();
        lock.lock();
        T item;
        try { item = queue.poll(); }
        finally { lock.unlock(); }
        empty.release();
        return item;
    }
}
```

Exercise 10: Parallel Word Frequency Counter

```java
public class ParallelWordCounter {

    public static Map<String, Long> count(List<String> documents) {
        return documents.parallelStream()
            .flatMap(doc -> Arrays.stream(doc.toLowerCase().split("\\W+")))
            .filter(w -> !w.isEmpty())
            .collect(Collectors.groupingBy(Function.identity(),
                ConcurrentHashMap::new, Collectors.counting()));
```

```
    }
}
```

Exercise 11: Scheduled Task Executor

```
public class TaskScheduler {
    private final ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(3);

    public ScheduledFuture<?> scheduleOnce(Runnable task, long delay, TimeUnit
unit) {
        return scheduler.schedule(task, delay, unit);
    }

    public ScheduledFuture<?> scheduleRepeating(Runnable task, long period,
TimeUnit unit) {
        return scheduler.scheduleAtFixedRate(task, 0, period, unit);
    }

    public void shutdown() {
        scheduler.shutdown();
    }
}
```

Exercise 12: Thread Barrier Example

```
public class MatrixProcessor {
    public static void processInPhases(int workers) throws Exception {
        CyclicBarrier barrier = new CyclicBarrier(workers, () ->
            System.out.println("--- Phase complete, all workers synchronized --
-"));

        for (int i = 0; i < workers; i++) {
            final int id = i;
            new Thread(() -> {
                try {
                    System.out.println("Worker " + id + " phase 1");
                    barrier.await(); // Wait for all to complete phase 1
                    System.out.println("Worker " + id + " phase 2");
                    barrier.await(); // Wait for all to complete phase 2
```

```
                } catch (Exception e) {}
            }).start();
        }
    }
}
```

Exercise 13: Future Timeout Handler

```
public class TimeoutHandler {

    public static <T> T executeWithTimeout(Callable<T> task, long timeout,
                                           TimeUnit unit, T fallback) {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<T> future = executor.submit(task);
        try {
            return future.get(timeout, unit);
        } catch (TimeoutException e) {
            future.cancel(true);
            return fallback;
        } catch (Exception e) {
            return fallback;
        } finally {
            executor.shutdown();
        }
    }
}
```

Exercise 14: Thread-Safe Stack

```
public class ConcurrentStack<T> {
    private final Deque<T> stack = new ArrayDeque<>();
    private final ReentrantLock lock = new ReentrantLock();

    public void push(T item) {
        lock.lock();
        try { stack.push(item); }
        finally { lock.unlock(); }
    }

    public T pop() {
```

```
        lock.lock();
        try { return stack.isEmpty() ? null : stack.pop(); }
        finally { lock.unlock(); }
    }

    public T peek() {
        lock.lock();
        try { return stack.isEmpty() ? null : stack.peek(); }
        finally { lock.unlock(); }
    }
}
```

Exercise 15: Parallel File Search

```
public class ParallelFileSearch {

    public static List<Path> search(Path root, String pattern) throws Exception
{
        ForkJoinPool pool = new ForkJoinPool();
        return pool.invoke(new FileSearchTask(root, pattern));
    }

    static class FileSearchTask extends RecursiveTask<List<Path>> {
        private final Path dir;
        private final String pattern;

        FileSearchTask(Path dir, String pattern) {
            this.dir = dir;
            this.pattern = pattern;
        }

        @Override
        protected List<Path> compute() {
            List<Path> results = new ArrayList<>();
            List<FileSearchTask> subTasks = new ArrayList<>();

            try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir))
{
                for (Path path : stream) {
                    if (Files.isDirectory(path)) {
                        FileSearchTask subTask = new FileSearchTask(path,
pattern);
                        subTasks.add(subTask);
                        subTask.fork();
```

```
                    } else if (path.getFileName().toString().contains(pattern))
{
                        results.add(path);
                    }
                }
            } catch (IOException e) {}

            for (FileSearchTask task : subTasks) {
                results.addAll(task.join());
            }
            return results;
        }
    }
}
```

Exercise 16: Async Pipeline with Error Handling

```
public class AsyncPipeline {

    public CompletableFuture<OrderResult> processOrder(OrderRequest request) {
        return CompletableFuture.supplyAsync(() -> validateOrder(request))
            .thenApplyAsync(validated -> calculatePrice(validated))
            .thenComposeAsync(priced ->
                CompletableFuture.supplyAsync(() -> processPayment(priced)))
            .thenApplyAsync(paid -> sendConfirmation(paid))
            .handle((result, ex) -> {
                if (ex != null) {
                    logError(ex);
                    return OrderResult.failed(ex.getMessage());
                }
                return result;
            });
    }
}
```

Exercise 17: Multi-Producer Multi-Consumer

```
public class MultiProducerConsumer {

    public static void main(String[] args) {
```

```java
        BlockingQueue<String> queue = new LinkedBlockingQueue<>(50);
        ExecutorService producers = Executors.newFixedThreadPool(3);
        ExecutorService consumers = Executors.newFixedThreadPool(5);

        // 3 producers
        for (int i = 0; i < 3; i++) {
            final int id = i;
            producers.submit(() -> {
                for (int j = 0; j < 100; j++) {
                    try {
                        queue.put("P" + id + "-Item" + j);
                    } catch (InterruptedException e) { break; }
                }
            });
        }

        // 5 consumers
        for (int i = 0; i < 5; i++) {
            final int id = i;
            consumers.submit(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    try {
                        String item = queue.poll(2, TimeUnit.SECONDS);
                        if (item == null) break;
                        System.out.println("C" + id + " consumed: " + item);
                    } catch (InterruptedException e) { break; }
                }
            });
        }

        producers.shutdown();
        consumers.shutdown();
    }
}
```

Exercise 18: Deadlock Detection Utility

```java
public class DeadlockDetector {

    public static void startMonitoring(long interval) {
        ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();
        scheduler.scheduleAtFixedRate(() -> {
            ThreadMXBean bean = ManagementFactory.getThreadMXBean();
```

```
            long[] deadlocked = bean.findDeadlockedThreads();
            if (deadlocked != null) {
                System.err.println("DEADLOCK DETECTED!");
                ThreadInfo[] infos = bean.getThreadInfo(deadlocked, true,
true);
                for (ThreadInfo info : infos) {
                    System.err.println(info);
                }
            }
        }, 0, interval, TimeUnit.SECONDS);
    }
}
```

Exercise 19: Rate Limiter

```
public class SlidingWindowRateLimiter {
    private final ConcurrentHashMap<String, Deque<Long>> windows = new
ConcurrentHashMap<>();
    private final int maxRequests;
    private final long windowMs;

    public SlidingWindowRateLimiter(int maxRequests, Duration window) {
        this.maxRequests = maxRequests;
        this.windowMs = window.toMillis();
    }

    public synchronized boolean isAllowed(String clientId) {
        long now = System.currentTimeMillis();
        Deque<Long> timestamps = windows.computeIfAbsent(clientId, k -> new
ArrayDeque<>());

        while (!timestamps.isEmpty() && now - timestamps.peekFirst() >
windowMs) {
            timestamps.pollFirst();
        }

        if (timestamps.size() < maxRequests) {
            timestamps.addLast(now);
            return true;
        }
        return false;
```

```
    }
}
```

Exercise 20: Thread Pool Monitor

```
public class ThreadPoolMonitor {

    public static void monitor(ThreadPoolExecutor pool, String name, long
intervalSec) {
        ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();
        scheduler.scheduleAtFixedRate(() -> {
            System.out.printf("[%s] Active: %d | Pool: %d | Queue: %d |
Completed: %d%n",
                name,
                pool.getActiveCount(),
                pool.getPoolSize(),
                pool.getQueue().size(),
                pool.getCompletedTaskCount()
            );
        }, 0, intervalSec, TimeUnit.SECONDS);
    }
}
```

Chapter 20: Best Practices

20.1 Enterprise-Level Multithreading Best Practices

1. Always Use Thread Pools

```
// [N] NEVER create raw threads in production
new Thread(() -> processOrder(order)).start();

// [Y] ALWAYS use managed thread pools
executor.submit(() -> processOrder(order));
```

2. Always Name Your Threads

```
ThreadFactory factory = r -> {
    Thread t = new Thread(r, "order-processor-" + counter.incrementAndGet());
    t.setDaemon(true);
    return t;
};
```

3. Always Use Bounded Queues

```
// [N] Unbounded queue = OOM risk
new LinkedBlockingQueue<>()

// [Y] Bounded queue with rejection policy
new LinkedBlockingQueue<>(500)
```

4. Separate Pools for CPU and IO

```
@Bean("cpuPool") // Size = cores
@Bean("ioPool")  // Size = cores x (1 + wait/compute)
```

5. Always Clean Up ThreadLocal

```java
try {
    ThreadLocalContext.set(data);
    // ... business logic
} finally {
    ThreadLocalContext.remove(); // Prevent memory leaks!
}
```

6. Handle InterruptedException Properly

```java
// [N] Swallowing interrupt
catch (InterruptedException e) { /* ignore */ }

// [Y] Restore interrupt flag
catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // Restore flag
    throw new RuntimeException("Thread was interrupted", e);
}
```

7. Prefer Higher-Level Concurrency Utilities

```
Use                      Instead of
--------------------     ------------------
ExecutorService          new Thread()
CompletableFuture        Future + get()
BlockingQueue            wait/notify
ConcurrentHashMap        Collections.synchronizedMap()
AtomicInteger            synchronized counter
ReentrantLock            synchronized (when advanced features needed)
```

8. Always Set Timeouts

```java
future.get(5, TimeUnit.SECONDS);      // Don't block forever
lock.tryLock(2, TimeUnit.SECONDS);     // Don't wait forever
executor.awaitTermination(30, TimeUnit.SECONDS);
```

9. Graceful Shutdown

```java
@PreDestroy
public void shutdown() {
    executor.shutdown();
    if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
        List<Runnable> dropped = executor.shutdownNow();
        log.warn("Dropped {} tasks on shutdown", dropped.size());
    }
}
```

10. Monitor Everything

```java
// Expose thread pool metrics
@Bean
public MeterBinder threadPoolMetrics(ThreadPoolExecutor pool) {
    return registry -> {
        Gauge.builder("thread.pool.active", pool,
ThreadPoolExecutor::getActiveCount)
             .register(registry);
        Gauge.builder("thread.pool.queue.size", pool, e -> e.getQueue().size())
             .register(registry);
    };
}
```

End of Guide

Document Version: 1.0
Last Updated: March 2026
Topics Covered: 20 Chapters, 80+ Interview Questions, 20 Coding Exercises

Senior Engineer Decision Framework

Concurrency Approach Selection

```
Step 1: Do you actually NEED multithreading?
  +- Is the current approach fast enough? -> Don't add complexity
  +- Is the bottleneck IO or CPU?
  |   +- IO: Can you batch requests? Use async IO? Increase pool size?
```

```
  |   +- CPU: Can you optimize the algorithm first?
  +- Is the data truly shared? Can you partition it?

Step 2: Choose the right concurrency model
  +- Simple background task -> @Async or CompletableFuture.runAsync()
  +- Parallel independent calls -> CompletableFuture.allOf()
  +- Producer-consumer pipeline -> BlockingQueue + dedicated consumers
  +- CPU-bound batch processing -> parallel streams or ForkJoinPool
  +- High-concurrency IO (10K+) -> Virtual threads (Java 21) or WebFlux
  +- Real-time streaming -> Kafka/RxJava/Project Reactor

Step 3: Choose the right synchronization
  +- Can data be immutable? -> No synchronization needed (BEST)
  +- Can data be thread-local? -> ThreadLocal or ScopedValue
  +- Simple counter? -> AtomicInteger or LongAdder
  +- Key-value access? -> ConcurrentHashMap (always)
  +- Read-heavy shared state? -> StampedLock or ReadWriteLock
  +- Complex multi-field update? -> synchronized or ReentrantLock
```

Production Readiness Checklist for Concurrent Systems

```
Before deploying a multithreaded system to production:
-----------------------------------------------------

Thread Pool Configuration:
  [ ] All pools use bounded queues
  [ ] Pool sizes are based on workload type (CPU vs IO)
  [ ] Rejection policies are set (CallerRunsPolicy recommended)
  [ ] Threads are named (for debugging)
  [ ] Graceful shutdown is implemented (@PreDestroy)

Safety:
  [ ] No raw Thread creation (use ExecutorService)
  [ ] ThreadLocal values are removed after use
  [ ] InterruptedException is handled properly
  [ ] No synchronized blocks around IO/network calls
  [ ] Lock ordering is consistent where nested locks exist

Monitoring:
  [ ] Thread pool metrics are exposed (active, queue size, rejected)
  [ ] Connection pool metrics are monitored
  [ ] Alerting is set for pool exhaustion
  [ ] Thread dump automation is in place for incidents

Testing:
  [ ] Concurrent unit tests with CountDownLatch/CyclicBarrier
```

```
  [ ] Load tests with realistic concurrency levels
  [ ] Stress tests to verify rejection and graceful degradation
  [ ] Thread dump analysis performed under load
```